

Foundations of Artificial Intelligence

Chapter 04: **Beyond Classical Search**

Roberto Sebastiani

DISI, Università di Trento, Italy – `roberto.sebastiani@unitn.it`

`https://disi.unitn.it/rseba/DIDATTICA/fai\_2026/`

Teaching assistant: **Nicola Debole**, `nicola.debole@unitn.it`,

M.S. Course “Artificial Intelligence Systems”, academic year 2026-2027

Last update: Friday 25th September, 2026, 09:24

Copyright notice: Most examples and images displayed in the slides of this course are taken from [Russell & Norwig, “Artificial Intelligence, a Modern Approach”, 3rd and 4th ed., Pearson], including explicitly figures from the above-mentioned books, so that their copyright is detained by the authors. A few other material (text, figures, examples) is authored by (in alphabetical order): Pieter Abbeel, Bonnie J. Dorr, Anca Dragan, Dan Klein, Nikita Kitaev, Tom Lenaerts, Michela Milano, Paolo Morettin, Dana Nau, Maria Simi, who detain its copyright.

These slides cannot be displayed in public without the permission of the author.

Generalities

- So far we address a single category of problems:
 - 1 observable,
 - 2 deterministic,
 - 3 with known environment,
 - 4 s.t. the solution is a sequence of actions.
- What happens when these assumptions are relaxed?
- In order we will:
 - release condition 4 $\implies$ local search
 - release condition 2 $\implies$ search with non-deterministic actions
 - release condition 1 $\implies$ search with no observability or with partial observability
 - release condition 3 $\implies$ online search

Generalities

- So far we address a single category of problems:
 - 1 observable,
 - 2 deterministic,
 - 3 with known environment,
 - 4 s.t. the solution is a sequence of actions.
- What happens when these assumptions are relaxed?
- In order we will:
 - release condition 4 $\implies$ local search
 - release condition 2 $\implies$ search with non-deterministic actions
 - release condition 1 $\implies$ search with no observability or with partial observability
 - release condition 3 $\implies$ online search

- 1 Local Search and Optimization
 - General Ideas
 - Hill-Climbing
 - Simulated Annealing
 - Local Beam Search & Genetic Algorithms
- 2 Search with Nondeterministic Actions
- 3 Search with Partial or No Observations (Deterministic/Nondeterministic Actions)
 - Search with No Observations
 - Search with Partial Observations
- 4 Online Search (aka Exploration)

- 1 Local Search and Optimization
 - General Ideas
 - Hill-Climbing
 - Simulated Annealing
 - Local Beam Search & Genetic Algorithms
- 2 Search with Nondeterministic Actions
- 3 Search with Partial or No Observations (Deterministic/Nondeterministic Actions)
 - Search with No Observations
 - Search with Partial Observations
- 4 Online Search (aka Exploration)

- 1 Local Search and Optimization
 - General Ideas
 - Hill-Climbing
 - Simulated Annealing
 - Local Beam Search & Genetic Algorithms
- 2 Search with Nondeterministic Actions
- 3 Search with Partial or No Observations (Deterministic/Nondeterministic Actions)
 - Search with No Observations
 - Search with Partial Observations
- 4 Online Search (aka Exploration)

Recall: Generalities

- So far we addressed a single category of problems:
 - ① observable,
 - ② deterministic,
 - ③ with known environment,
 - ④ s.t. the solution is a sequence of actions.
- What happens when these assumptions are relaxed?
- In order we will:
 - release condition 4 $\implies$ local search
 - release condition 2 $\implies$ search with non-deterministic actions
 - release condition 1 $\implies$ search with no observability or with partial observability
 - release condition 3 $\implies$ online search

General Ideas

- Search techniques: **systematic exploration of search space**
 - solution to problem: **the path to the goal state**
 - ex: **8-puzzle**
- With many problems, **the path to goal is irrelevant**
 - **goals expressed as conditions**, not as explicit list of goal states
 - solution to problem: only **the goal state** itself
 - ex: **N-queens**
 - many important applications:
integrated-circuit design, factory-floor layout, job-shop scheduling, automatic programming, telecommunications network optimization, vehicle routing, portfolio management...
- The state space is a set of “complete” configurations
 - **decision problems**: find goal configuration satisfying constraints/rules (ex: N-queens)
 - **optimization problems**: find **optimal** configurations
(ex: Travelling Salesperson Problem TSP)
- If so, we can use **iterative-improvement algorithms** (in particular **local search algorithms**):
 - **keep a single “current” state**, try to improve it

General Ideas

- Search techniques: **systematic exploration of search space**
 - solution to problem: **the path to the goal state**
 - ex: **8-puzzle**
- With many problems, **the path to goal is irrelevant**
 - **goals expressed as conditions, not as explicit list of goal states**
 - solution to problem: only **the goal state** itself
 - ex: **N-queens**
 - many important applications:
integrated-circuit design, factory-floor layout, job-shop scheduling, automatic programming, telecommunications network optimization, vehicle routing, portfolio management...
- The state space is a set of “complete” configurations
 - **decision problems**: find goal configuration satisfying constraints/rules (ex: N-queens)
 - **optimization problems**: find **optimal** configurations
(ex: Travelling Salesperson Problem TSP)
- If so, we can use **iterative-improvement algorithms** (in particular **local search algorithms**):
 - **keep a single “current” state, try to improve it**

General Ideas

- Search techniques: **systematic exploration of search space**
 - solution to problem: **the path to the goal state**
 - ex: **8-puzzle**
- With many problems, **the path to goal is irrelevant**
 - **goals expressed as conditions, not as explicit list of goal states**
 - solution to problem: only **the goal state** itself
 - ex: **N-queens**
 - many important applications:
integrated-circuit design, factory-floor layout, job-shop scheduling, automatic programming, telecommunications network optimization, vehicle routing, portfolio management...
- The state space is a set of “complete” configurations
 - **decision problems**: find goal configuration satisfying constraints/rules (ex: **N-queens**)
 - **optimization problems**: find **optimal** configurations
(ex: **Travelling Salesperson Problem TSP**)
- If so, we can use **iterative-improvement algorithms** (in particular **local search algorithms**):
 - keep a single “current” state, try to improve it

General Ideas

- Search techniques: **systematic exploration of search space**
 - solution to problem: **the path to the goal state**
 - ex: **8-puzzle**
- With many problems, **the path to goal is irrelevant**
 - **goals expressed as conditions, not as explicit list of goal states**
 - solution to problem: only **the goal state** itself
 - ex: **N-queens**
 - many important applications:
integrated-circuit design, factory-floor layout, job-shop scheduling, automatic programming, telecommunications network optimization, vehicle routing, portfolio management...
- The state space is a set of “complete” configurations
 - **decision problems**: find goal configuration satisfying constraints/rules (ex: **N-queens**)
 - **optimization problems**: find **optimal** configurations
(ex: **Travelling Salesperson Problem TSP**)
- If so, we can use **iterative-improvement algorithms** (in particular **local search algorithms**):
 - **keep a single “current” state, try to improve it**

Local Search

- Idea: use single current state and move to “neighbouring” states
 - operate using a single current node
 - the paths followed by the search are not retained
- Two key advantages:
 - use very little memory (usually constant)
 - can often find reasonable solutions in large or infinite (continuous) state spaces, for which systematic algorithms are unsuitable
- Also useful for pure optimization problems
 - find the best state according to an objective function
 - often do not fit the “standard” search model of previous chapter
 - ex: Darwinian survival of the fittest: metaphor for optimization, but no “goal test” and no “path cost”
- A complete local search algorithm: guaranteed to always find a solution (if exists)
- A optimal local search algorithm: guaranteed to always find a maximum/minimum solution
 - maximization and minimization dual (switch sign)

Local Search

- Idea: **use single current state and move to “neighbouring” states**
 - operate using a single current node
 - the paths followed by the search are not retained
- Two key advantages:
 - **use very little memory** (usually constant)
 - **can often find reasonable solutions in large or infinite (continuous) state spaces**, for which systematic algorithms are unsuitable
- Also useful for **pure optimization problems**
 - find the best state according to an **objective function**
 - often do not fit the “standard” search model of previous chapter
 - ex: **Darwinian survival of the fittest**: metaphor for optimization, but no “goal test” and no “path cost”
- A **complete** local search algorithm: **guaranteed to always find a solution (if exists)**
- A **optimal** local search algorithm: **guaranteed to always find a maximum/minimum solution**
 - maximization and minimization dual (switch sign)

Local Search

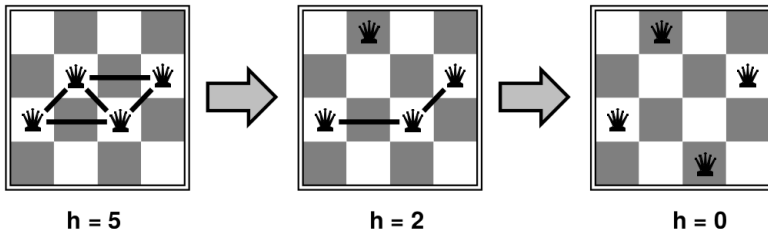
- Idea: **use single current state and move to “neighbouring” states**
 - operate using a single current node
 - the paths followed by the search are not retained
- Two key advantages:
 - **use very little memory** (usually constant)
 - **can often find reasonable solutions in large or infinite (continuous) state spaces**, for which systematic algorithms are unsuitable
- Also useful for **pure optimization problems**
 - find the best state according to an **objective function**
 - often do not fit the “standard” search model of previous chapter
 - ex: **Darwinian survival of the fittest**: metaphor for optimization, but no “goal test” and no “path cost”
- A **complete** local search algorithm: **guaranteed to always find a solution (if exists)**
- A **optimal** local search algorithm: **guaranteed to always find a maximum/minimum solution**
 - maximization and minimization dual (switch sign)

Local Search

- Idea: **use single current state and move to “neighbouring” states**
 - operate using a single current node
 - the paths followed by the search are not retained
- Two key advantages:
 - **use very little memory** (usually constant)
 - **can often find reasonable solutions in large or infinite (continuous) state spaces**, for which systematic algorithms are unsuitable
- Also useful for **pure optimization problems**
 - find the best state according to an **objective function**
 - often do not fit the “standard” search model of previous chapter
 - ex: **Darwinian survival of the fittest**: metaphor for optimization, but no “goal test” and no “path cost”
- A **complete** local search algorithm: **guaranteed to always find a solution (if exists)**
- A **optimal** local search algorithm: **guaranteed to always find a maximum/minimum solution**
 - maximization and minimization dual (switch sign)

Local Search Example: N-Queens

- One queen per column (incremental representation)
- Cost (h): # of queen pairs on the same row, column, or diagonal
- Goal: $h=0$
- Step: move a queen vertically to reduce number of conflicts

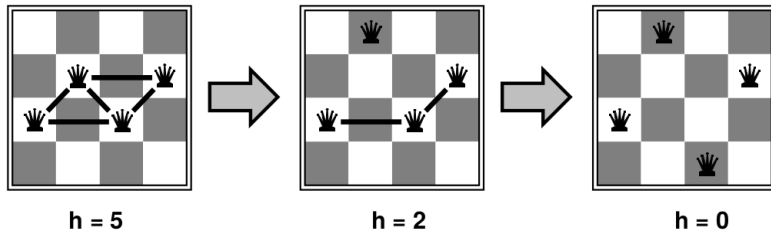


(© S. Russell & P. Norwig, AIMA)

Almost always solves N-queens problems almost instantaneously for very large N (e.g., $N=1$ million)

Local Search Example: N-Queens

- One queen per column (incremental representation)
- Cost (h): # of queen pairs on the same row, column, or diagonal
- Goal: $h=0$
- Step: move a queen vertically to reduce number of conflicts



(© S. Russell & P. Norwig, AIMA)

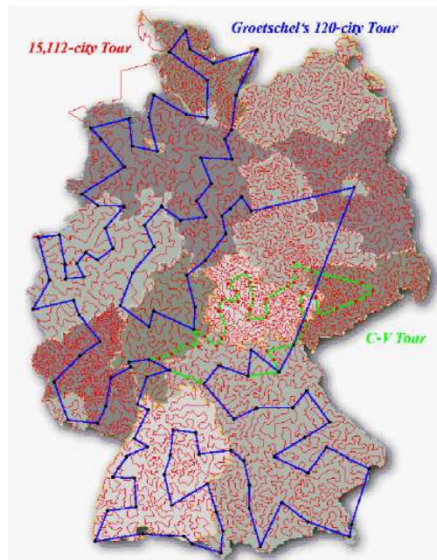
Almost always solves N-queens problems almost instantaneously for very large N (e.g., N=1million)

Optimization Local Search Example: TSP

Travelling Salesperson Problem (TSP)

Given an undirected graph, with n nodes and each arc associated with a positive value, find the Hamiltonian tour with the minimum total cost.

Very hard for classic search!

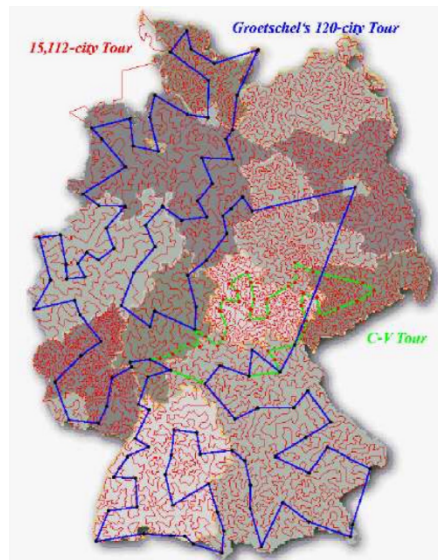


Optimization Local Search Example: TSP

Travelling Salesperson Problem (TSP)

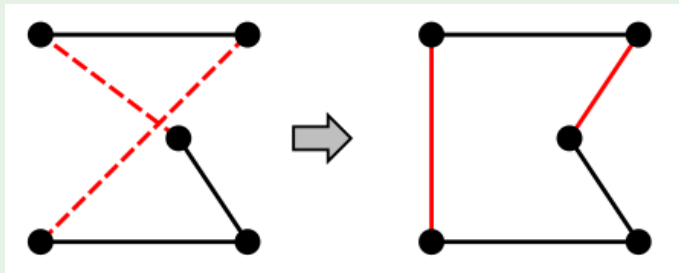
Given an undirected graph, with n nodes and each arc associated with a positive value, find the Hamiltonian tour with the minimum total cost.

Very hard for classic search!



Optimization Local Search Example: TSP

- State represented as a permutation of numbers $(1, 2, \dots, n)$
- Cost (h): total cycle length
- Start with any complete tour
- Step: (2-swap) perform pairwise exchange

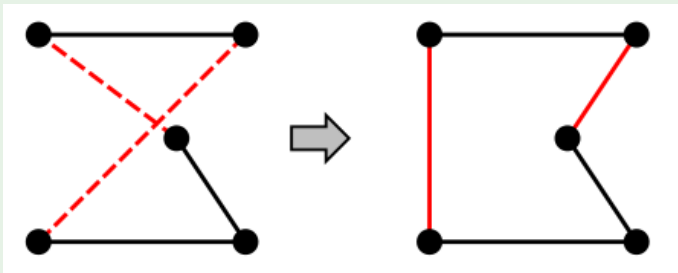


(© S. Russell & P. Norwig, AIMA)

Variants of this approach get within 1% of optimal very quickly with thousands of cities

Optimization Local Search Example: TSP

- State represented as a permutation of numbers $(1, 2, \dots, n)$
- Cost (h): total cycle length
- Start with any complete tour
- Step: (2-swap) perform pairwise exchange



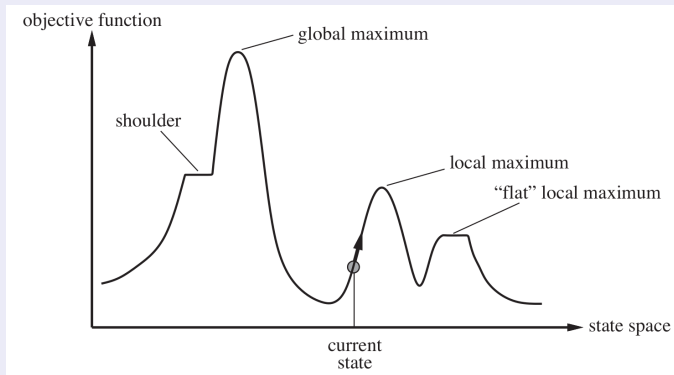
(© S. Russell & P. Norwig, AIMA)

Variants of this approach get within 1% of optimal very quickly with thousands of cities

Local Search: State-Space Landscape

State-space landscape (Maximization)

- Local search algorithms **explore state-space landscape**
 - state space n -dimensional (and typically discrete)
 - move to “nearby” states (**neighbours**)
- **NP-Hard problems may have exponentially-many local optima**



Remark: Handling randomness in exercises

- Task: Pick randomly one choice among some (weighted) options
 - In computer implementation: pseudo-random number generation
 - In our exercises: we need a general weighted sampling procedure
- Idea: Normalize the vector of options according to the weights $p_i = \frac{w_i}{\sum_{i=1}^N w_i}$
(a sequence of) pre-determined choices $c \in (0, 1]$ are used to sample

$$\left[\sum_{i=1}^{j-1} p_i < c \leq \sum_{i=1}^j p_i \right] \iff \text{element } j \text{ is selected}$$

- Sequence of samples are obtained by (cycling through) a choice vector $[c_1, \dots, c_n]$, $c_i \in (0, 1]$

Example

● Choice vector: $[\frac{1}{2}, \frac{1}{10}, \frac{9}{10}] \implies$ Choice sequence: 2, 3, 3

Remark: Handling randomness in exercises

- Task: Pick randomly one choice among some (weighted) options
 - In computer implementation: pseudo-random number generation
 - In our exercises: we need a general weighted sampling procedure
- Idea: Normalize the vector of options according to the weights $p_i = \frac{w_i}{\sum_{i=1}^N w_i}$
(a sequence of) pre-determined choices $c \in (0, 1]$ are used to sample

$$\left[\sum_{i=1}^{j-1} p_i < c \leq \sum_{i=1}^j p_i \right] \iff \text{element } j \text{ is selected}$$

- Sequence of samples are obtained by (cycling through) a choice vector $[c_1, \dots, c_n]$, $c_i \in (0, 1]$

Example



- Choice vector: $[\frac{1}{2}, \frac{1}{10}, \frac{9}{10}] \implies$ Choice sequence: B, A, D

Remark: Handling randomness in exercises

- Task: Pick randomly one choice among some (weighted) options
 - In computer implementation: pseudo-random number generation
 - In our exercises: we need a general weighted sampling procedure
- Idea: Normalize the vector of options according to the weights $p_i = \frac{w_i}{\sum_{i=1}^N w_i}$
(a sequence of) pre-determined choices $c \in (0, 1]$ are used to sample

$$\left[\sum_{i=1}^{j-1} p_i < c \leq \sum_{i=1}^j p_i \right] \iff \text{element } j \text{ is selected}$$

- Sequence of samples are obtained by (cycling through) a choice vector $[c_1, \dots, c_n]$, $c_i \in (0, 1]$

Example



$$3/7 < \frac{1}{2} \leq 4/7 \rightarrow B \text{ is selected}$$

- Choice vector: $[\frac{1}{2}, \frac{1}{10}, \frac{9}{10}] \implies$ Choice sequence: B, A, D

Remark: Handling randomness in exercises

- Task: Pick randomly one choice among some (weighted) options
 - In computer implementation: pseudo-random number generation
 - In our exercises: we need a general weighted sampling procedure
- Idea: Normalize the vector of options according to the weights $p_i = \frac{w_i}{\sum_{i=1}^N w_i}$
(a sequence of) pre-determined choices $c \in (0, 1]$ are used to sample

$$\left[\sum_{i=1}^{j-1} p_i < c \leq \sum_{i=1}^j p_i \right] \iff \text{element } j \text{ is selected}$$

- Sequence of samples are obtained by (cycling through) a choice vector $[c_1, \dots, c_n]$, $c_i \in (0, 1]$

Example



$$0 < \frac{1}{10} \leq 3/7 \rightarrow \text{A is selected}$$

- Choice vector: $[\frac{1}{2}, \frac{1}{10}, \frac{9}{10}] \implies$ Choice sequence: B, A, D

Remark: Handling randomness in exercises

- Task: Pick randomly one choice among some (weighted) options
 - In computer implementation: pseudo-random number generation
 - In our exercises: we need a general weighted sampling procedure
- Idea: Normalize the vector of options according to the weights $p_i = \frac{w_i}{\sum_{i=1}^N w_i}$
(a sequence of) pre-determined choices $c \in (0, 1]$ are used to sample

$$\left[\sum_{i=1}^{j-1} p_i < c \leq \sum_{i=1}^j p_i \right] \iff \text{element } j \text{ is selected}$$

- Sequence of samples are obtained by (cycling through) a choice vector $[c_1, \dots, c_n]$, $c_i \in (0, 1]$

Example



$$6/7 < \frac{9}{10} \leq 1 \rightarrow D \text{ is selected}$$

- Choice vector: $[\frac{1}{2}, \frac{1}{10}, \frac{9}{10}] \implies$ Choice sequence: B, A, D

Handling randomness in exercises: Examples

Examples

choice vector = $[1/4, 9/10, 2/3]$

- *weighted options* = $[A : 3, B : 6]$

⇒ *normalized weighted options* = $[A : 1/3, B : 2/3]$

⇒ $0 < 1/4 \leq 1/3 \rightarrow A$ is selected

- *weighted options* = $[C : 3]$

⇒ *normalized weighted options* = $[C : 1]$

⇒ $0 < 9/10 \leq 1 \rightarrow C$ is selected

- *weighted options* = $[D : 1, E : 1]$

⇒ *normalized weighted options* = $[D : 1/2, E : 1/2]$

⇒ $1/2 < 2/3 \leq 1 \rightarrow E$ is selected

- *weighted options* = $[F : 2, G : 10, H : 4]$

⇒ *normalized weighted options* = $[F : 1/8, G : 5/8, H : 2/8]$

⇒ $1/8 < 1/4 \leq 6/8 \rightarrow G$ is selected

Handling randomness in exercises: Examples

Examples

choice vector = $[1/4, 9/10, 2/3]$

- *weighted options* = $[A : 3, B : 6]$

⇒ *normalized weighted options* = $[A : 1/3, B : 2/3]$

⇒ $0 < 1/4 \leq 1/3 \rightarrow A$ is selected

- *weighted options* = $[C : 3]$

⇒ *normalized weighted options* = $[C : 1]$

⇒ $0 < 9/10 \leq 1 \rightarrow C$ is selected

- *weighted options* = $[D : 1, E : 1]$

⇒ *normalized weighted options* = $[D : 1/2, E : 1/2]$

⇒ $1/2 < 2/3 \leq 1 \rightarrow E$ is selected

- *weighted options* = $[F : 2, G : 10, H : 4]$

⇒ *normalized weighted options* = $[F : 1/8, G : 5/8, H : 2/8]$

⇒ $1/8 < 1/4 \leq 6/8 \rightarrow G$ is selected

Handling randomness in exercises: Examples

Examples

choice vector = $[1/4, 9/10, 2/3]$

- *weighted options* = $[A : 3, B : 6]$

⇒ *normalized weighted options* = $[A : 1/3, B : 2/3]$

⇒ $0 < 1/4 \leq 1/3 \rightarrow A$ is selected

- *weighted options* = $[C : 3]$

⇒ *normalized weighted options* = $[C : 1]$

⇒ $0 < 9/10 \leq 1 \rightarrow C$ is selected

- *weighted options* = $[D : 1, E : 1]$

⇒ *normalized weighted options* = $[D : 1/2, E : 1/2]$

⇒ $1/2 < 2/3 \leq 1 \rightarrow E$ is selected

- *weighted options* = $[F : 2, G : 10, H : 4]$

⇒ *normalized weighted options* = $[F : 1/8, G : 5/8, H : 2/8]$

⇒ $1/8 < 1/4 \leq 6/8 \rightarrow G$ is selected

Handling randomness in exercises: Examples

Examples

choice vector = $[1/4, 9/10, 2/3]$

- *weighted options* = $[A : 3, B : 6]$

⇒ *normalized weighted options* = $[A : 1/3, B : 2/3]$

⇒ $0 < 1/4 \leq 1/3 \rightarrow A$ is selected

- *weighted options* = $[C : 3]$

⇒ *normalized weighted options* = $[C : 1]$

⇒ $0 < 9/10 \leq 1 \rightarrow C$ is selected

- *weighted options* = $[D : 1, E : 1]$

⇒ *normalized weighted options* = $[D : 1/2, E : 1/2]$

⇒ $1/2 < 2/3 \leq 1 \rightarrow E$ is selected

- *weighted options* = $[F : 2, G : 10, H : 4]$

⇒ *normalized weighted options* = $[F : 1/8, G : 5/8, H : 2/8]$

⇒ $1/8 < 1/4 \leq 6/8 \rightarrow G$ is selected

- 1 Local Search and Optimization
 - General Ideas
 - **Hill-Climbing**
 - Simulated Annealing
 - Local Beam Search & Genetic Algorithms
- 2 Search with Nondeterministic Actions
- 3 Search with Partial or No Observations (Deterministic/Nondeterministic Actions)
 - Search with No Observations
 - Search with Partial Observations
- 4 Online Search (aka Exploration)

Hill-Climbing Search (aka Greedy Local Search)

Hill-Climbing

- Very-basic local search algorithm
- Idea: **a move is performed only if the solution it produces is better than the current solution**
 - (steepest-ascent version): **selects the neighbour with best score improvement**
(select randomly among best neighbours if ≥ 1)
 - does not look ahead of immediate neighbors of the current state
 - **stops as soon as it finds a (possibly local) minimum**
- Several variants (Stochastic H.C., Random-Restart H.C., ...)
- Often used as part of more complex local-search algorithms

function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

loop do

neighbor $\leftarrow$ a highest-valued successor of *current*

if *neighbor*.VALUE $\leq$ *current*.VALUE **then return** *current*.STATE

current $\leftarrow$ *neighbor*

Hill-Climbing Search (aka Greedy Local Search)

Hill-Climbing

- Very-basic local search algorithm
- Idea: **a move is performed only if the solution it produces is better than the current solution**
 - (steepest-ascent version): **selects the neighbour with best score improvement**
(select randomly among best neighbours if ≥ 1)
 - does not look ahead of immediate neighbors of the current state
 - **stops as soon as it finds a (possibly local) minimum**
- Several variants (Stochastic H.C., Random-Restart H.C., ...)
- Often used as part of more complex local-search algorithms

function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

loop do

neighbor $\leftarrow$ a highest-valued successor of *current*

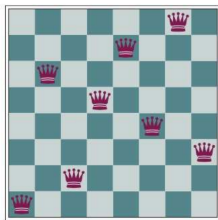
if *neighbor*.VALUE $\leq$ *current*.VALUE **then return** *current*.STATE

current $\leftarrow$ *neighbor*

Hill-Climbing Search: Example

8-queen puzzle (minimization)

- Neighbour states: generated by moving one queen vertically
 - Cost (h): # of queen pairs on the same row, column, or diagonal
 - Goal: $h=0$
- Two example scenarios ((b) $\Rightarrow$ (a) in 5 steps):
 - (a) $h=1$, but all neighbours have higher costs $\Rightarrow$ local minimum
 - (b) 8-queens state with heuristic cost estimate $h = 17$ (12d, 5h)
 - 8 moves with minimum cost 12: hill-climbing will pick one



(a)

A chessboard diagram labeled (b) showing a state where the heuristic cost $h=17$ (12d, 5h). The queens are positioned at (row, column): (1, 6), (2, 8), (3, 2), (4, 4), (5, 7), (6, 1), (7, 5), and (8, 3). There are 12 diagonal conflicts and 5 horizontal/vertical conflicts.

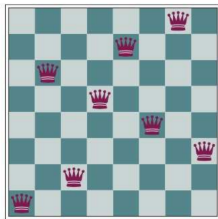
18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	17	13	16	13	16
17	14	17	15	17	14	16	16
17	17	16	18	15	17	15	17
18	14	17	15	15	14	17	16
14	14	13	17	12	14	12	18

(b)

Hill-Climbing Search: Example

8-queen puzzle (minimization)

- Neighbour states: generated by moving one queen vertically
 - Cost (h): # of queen pairs on the same row, column, or diagonal
 - Goal: $h=0$
- Two example scenarios $((b) \implies (a) \text{ in 5 steps })$:
 - (a) $h=1$, but all neighbours have higher costs $\implies$ local minimum
 - (b) 8-queens state with heuristic cost estimate $h = 17$ (12d, 5h)
 - 8 moves with minimum cost 12: hill-climbing will pick one



(a)

A chessboard diagram labeled (b) showing a state where the heuristic cost $h=17$ (12d, 5h). The board has a light blue and dark blue checkerboard pattern. Eight queens are placed on the board. The queens are located at the following (row, column) positions: (1, 2), (2, 7), (3, 1), (4, 5), (5, 8), (6, 3), (7, 6), and (8, 4). There are 12 diagonal conflicts and 5 column conflicts, resulting in a total cost of 17.

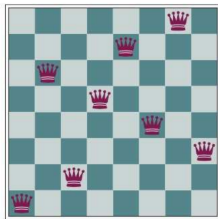
18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	17	13	16	13	16
17	14	17	15	17	14	16	16
17	17	16	18	15	17	15	17
18	14	17	15	15	14	17	16
14	14	13	17	12	14	12	18

(b)

Hill-Climbing Search: Example

8-queen puzzle (minimization)

- Neighbour states: generated by moving one queen vertically
 - Cost (h): # of queen pairs on the same row, column, or diagonal
 - Goal: $h=0$
- Two example scenarios $((b) \implies (a) \text{ in 5 steps })$:
 - (a) $h=1$, but all neighbours have higher costs $\implies$ local minimum
 - (b) 8-queens state with heuristic cost estimate $h = 17$ (12d, 5h)
 - 8 moves with minimum cost 12: hill-climbing will pick one



(a)

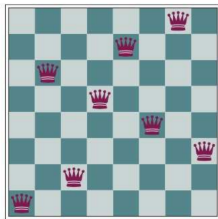
18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	17	13	16	13	16
17	14	17	15	17	14	16	16
17	17	16	18	15	17	15	17
18	14	17	15	15	14	17	16
14	14	13	17	12	14	12	18

(b)

Hill-Climbing Search: Example

8-queen puzzle (minimization)

- Neighbour states: generated by moving one queen vertically
 - Cost (h): # of queen pairs on the same row, column, or diagonal
 - Goal: $h=0$
- Two example scenarios $((b) \implies (a) \text{ in 5 steps })$:
 - (a) $h=1$, but all neighbours have higher costs $\implies$ local minimum
 - (b) 8-queens state with heuristic cost estimate $h = 17$ (12d, 5h)
 - 8 moves with minimum cost 12: hill-climbing will pick one



(a)

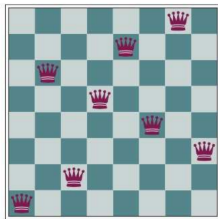
18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	17	13	16	13	16
17	14	17	15	17	14	16	16
17	17	16	18	15	17	15	17
18	14	17	15	15	14	17	16
14	14	13	17	12	14	12	18

(b)

Hill-Climbing Search: Example

8-queen puzzle (minimization)

- Neighbour states: generated by moving one queen vertically
 - Cost (h): # of queen pairs on the same row, column, or diagonal
 - Goal: $h=0$
- Two example scenarios $((b) \implies (a) \text{ in 5 steps })$:
 - (a) $h=1$, but all neighbours have higher costs $\implies$ local minimum
 - (b) 8-queens state with heuristic cost estimate $h = 17$ (12d, 5h)
 - 8 moves with minimum cost 12: hill-climbing will pick one



(a)

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	17	13	16	13	16
17	14	17	15	17	14	16	16
17	17	16	18	15	17	15	17
18	14	17	15	15	14	17	16
14	14	13	17	12	14	12	18

(b)

- Consider the 8-Puzzle example of A^* in Chapter 3.
Consider $h(n)$ = sum of manhattan distances of tiles to their goal position.
 - Try to solve it by hill-climbing.
 - Try to solve it with other initial configurations

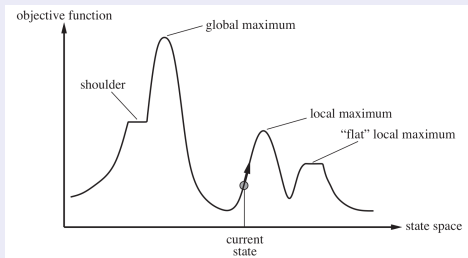
Hill-Climbing Search: Drawbacks

- **Incomplete**: gets stuck in **local optima**, **flat local optima** & **shoulders** (aka **plateaux**), **ridges** (sequences of local optima)
 - Ex: with 8-queens, gets stuck 86% of the time, fast when succeed

note: converges very fast till (local) minima or plateaux
- Possible idea: **allow 0-progress moves** (aka **sideways moves**)
 - pros: may allow getting out of shoulders
 - cons: may cause infinite loops with flat local optima

⇒ set a limit to consecutive sideways moves (e.g. 100)

 - Ex: with 8-queens, pass from 14% to 94% success, slower



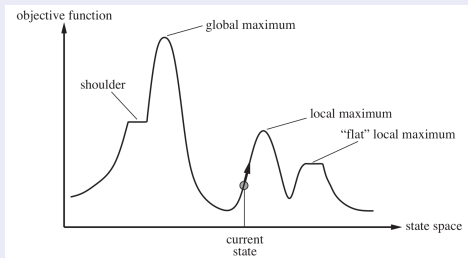
Hill-Climbing Search: Drawbacks

- **Incomplete**: gets stuck in **local optima**, **flat local optima** & **shoulders** (aka **plateaux**), **ridges** (sequences of local optima)
 - Ex: with 8-queens, gets stuck 86% of the time, fast when succeed

note: converges very fast till (local) minima or plateaux
- Possible idea: **allow 0-progress moves** (aka **sideways moves**)
 - pros: may allow getting out of shoulders
 - cons: may cause infinite loops with flat local optima

⇒ set a limit to consecutive sideways moves (e.g. 100)

 - Ex: with 8-queens, pass from 14% to 94% success, slower



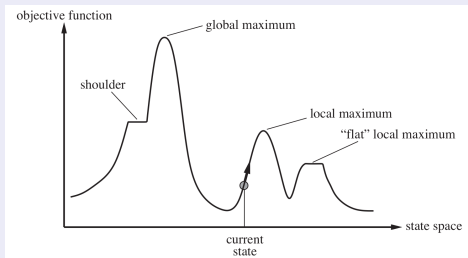
Hill-Climbing Search: Drawbacks

- **Incomplete**: gets stuck in **local optima**, **flat local optima** & **shoulders** (aka **plateaux**), **ridges** (sequences of local optima)
 - Ex: with 8-queens, gets stuck 86% of the time, fast when succeed

note: converges very fast till (local) minima or plateaux
- Possible idea: **allow 0-progress moves** (aka **sideways moves**)
 - pros: **may allow getting out of shoulders**
 - cons: **may cause infinite loops with flat local optima**

⇒ set a limit to consecutive sideways moves (e.g. 100)

 - Ex: with 8-queens, pass from 14% to 94% success, slower



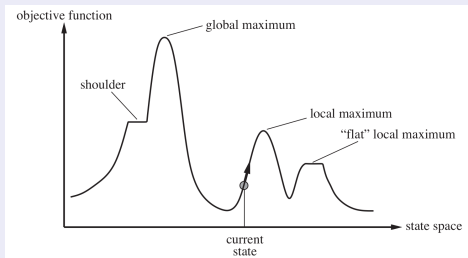
Hill-Climbing Search: Drawbacks

- **Incomplete**: gets stuck in **local optima**, **flat local optima** & **shoulders** (aka **plateaux**), **ridges** (sequences of local optima)
 - Ex: with 8-queens, gets stuck 86% of the time, fast when succeed

note: converges very fast till (local) minima or plateaux
- Possible idea: **allow 0-progress moves** (aka **sideways moves**)
 - pros: **may allow getting out of shoulders**
 - cons: **may cause infinite loops with flat local optima**

⇒ set a limit to consecutive sideways moves (e.g. 100)

 - Ex: with 8-queens, pass from 14% to 94% success, slower



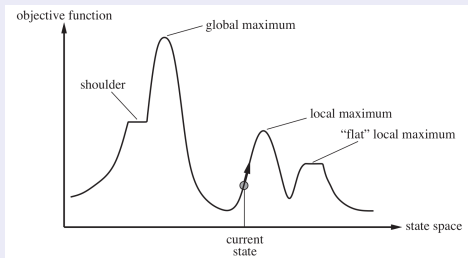
Hill-Climbing Search: Drawbacks

- **Incomplete**: gets stuck in **local optima**, **flat local optima** & **shoulders** (aka **plateaux**), **ridges** (sequences of local optima)
 - Ex: **with 8-queens**, gets stuck 86% of the time, fast when succeed

note: converges very fast till (local) minima or plateaux
- Possible idea: **allow 0-progress moves** (aka **sideways moves**)
 - pros: **may allow getting out of shoulders**
 - cons: **may cause infinite loops with flat local optima**

⇒ set a limit to consecutive sideways moves (e.g. 100)

 - Ex: **with 8-queens**, pass from 14% to 94% success, slower



Hill-climbing: Variations

- **Stochastic hill-climbing**
 - random selection among the uphill moves
 - selection probability can vary with the steepness of uphill move
 - sometimes slower, but often finds better solutions
- **First-choice hill-climbing**
 - generates successors randomly until a better one is found
 - good when there are large amounts of successors
- **Random-restart hill-climbing**
 - conducts a series of hill-climbing searches from randomly generated initial states
 - tries to avoid getting stuck in local maxima

- 1 Local Search and Optimization
 - General Ideas
 - Hill-Climbing
 - **Simulated Annealing**
 - Local Beam Search & Genetic Algorithms
- 2 Search with Nondeterministic Actions
- 3 Search with Partial or No Observations (Deterministic/Nondeterministic Actions)
 - Search with No Observations
 - Search with Partial Observations
- 4 Online Search (aka Exploration)

Simulated Annealing

- Inspired to statistical-mechanics analysis of metallurgical annealing (Boltzmann's state distributions)
- Idea: **Escape local maxima by allowing "bad" moves...**
 - "bad move": move toward states with worse value
 - typically pick a move taken at random ("random walk")
- ... but gradually decrease their size and frequency.
 - sideways moves progressively less likely
- Analogy: get a ball into the deepest crevice in a bumpy surface
 - initially shaking hard ("high temperature")
 - progressively shaking less hard ("decrease the temperature")

Widely used in large-scale optimization tasks (e.g. VLSI layout problems, factory scheduling,...)

Simulated Annealing

- Inspired to statistical-mechanics analysis of metallurgical annealing (Boltzmann's state distributions)
- Idea: **Escape local maxima by allowing "bad" moves...**
 - "bad move": move toward states with worse value
 - typically pick a move taken at random ("random walk")
- ... **but gradually decrease their size and frequency.**
 - sideways moves progressively less likely
- Analogy: get a ball into the deepest crevice in a bumpy surface
 - initially shaking hard ("high temperature")
 - progressively shaking less hard ("decrease the temperature")

Widely used in large-scale optimization tasks (e.g. **VLSI layout problems, factory scheduling,...**)

Simulated Annealing

- Inspired to statistical-mechanics analysis of metallurgical annealing (Boltzmann's state distributions)
- Idea: **Escape local maxima by allowing “bad” moves...**
 - “bad move”: move toward states with worse value
 - typically pick a move taken at random (“**random walk**”)
- **... but gradually decrease their size and frequency.**
 - sideways moves progressively less likely
- Analogy: get a ball into the deepest crevice in a bumpy surface
 - initially shaking hard (“high temperature”)
 - progressively shaking less hard (“decrease the temperature”)

Widely used in large-scale optimization tasks (e.g. **VLSI layout problems, factory scheduling,...**)

Simulated Annealing

- Inspired to statistical-mechanics analysis of metallurgical annealing (Boltzmann's state distributions)
- Idea: **Escape local maxima by allowing “bad” moves...**
 - “bad move”: move toward states with worse value
 - typically pick a move taken at random (“**random walk**”)
- ... **but gradually decrease their size and frequency.**
 - sideways moves progressively less likely
- Analogy: get a ball into the deepest crevice in a bumpy surface
 - initially shaking hard (“high temperature”)
 - progressively shaking less hard (“decrease the temperature”)

Widely used in large-scale optimization tasks (e.g. **VLSI layout problems, factory scheduling,...**)

Simulated Annealing [cont.]

Simulated Annealing (minimization)

- A “temperature” parameter T slowly decreases with steps (“schedule”)
- The probability of picking a “bad move”:
 - decreases exponentially with the “badness” of the move $|\Delta E|$ (bad move if $\Delta E < 0$)
 - decreases as the “temperature” T goes down
- If schedule lowers T slowly enough,
then the algorithm will find a global optimum with probability approaching 1

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  current  $\leftarrow$  problem.INITIAL
  for  $t = 1$  to  $\infty$  do
     $T \leftarrow$  schedule( $t$ )
    if  $T = 0$  then return current
    next  $\leftarrow$  a randomly selected successor of current
     $\Delta E \leftarrow$  VALUE(current) – VALUE(next)
    if  $\Delta E > 0$  then current  $\leftarrow$  next    // if improved, then it is accepted
    else current  $\leftarrow$  next only with probability  $e^{\Delta E/T}$  //if worse, accepted with probability
                                                    // which decreases with time
```

Simulated Annealing: Example

Example: 8-puzzle

Current
State:

4	1	3
2		6
7	5	8

Possible
Successor
States:

(a)

(b)

(c)

(d)

- Consider the Simulated Annealing procedure with
 - $\text{schedule}(t) \stackrel{\text{def}}{=} 2^{-t}$ and
 - Value(n) = the sum of manhattan distances of tiles to their goal position.
- At steps $t_1 = 1$ and $t_2 = 5$, consider the current state and its possible successors (a)-(d).
 - What are the probabilities $p_{(a)}^{t_i}, p_{(b)}^{t_i}, p_{(c)}^{t_i}, p_{(d)}^{t_i}$ that the next state is (a), (b), (c), (d), for each t_i ?
 - What is the probability $p_{\text{curr}}^{t_i}$ that the next state is still the current one, for each t_i ?

(b) and (c): $\Delta E = 6 - 5 = 1 > 0 \implies p_{(b)}^h = p_{(c)}^h = 1/4$, for each t_i

(a) and (d): $\Delta E = 6 - 7 = -1 < 0 \implies$

$$t_1: p_{(a)}^h = p_{(d)}^h = 1/4 \cdot e^{-1/(2^{-1})} = 0.0338; \quad p_{\text{curr}}^h = 1 - 2 \cdot 1/4 - 2 \cdot p_{(a)}^h = 0.432$$

$$t_2: p_{(a)}^b = p_{(d)}^b = 1/4 \cdot e^{-1/(2^{-5})} = 3.17 \cdot 10^{-15}; \quad p_{\text{curr}}^b = 1 - 2 \cdot 1/4 - 2 \cdot p_{(a)}^b = 0.5$$

Simulated Annealing: Example

Example: 8-puzzle

Value:	6	7	5	5	7																																														
Current State:	<table><tr><td>4</td><td>1</td><td>3</td></tr><tr><td>2</td><td></td><td>6</td></tr><tr><td>7</td><td>5</td><td>8</td></tr></table>	4	1	3	2		6	7	5	8	Possible Successor States:	<table><tr><td>4</td><td></td><td>3</td></tr><tr><td>2</td><td>1</td><td>6</td></tr><tr><td>7</td><td>5</td><td>8</td></tr></table>	4		3	2	1	6	7	5	8	<table><tr><td>4</td><td>1</td><td>3</td></tr><tr><td></td><td>2</td><td>6</td></tr><tr><td>7</td><td>5</td><td>8</td></tr></table>	4	1	3		2	6	7	5	8	<table><tr><td>4</td><td>1</td><td>3</td></tr><tr><td>2</td><td>5</td><td>6</td></tr><tr><td>7</td><td></td><td>8</td></tr></table>	4	1	3	2	5	6	7		8	<table><tr><td>4</td><td>1</td><td>3</td></tr><tr><td>2</td><td>6</td><td></td></tr><tr><td>7</td><td>5</td><td>8</td></tr></table>	4	1	3	2	6		7	5	8
4	1	3																																																	
2		6																																																	
7	5	8																																																	
4		3																																																	
2	1	6																																																	
7	5	8																																																	
4	1	3																																																	
	2	6																																																	
7	5	8																																																	
4	1	3																																																	
2	5	6																																																	
7		8																																																	
4	1	3																																																	
2	6																																																		
7	5	8																																																	
		(a)	(b)	(c)	(d)																																														

- Consider the Simulated Annealing procedure with
 - $\text{schedule}(t) \stackrel{\text{def}}{=} 2^{-t}$ and
 - $\text{Value}(n)$ = the sum of manhattan distances of tiles to their goal position.
- At steps $t_1 = 1$ and $t_2 = 5$, consider the current state and its possible successors (a)-(d).
 - What are the probabilities $p_{(a)}^{t_i}, p_{(b)}^{t_i}, p_{(c)}^{t_i}, p_{(d)}^{t_i}$ that the next state is (a), (b), (c), (d), for each t_i ?
 - What is the probability $p_{\text{curr}}^{t_i}$ that the next state is still the current one, for each t_i ?

(b) and (c): $\Delta E = 6 - 5 = 1 > 0 \implies p_{(b)}^{t_i} = p_{(c)}^{t_i} = 1/4$, for each t_i

(a) and (d): $\Delta E = 6 - 7 = -1 < 0 \implies$

$$t_1: p_{(a)}^{t_1} = p_{(d)}^{t_1} = 1/4 \cdot e^{-1/(2^{-1})} = 0.0338; \quad p_{\text{curr}}^{t_1} = 1 - 2 \cdot 1/4 - 2 \cdot p_{(a)}^{t_1} = 0.432$$

$$t_2: p_{(a)}^{t_2} = p_{(d)}^{t_2} = 1/4 \cdot e^{-1/(2^{-5})} = 3.17 \cdot 10^{-15}; \quad p_{\text{curr}}^{t_2} = 1 - 2 \cdot 1/4 - 2 \cdot p_{(a)}^{t_2} = 0.5$$

Simulated Annealing: Example

Example: 8-puzzle

Value:	6	7	5	5	7																																														
Current State:	<table><tr><td>4</td><td>1</td><td>3</td></tr><tr><td>2</td><td></td><td>6</td></tr><tr><td>7</td><td>5</td><td>8</td></tr></table>	4	1	3	2		6	7	5	8	Possible Successor States:	<table><tr><td>4</td><td></td><td>3</td></tr><tr><td>2</td><td>1</td><td>6</td></tr><tr><td>7</td><td>5</td><td>8</td></tr></table>	4		3	2	1	6	7	5	8	<table><tr><td>4</td><td>1</td><td>3</td></tr><tr><td></td><td>2</td><td>6</td></tr><tr><td>7</td><td>5</td><td>8</td></tr></table>	4	1	3		2	6	7	5	8	<table><tr><td>4</td><td>1</td><td>3</td></tr><tr><td>2</td><td>5</td><td>6</td></tr><tr><td>7</td><td></td><td>8</td></tr></table>	4	1	3	2	5	6	7		8	<table><tr><td>4</td><td>1</td><td>3</td></tr><tr><td>2</td><td>6</td><td></td></tr><tr><td>7</td><td>5</td><td>8</td></tr></table>	4	1	3	2	6		7	5	8
4	1	3																																																	
2		6																																																	
7	5	8																																																	
4		3																																																	
2	1	6																																																	
7	5	8																																																	
4	1	3																																																	
	2	6																																																	
7	5	8																																																	
4	1	3																																																	
2	5	6																																																	
7		8																																																	
4	1	3																																																	
2	6																																																		
7	5	8																																																	
		(a)	(b)	(c)	(d)																																														

- Consider the Simulated Annealing procedure with
 - $\text{schedule}(t) \stackrel{\text{def}}{=} 2^{-t}$ and
 - $\text{Value}(n)$ = the sum of manhattan distances of tiles to their goal position.
- At steps $t_1 = 1$ and $t_2 = 5$, consider the current state and its possible successors (a)-(d).
 - What are the probabilities $p_{(a)}^{t_i}, p_{(b)}^{t_i}, p_{(c)}^{t_i}, p_{(d)}^{t_i}$ that the next state is (a), (b), (c), (d), for each t_i ?
 - What is the probability $p_{curr}^{t_i}$ that the next state is still the current one, for each t_i ?

(b) and (c): $\Delta E = 6 - 5 = 1 > 0 \implies p_{(b)}^{t_i} = p_{(c)}^{t_i} = 1/4$, for each t_i

(a) and (d): $\Delta E = 6 - 7 = -1 < 0 \implies$

$$t_1: p_{(a)}^{t_1} = p_{(d)}^{t_1} = 1/4 \cdot e^{-1/(2^{-1})} = 0.0338; \quad p_{curr}^{t_1} = 1 - 2 \cdot 1/4 - 2 \cdot p_{(a)}^{t_1} = 0.432$$

$$t_2: p_{(a)}^{t_2} = p_{(d)}^{t_2} = 1/4 \cdot e^{-1/(2^{-5})} = 3.17 \cdot 10^{-15}; \quad p_{curr}^{t_2} = 1 - 2 \cdot 1/4 - 2 \cdot p_{(a)}^{t_2} = 0.5$$

Simulated Annealing: Exercise

- Consider the scenario of the previous example.
 - Consider the following move order: $\{up, left, down, right\}$
 - Consider the following choice vector: $[1/5, 1/100, 1/3, 1/1000]$
 - $t = t_1 = 1$

Describe the sequence of the first two steps of the execution.

- 1 Local Search and Optimization
 - General Ideas
 - Hill-Climbing
 - Simulated Annealing
 - Local Beam Search & Genetic Algorithms
- 2 Search with Nondeterministic Actions
- 3 Search with Partial or No Observations (Deterministic/Nondeterministic Actions)
 - Search with No Observations
 - Search with Partial Observations
- 4 Online Search (aka Exploration)

Local Beam Search

Local Beam Search

Idea: **keep track of k states instead of one**

- Initially: k random states
 - Step:
 - 1 determine all successors of k states
 - 2 if any of successors is goal $\Rightarrow$ finished
 - 3 **else select k best from successors**
 - Different from k searches run in parallel:
 - searches that find good states recruit other searches to join them
 $\Rightarrow$ information is shared among k search threads
 - Lack of diversity: **quite often, all k states end up in the same local hill**
- $\Rightarrow$ **Stochastic Local Beam**: choose k successors randomly, with probability proportional to state success.

Resembles natural selection with asexual reproduction:

the successors (offspring) of a state (organism) populate the next generation according to its value (fitness), with a random component.

Local Beam Search

Local Beam Search

Idea: **keep track of k states instead of one**

- Initially: k random states
 - Step:
 - determine all successors of k states
 - if any of successors is goal $\Rightarrow$ finished
 - else select k best from successors**
 - Different from k searches run in parallel:
 - searches that find good states recruit other searches to join them
 $\Rightarrow$ information is shared among k search threads
 - Lack of diversity: **quite often, all k states end up in the same local hill**
- $\Rightarrow$ **Stochastic Local Beam**: choose k successors randomly, with probability proportional to state success.

Resembles natural selection with asexual reproduction:

the successors (offspring) of a state (organism) populate the next generation according to its value (fitness), with a random component.

Local Beam Search

Local Beam Search

Idea: **keep track of k states instead of one**

- Initially: k random states
- Step:
 - determine all successors of k states
 - if any of successors is goal $\Rightarrow$ finished
 - else select k best from successors**
- Different from k searches run in parallel:
 - searches that find good states recruit other searches to join them
 $\Rightarrow$ information is shared among k search threads
- Lack of diversity: **quite often, all k states end up in the same local hill**

$\Rightarrow$ **Stochastic Local Beam**: choose k successors randomly,
with probability proportional to state success.

Resembles natural selection with asexual reproduction:

the successors (offspring) of a state (organism) populate the next generation according to its value (fitness), with a random component.

Local Beam Search

Local Beam Search

Idea: **keep track of k states instead of one**

- Initially: k random states
- Step:
 - 1 determine all successors of k states
 - 2 if any of successors is goal $\Rightarrow$ finished
 - 3 **else select k best from successors**
- Different from k searches run in parallel:
 - searches that find good states recruit other searches to join them
 $\Rightarrow$ information is shared among k search threads
- Lack of diversity: **quite often, all k states end up in the same local hill**

$\Rightarrow$ **Stochastic Local Beam**: choose k successors randomly,
with probability proportional to state success.

Resembles natural selection with asexual reproduction:

the successors (offspring) of a state (organism) populate the next generation according to its value (fitness), with a random component.

Genetic Algorithms

- Variant of local beam search: **successor states generated by combining two parent states** (rather than one single state)
 - States represented as strings over a finite alphabet (e.g. $\{0, 1\}$)
- Initially: pick k random states
- Step:
 - parent states are rated according to a fitness function
 - k parent pairs are selected at random for reproduction, with probability increasing with their fitness
 - for each parent pair
 - generate a new state by combining the two parents
- Ends when some state is fit enough (or timeout)
- **Many algorithm variants available**

Resembles natural selection, with sexual reproduction

Genetic Algorithms

- Variant of local beam search: **successor states generated by combining two parent states** (rather than one single state)
 - States represented as strings over a finite alphabet (e.g. $\{0, 1\}$)
- Initially: pick k random states
- Step:
 - 1 parent states are rated according to a **fitness function**
 - 2 k parent pairs are selected at random for reproduction, **with probability increasing with their fitness**
 - gender and monogamy not considered
 - 3 for each parent pair
 - 1 a **crossover point** is chosen randomly
 - 2 a **new state** is created by crossing over the parent strings
 - 3 the offspring state is subject to (low-probability) random mutation
- Ends when some state is fit enough (or timeout)
- **Many algorithm variants available**

Resembles natural selection, with **sexual reproduction**

Genetic Algorithms

- Variant of local beam search: **successor states generated by combining two parent states** (rather than one single state)
 - States represented as strings over a finite alphabet (e.g. $\{0, 1\}$)
- Initially: pick k random states
- Step:
 - 1 parent states are rated according to a **fitness function**
 - 2 k parent pairs are selected at random for reproduction, **with probability increasing with their fitness**
 - gender and monogamy not considered
 - 3 for each parent pair
 - 1 a crossover point is chosen randomly
 - 2 a new state is created by crossing over the parent strings
 - 3 the offspring state is subject to (low-probability) random mutation
- Ends when some state is fit enough (or timeout)
- **Many algorithm variants available**

Resembles natural selection, with **sexual reproduction**

Genetic Algorithms

- Variant of local beam search: **successor states generated by combining two parent states** (rather than one single state)
 - States represented as strings over a finite alphabet (e.g. $\{0, 1\}$)
- Initially: pick k random states
- Step:
 - 1 parent states are rated according to a **fitness function**
 - 2 k parent pairs are selected at random for reproduction, **with probability increasing with their fitness**
 - gender and monogamy not considered
 - 3 for each parent pair
 - 1 a **crossover point** is chosen randomly
 - 2 **a new state is created** by crossing over the parent strings
 - 3 the offspring state is subject to (low-probability) random mutation
- Ends when some state is fit enough (or timeout)
- Many algorithm variants available

Resembles natural selection, with **sexual reproduction**

Genetic Algorithms

- Variant of local beam search: **successor states generated by combining two parent states** (rather than one single state)
 - States represented as strings over a finite alphabet (e.g. $\{0, 1\}$)
- Initially: pick k random states
- Step:
 - 1 parent states are rated according to a **fitness function**
 - 2 k parent pairs are selected at random for reproduction, **with probability increasing with their fitness**
 - gender and monogamy not considered
 - 3 for each parent pair
 - 1 a **crossover point** is chosen randomly
 - 2 **a new state is created** by crossing over the parent strings
 - 3 the offspring state is subject to (low-probability) random mutation
- Ends when some state is fit enough (or timeout)
- Many algorithm variants available

Resembles natural selection, with **sexual reproduction**

Genetic Algorithms

- Variant of local beam search: **successor states generated by combining two parent states** (rather than one single state)
 - States represented as strings over a finite alphabet (e.g. $\{0, 1\}$)
- Initially: pick k random states
- Step:
 - 1 parent states are rated according to a **fitness function**
 - 2 k parent pairs are selected at random for reproduction, **with probability increasing with their fitness**
 - gender and monogamy not considered
 - 3 for each parent pair
 - 1 a **crossover point** is chosen randomly
 - 2 **a new state is created** by crossing over the parent strings
 - 3 the offspring state is subject to (low-probability) random mutation
- Ends when some state is fit enough (or timeout)
- **Many algorithm variants available**

Resembles natural selection, with **sexual reproduction**

Genetic Algorithms

- Variant of local beam search: **successor states generated by combining two parent states** (rather than one single state)
 - States represented as strings over a finite alphabet (e.g. $\{0, 1\}$)
- Initially: pick k random states
- Step:
 - 1 parent states are rated according to a **fitness function**
 - 2 k parent pairs are selected at random for reproduction, **with probability increasing with their fitness**
 - gender and monogamy not considered
 - 3 for each parent pair
 - 1 a **crossover point** is chosen randomly
 - 2 **a new state is created** by crossing over the parent strings
 - 3 the offspring state is subject to (low-probability) random mutation
- Ends when some state is fit enough (or timeout)
- **Many algorithm variants available**

Resembles natural selection, with **sexual reproduction**

Genetic Algorithms

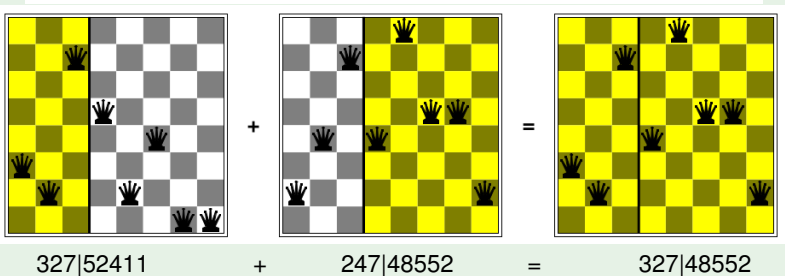
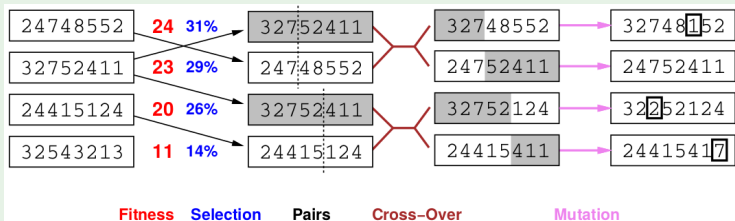
```
function GENETIC-ALGORITHM(population, fitness) returns an individual
  repeat
    weights  $\leftarrow$  WEIGHTED-BY(population, fitness) // list of fitness values for each individual
    population2  $\leftarrow$  empty list
    for  $i = 1$  to SIZE(population) do
      parent1, parent2  $\leftarrow$  WEIGHTED-RANDOM-CHOICES(population, weights, 2)
      child  $\leftarrow$  REPRODUCE(parent1, parent2)
      if (small random probability) then child  $\leftarrow$  MUTATE(child)
      add child to population2
    population  $\leftarrow$  population2
  until some individual is fit enough, or enough time has elapsed
  return the best individual in population, according to fitness
```

```
function REPRODUCE(parent1, parent2) returns an individual
   $n \leftarrow$  LENGTH(parent1)
   $c \leftarrow$  random number from 1 to  $n$ 
  return APPEND(SUBSTRING(parent1, 1,  $c$ ), SUBSTRING(parent2,  $c + 1$ ,  $n$ ))
```

Genetic Algorithms: Example

Example: 8-Queens

$state[i]$: (upward) position of the queen in i th column



Genetic Algorithms: Example

Example: String optimization

- States (**individuals**) are m -strings over a finite alphabet of size k
- The goal is maximizing the objective function f (**fitness**)
- We evolve a collection of states P (**population**) of size n
- Starting from P_0 , at each iteration i , we generate n “new” individuals:

- two parents are randomly sampled from P_{i-1} according to their fitness
- a crossover point in $s \in [1, m-1]$ is randomly selected

e.g. $[a_1|b_2b_3b_4 : 1/3, \quad a_1a_2|b_3b_4 : 1/3, \quad a_1a_2a_3|b_4 : 1/3]$

- Mutations to the offspring can happen with small probability p

e.g. $\forall i \in [1, m] \begin{cases} m(c_i) = c_i : 1 - p & \text{no mutation} \\ m(c_i) = (c_i + 1) \bmod k : p & \text{mutation} \end{cases}$

- The population P_i is selected:
 - No elitism: keep the “new” n individuals (i.e. P_{i-1} is discarded)
 - Elitism: retain the n best scoring individuals

Genetic Algorithms: Example

Example: String optimization

- States (**individuals**) are m -strings over a finite alphabet of size k
- The goal is maximizing the objective function f (**fitness**)
- We evolve a collection of states P (**population**) of size n
- Starting from P_0 , at each iteration i , we generate n “new” individuals:

- two parents are randomly sampled from P_{i-1} according to their fitness
- a crossover point in $s \in [1, m-1]$ is randomly selected

e.g. $[a_1|b_2b_3b_4 : 1/3, \quad a_1a_2|b_3b_4 : 1/3, \quad a_1a_2a_3|b_4 : 1/3]$

- Mutations to the offspring can happen with small probability p

e.g. $\forall i \in [1, m] \begin{cases} m(c_i) = c_i : 1 - p & \text{no mutation} \\ m(c_i) = (c_i + 1) \bmod k : p & \text{mutation} \end{cases}$

- The population P_i is selected:
 - No elitism: keep the “new” n individuals (i.e. P_{i-1} is discarded)
 - Elitism: retain the n best scoring individuals

Genetic Algorithms: Example

Example: String optimization

- States (**individuals**) are m -strings over a finite alphabet of size k
- The goal is maximizing the objective function f (**fitness**)
- We evolve a collection of states P (**population**) of size n
- Starting from P_0 , at each iteration i , we generate n “new” individuals:

- two parents are randomly sampled from P_{i-1} according to their fitness
- a crossover point in $s \in [1, m-1]$ is randomly selected

e.g. $[a_1|b_2b_3b_4 : 1/3, \quad a_1a_2|b_3b_4 : 1/3, \quad a_1a_2a_3|b_4 : 1/3]$

- Mutations to the offspring can happen with small probability p

e.g. $\forall i \in [1, m] \begin{cases} m(c_i) = c_i : 1 - p & \text{no mutation} \\ m(c_i) = (c_i + 1) \bmod k : p & \text{mutation} \end{cases}$

- The population P_i is selected:
 - No elitism: keep the “new” n individuals (i.e. P_{i-1} is discarded)
 - Elitism: retain the n best scoring individuals

Genetic Algorithms: Example

Example: String optimization

- States (**individuals**) are m -strings over a finite alphabet of size k
- The goal is maximizing the objective function f (**fitness**)
- We evolve a collection of states P (**population**) of size n
- Starting from P_0 , at each iteration i , we generate n “new” individuals:

- two parents are randomly sampled from P_{i-1} according to their fitness
- a crossover point in $s \in [1, m-1]$ is randomly selected

e.g. $[a_1|b_2b_3b_4 : 1/3, \quad a_1a_2|b_3b_4 : 1/3, \quad a_1a_2a_3|b_4 : 1/3]$

- Mutations to the offspring can happen with small probability p

e.g. $\forall i \in [1, m] \begin{cases} m(c_i) = c_i : 1 - p & \text{no mutation} \\ m(c_i) = (c_i + 1) \bmod k : p & \text{mutation} \end{cases}$

- The population P_i is selected:
 - No elitism: keep the “new” n individuals (i.e. P_{i-1} is discarded)
 - Elitism: retain the n best scoring individuals

Genetic Algorithms: Example

Example: String optimization

- States (**individuals**) are m -strings over a finite alphabet of size k
- The goal is maximizing the objective function f (**fitness**)
- We evolve a collection of states P (**population**) of size n
- Starting from P_0 , at each iteration i , we generate n “new” individuals:
 - two parents are randomly sampled from P_{i-1} according to their fitness
 - a crossover point in $s \in [1, m-1]$ is randomly selected

e.g. $[a_1|b_2b_3b_4 : 1/3, \quad a_1a_2|b_3b_4 : 1/3, \quad a_1a_2a_3|b_4 : 1/3]$

- Mutations to the offspring can happen with small probability p

e.g. $\forall i \in [1, m] \begin{cases} m(c_i) = c_i : 1 - p & \text{no mutation} \\ m(c_i) = (c_i + 1) \bmod k : p & \text{mutation} \end{cases}$

- The population P_i is selected:
 - No elitism: keep the “new” n individuals (i.e. P_{i-1} is discarded)
 - Elitism: retain the n best scoring individuals

Genetic Algorithms: Example

Example: String optimization

- States (**individuals**) are m -strings over a finite alphabet of size k
- The goal is maximizing the objective function f (**fitness**)
- We evolve a collection of states P (**population**) of size n
- Starting from P_0 , at each iteration i , we generate n “new” individuals:
 - two parents are randomly sampled from P_{i-1} according to their fitness
 - a crossover point in $s \in [1, m - 1]$ is randomly selected

e.g. $[a_1|b_2b_3b_4 : 1/3, \quad a_1a_2|b_3b_4 : 1/3, \quad a_1a_2a_3|b_4 : 1/3]$

- Mutations to the offspring can happen with small probability p

e.g. $\forall i \in [1, m] \begin{cases} m(c_i) = c_i : 1 - p & \text{no mutation} \\ m(c_i) = (c_i + 1) \bmod k : p & \text{mutation} \end{cases}$

- The population P_i is selected:
 - No elitism: keep the “new” n individuals (i.e. P_{i-1} is discarded)
 - Elitism: retain the n best scoring individuals

Genetic Algorithms: Example

Example: String optimization

- States (**individuals**) are m -strings over a finite alphabet of size k
- The goal is maximizing the objective function f (**fitness**)
- We evolve a collection of states P (**population**) of size n
- Starting from P_0 , at each iteration i , we generate n “new” individuals:
 - two parents are randomly sampled from P_{i-1} according to their fitness
 - a crossover point in $s \in [1, m - 1]$ is randomly selected

e.g. $[a_1|b_2b_3b_4 : 1/3, \quad a_1a_2|b_3b_4 : 1/3, \quad a_1a_2a_3|b_4 : 1/3]$

- Mutations to the offspring can happen with small probability p

e.g. $\forall i \in [1, m] \begin{cases} m(c_i) = c_i : 1 - p & \text{no mutation} \\ m(c_i) = (c_i + 1) \bmod k : p & \text{mutation} \end{cases}$

- The population P_i is selected:
 - No elitism: keep the “new” n individuals (i.e. P_{i-1} is discarded)
 - Elitism: retain the n best scoring individuals

Genetic Algorithms: Example

Example: String optimization

- States (**individuals**) are m -strings over a finite alphabet of size k
- The goal is maximizing the objective function f (**fitness**)
- We evolve a collection of states P (**population**) of size n
- Starting from P_0 , at each iteration i , we generate n “new” individuals:
 - two parents are randomly sampled from P_{i-1} according to their fitness
 - a crossover point in $s \in [1, m - 1]$ is randomly selected

e.g. $[a_1|b_2b_3b_4 : 1/3, \quad a_1a_2|b_3b_4 : 1/3, \quad a_1a_2a_3|b_4 : 1/3]$

- Mutations to the offspring can happen with small probability p

e.g. $\forall i \in [1, m] \begin{cases} m(c_i) = c_i : 1 - p & \text{no mutation} \\ m(c_i) = (c_i + 1) \bmod k : p & \text{mutation} \end{cases}$

- The population P_i is selected:
 - No elitism: keep the “new” n individuals (i.e. P_{i-1} is discarded)
 - Elitism: retain the n best scoring individuals

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = []$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = []$$

first $\sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{5}{9})$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = []$$

first $\sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{5}{9})$

$\rightarrow p_3^{(0)}$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(7)$$

$$p_2^{(0)} = 0002(1)$$

$$p_3^{(0)} = 1202(7)$$

$$p_4^{(0)} = 0012(3)]$$

$$P_1 = []$$

first $\sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{5}{9})$

second $\sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = 1)$

$\rightarrow p_3^{(0)}$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(7)$$

$$p_2^{(0)} = 0002(1)$$

$$p_3^{(0)} = 1202(7)$$

$$p_4^{(0)} = 0012(3)]$$

$$P_1 = []$$

first $\sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{5}{9})$

$\rightarrow p_3^{(0)}$

second $\sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = 1)$

$\rightarrow p_4^{(0)}$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(7)$$

$$p_2^{(0)} = 0002(1)$$

$$p_3^{(0)} = 1202(7)$$

$$p_4^{(0)} = 0012(3)]$$

$$P_1 = []$$

$$\text{first} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{5}{9})$$

$$\rightarrow p_3^{(0)}$$

$$\text{second} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = 1)$$

$$\rightarrow p_4^{(0)}$$

$$s \sim [1 : 1/3, 2 : 1/3, 3 : 1/3] (c = \frac{1}{2})$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = []$$

$$\textit{first} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{5}{9})$$

$$\rightarrow p_3^{(0)}$$

$$\textit{second} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = 1)$$

$$\rightarrow p_4^{(0)}$$

$$s \sim [1 : 1/3, 2 : 1/3, 3 : 1/3] (c = \frac{1}{2})$$

$$\rightarrow o = 12|12$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = []$$

$$\textit{first} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{5}{9})$$

$$\rightarrow p_3^{(0)}$$

$$\textit{second} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = 1)$$

$$\rightarrow p_4^{(0)}$$

$$s \sim [1 : 1/3, 2 : 1/3, 3 : 1/3] (c = \frac{1}{2})$$

$$\rightarrow o = 12|12$$

$$\textit{mut}_i \sim [\text{No} : 9/10, \text{Yes} : 1/10] (\mathbf{c} = \dots, \frac{1}{3}, \frac{5}{9}, \textcolor{blue}{1}, \frac{1}{2}, \dots),$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = []$$

$$\textit{first} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{5}{9})$$

$$\rightarrow p_3^{(0)}$$

$$\textit{second} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = 1)$$

$$\rightarrow p_4^{(0)}$$

$$s \sim [1 : 1/3, 2 : 1/3, 3 : 1/3] (c = \frac{1}{2})$$

$$\rightarrow o = 12|12$$

$$\textit{mut}_i \sim [\text{No} : 9/10, \text{Yes} : 1/10] (\mathbf{c} = \dots, \frac{1}{3}, \frac{5}{9}, \textcolor{blue}{1}, \frac{1}{2}, \dots),$$

$$\rightarrow \textit{mut}(o) = 12\textcolor{blue}{2}2$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11})]$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11})]$$

$$\textit{first} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{3})$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11})]$$

first $\sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{3})$

$\rightarrow p_1^{(0)}$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11})]$$

first $\sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{3})$

second $\sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{5}{9})$

$\rightarrow p_1^{(0)}$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11})]$$

$$\textit{first} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{3})$$

$$\rightarrow p_1^{(0)}$$

$$\textit{second} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{5}{9})$$

$$\rightarrow p_3^{(0)}$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})]$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11})]$$

$$\textit{first} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{3})$$

$$\rightarrow p_1^{(0)}$$

$$\textit{second} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{5}{9})$$

$$\rightarrow p_3^{(0)}$$

$$s \sim [1 : 1/3, 2 : 1/3, 3 : 1/3] (c = 1)$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11})]$$

$$\textit{first} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{3})$$

$$\rightarrow p_1^{(0)}$$

$$\textit{second} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{5}{9})$$

$$\rightarrow p_3^{(0)}$$

$$s \sim [1 : 1/3, 2 : 1/3, 3 : 1/3] (c = 1)$$

$$\rightarrow o = 012|2$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11})]$$

$$\textit{first} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{3})$$

$$\rightarrow p_1^{(0)}$$

$$\textit{second} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{5}{9})$$

$$\rightarrow p_3^{(0)}$$

$$s \sim [1 : 1/3, 2 : 1/3, 3 : 1/3] (c = 1)$$

$$\rightarrow o = 012|2$$

$$\textit{mut}_i \sim [\text{No} : 9/10, \text{Yes} : 1/10] (\mathbf{c} = \dots, \frac{1}{2}, \frac{1}{3}, \frac{5}{9}, \textcolor{blue}{1}, \dots),$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11})]$$

$$\textit{first} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{3})$$

$$\rightarrow p_1^{(0)}$$

$$\textit{second} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{5}{9})$$

$$\rightarrow p_3^{(0)}$$

$$s \sim [1 : 1/3, 2 : 1/3, 3 : 1/3] (c = 1)$$

$$\rightarrow o = 012|2$$

$$\textit{mut}_i \sim [\text{No} : 9/10, \text{Yes} : 1/10] (\mathbf{c} = \dots, \frac{1}{2}, \frac{1}{3}, \frac{5}{9}, \textcolor{blue}{1}, \dots),$$

$$\rightarrow \textit{mut}(o) = 012\textcolor{blue}{0}$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11}), 0120(\textcolor{red}{11})]$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11}), 0120(\textcolor{red}{11})]$$

$$\textit{first} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{2})$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11}), 0120(\textcolor{red}{11})]$$

$$\textit{first} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{2})$$

$$\rightarrow p_3^{(0)}$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11}), 0120(\textcolor{red}{11})]$$

$$\textit{first} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{2})$$

$$\rightarrow p_3^{(0)}$$

$$\textit{second} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{3})$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11}), 0120(\textcolor{red}{11})]$$

$$\textit{first} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{2})$$

$$\rightarrow p_3^{(0)}$$

$$\textit{second} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{3})$$

$$\rightarrow p_1^{(0)}$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(7)$$

$$p_2^{(0)} = 0002(1)$$

$$p_3^{(0)} = 1202(7)$$

$$p_4^{(0)} = 0012(3)]$$

$$P_1 = [1222(11), 0120(11)]$$

$$\text{first} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{2})$$

$$\rightarrow p_3^{(0)}$$

$$\text{second} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{3})$$

$$\rightarrow p_1^{(0)}$$

$$s \sim [1 : 1/3, 2 : 1/3, 3 : 1/3] (c = \frac{5}{9})$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11}), 0120(\textcolor{red}{11})]$$

$$\textit{first} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{2})$$

$$\rightarrow p_3^{(0)}$$

$$\textit{second} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{3})$$

$$\rightarrow p_1^{(0)}$$

$$s \sim [1 : 1/3, 2 : 1/3, 3 : 1/3] (c = \frac{5}{9})$$

$$\rightarrow o = 12|22$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11}), 0120(\textcolor{red}{11})]$$

$$\textit{first} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{2}) \rightarrow p_3^{(0)}$$

$$\textit{second} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{3}) \rightarrow p_1^{(0)}$$

$$s \sim [1 : 1/3, 2 : 1/3, 3 : 1/3] (c = \frac{5}{9}) \rightarrow o = 12|22$$

$$\textit{mut}_i \sim [\text{No} : 9/10, \text{Yes} : 1/10] (c = \dots, \textcolor{blue}{1}, \frac{1}{2}, \frac{1}{3}, \frac{5}{9}, \dots),$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11}), 0120(\textcolor{red}{11})]$$

$$\textit{first} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{2}) \rightarrow p_3^{(0)}$$

$$\textit{second} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{3}) \rightarrow p_1^{(0)}$$

$$s \sim [1 : 1/3, 2 : 1/3, 3 : 1/3] (c = \frac{5}{9}) \rightarrow o = 12|22$$

$$\textit{mut}_i \sim [\text{No} : 9/10, \text{Yes} : 1/10] (c = \dots, \textcolor{blue}{1}, \frac{1}{2}, \frac{1}{3}, \frac{5}{9}, \dots), \rightarrow \textit{mut}(o) = \textcolor{blue}{2}222$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11}), 0120(\textcolor{red}{11}), 2222(\textcolor{red}{13})]$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11}), 0120(\textcolor{red}{11}), 2222(\textcolor{red}{13})]$$

first $\sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = 1)$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11}), 0120(\textcolor{red}{11}), 2222(\textcolor{red}{13})]$$

first $\sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = 1)$

$\rightarrow p_4^{(0)}$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11}), 0120(\textcolor{red}{11}), 2222(\textcolor{red}{13})]$$

first $\sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = 1)$

second $\sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{2})$

$\rightarrow p_4^{(0)}$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11}), 0120(\textcolor{red}{11}), 2222(\textcolor{red}{13})]$$

first $\sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = 1)$

$\rightarrow p_4^{(0)}$

second $\sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{2})$

$\rightarrow p_3^{(0)}$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11}), 0120(\textcolor{red}{11}), 2222(\textcolor{red}{13})]$$

$$\textit{first} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = 1)$$

$$\rightarrow p_4^{(0)}$$

$$\textit{second} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{2})$$

$$\rightarrow p_3^{(0)}$$

$$s \sim [1 : 1/3, 2 : 1/3, 3 : 1/3] (c = \frac{1}{3})$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11}), 0120(\textcolor{red}{11}), 2222(\textcolor{red}{13})]$$

$$\textit{first} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = 1)$$

$$\rightarrow p_4^{(0)}$$

$$\textit{second} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{2})$$

$$\rightarrow p_3^{(0)}$$

$$s \sim [1 : 1/3, 2 : 1/3, 3 : 1/3] (c = \frac{1}{3})$$

$$\rightarrow o = 0|202$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(7)$$

$$p_2^{(0)} = 0002(1)$$

$$p_3^{(0)} = 1202(7)$$

$$p_4^{(0)} = 0012(3)]$$

$$P_1 = [1222(11), 0120(11), 2222(13)]$$

$$\text{first} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = 1) \rightarrow p_4^{(0)}$$

$$\text{second} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{2}) \rightarrow p_3^{(0)}$$

$$s \sim [1 : 1/3, 2 : 1/3, 3 : 1/3] (c = \frac{1}{3}) \rightarrow o = 0|202$$

$$\text{mut}_i \sim [\text{No} : 9/10, \text{Yes} : 1/10] (c = [\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]),$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11}), 0120(\textcolor{red}{11}), 2222(\textcolor{red}{13})]$$

$$\textit{first} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = 1) \rightarrow p_4^{(0)}$$

$$\textit{second} \sim [1 : 7/18, 2 : 1/18, 3 : 7/18, 4 : 3/18] (c = \frac{1}{2}) \rightarrow p_3^{(0)}$$

$$s \sim [1 : 1/3, 2 : 1/3, 3 : 1/3] (c = \frac{1}{3}) \rightarrow o = 0|202$$

$$\textit{mut}_i \sim [\text{No} : 9/10, \text{Yes} : 1/10] (\mathbf{c} = [\frac{5}{9}, \textcolor{blue}{1}, \frac{1}{2}, \frac{1}{3}]), \rightarrow \textit{mut}(o) = 0\textcolor{blue}{0}02$$

G.A. Example [cont.]

Example: String optimization

Alphabet: $\mathcal{A} = [0, 1, 2]$ (ternary)

Fitness:

$$f(s_1 s_2 s_3 s_4) = 5 + 2(s_1 + s_2) + 2(s_3 - s_4)$$

Prob. mutation: 0.1 (indep. for each character)

Elitism: No. **Choices:** $[\frac{5}{9}, 1, \frac{1}{2}, \frac{1}{3}]$

Initial:

$$P_0 = [p_1^{(0)} = 0122(\textcolor{red}{7})$$

$$p_2^{(0)} = 0002(\textcolor{red}{1})$$

$$p_3^{(0)} = 1202(\textcolor{red}{7})$$

$$p_4^{(0)} = 0012(\textcolor{red}{3})]$$

$$P_1 = [1222(\textcolor{red}{11}), 0120(\textcolor{red}{11}), 2222(\textcolor{red}{13}), 0002(\textcolor{red}{1})]$$

Genetic Algorithms: Intuitions, Pros & Cons

Intuitions

- **Selection** drives the population toward high fitness
- **Crossover** combines good parts from good solutions (but it might achieve the opposite effect)
- **Mutation** introduces diversity

Pros & Cons

- Pros:
 - extremely simple
 - general purpose
 - tractable theoretical models
- Cons:
 - not completely understood
 - good coding is crucial (e.g., Gray codes for numbers)
 - too simple genetic operators

Widespread impact on optimization problems, i.e. circuit layout and job-shop scheduling

Genetic Algorithms: Intuitions, Pros & Cons

Intuitions

- **Selection** drives the population toward high fitness
- **Crossover** combines good parts from good solutions (but it might achieve the opposite effect)
- **Mutation** introduces diversity

Pros & Cons

- Pros:
 - extremely simple
 - general purpose
 - tractable theoretical models
- Cons:
 - not completely understood
 - good coding is crucial (e.g., Gray codes for numbers)
 - too simple genetic operators

Widespread impact on optimization problems, i.e. circuit layout and job-shop scheduling

Genetic Algorithms: Intuitions, Pros & Cons

Intuitions

- **Selection** drives the population toward high fitness
- **Crossover** combines good parts from good solutions (but it might achieve the opposite effect)
- **Mutation** introduces diversity

Pros & Cons

- Pros:
 - extremely simple
 - general purpose
 - tractable theoretical models
- Cons:
 - not completely understood
 - good coding is crucial (e.g., Gray codes for numbers)
 - too simple genetic operators

Widespread impact on optimization problems, i.e. circuit layout and job-shop scheduling

Genetic Algorithms: Intuitions, Pros & Cons

Intuitions

- **Selection** drives the population toward high fitness
- **Crossover** combines good parts from good solutions (but it might achieve the opposite effect)
- **Mutation** introduces diversity

Pros & Cons

- Pros:
 - extremely simple
 - general purpose
 - tractable theoretical models
- Cons:
 - not completely understood
 - good coding is crucial (e.g., Gray codes for numbers)
 - too simple genetic operators

Widespread impact on optimization problems, i.e. circuit layout and job-shop scheduling

Genetic Algorithms: Intuitions, Pros & Cons

Intuitions

- **Selection** drives the population toward high fitness
- **Crossover** combines good parts from good solutions (but it might achieve the opposite effect)
- **Mutation** introduces diversity

Pros & Cons

- Pros:
 - extremely simple
 - general purpose
 - tractable theoretical models
- Cons:
 - not completely understood
 - good coding is crucial (e.g., Gray codes for numbers)
 - too simple genetic operators

Widespread impact on optimization problems, i.e. circuit layout and job-shop scheduling

Genetic Algorithms: Intuitions, Pros & Cons

Intuitions

- **Selection** drives the population toward high fitness
- **Crossover** combines good parts from good solutions (but it might achieve the opposite effect)
- **Mutation** introduces diversity

Pros & Cons

- Pros:
 - extremely simple
 - general purpose
 - tractable theoretical models
- Cons:
 - not completely understood
 - good coding is crucial (e.g., Gray codes for numbers)
 - too simple genetic operators

Widespread impact on optimization problems, i.e. circuit layout and job-shop scheduling

Genetic Algorithms: Intuitions, Pros & Cons

Intuitions

- **Selection** drives the population toward high fitness
- **Crossover** combines good parts from good solutions (but it might achieve the opposite effect)
- **Mutation** introduces diversity

Pros & Cons

- Pros:
 - extremely simple
 - general purpose
 - tractable theoretical models
- Cons:
 - not completely understood
 - good coding is crucial (e.g., Gray codes for numbers)
 - too simple genetic operators

Widespread impact on optimization problems, i.e. circuit layout and job-shop scheduling

Genetic Algorithms: Intuitions, Pros & Cons

Intuitions

- **Selection** drives the population toward high fitness
- **Crossover** combines good parts from good solutions (but it might achieve the opposite effect)
- **Mutation** introduces diversity

Pros & Cons

- Pros:
 - extremely simple
 - general purpose
 - tractable theoretical models
- Cons:
 - not completely understood
 - good coding is crucial (e.g., Gray codes for numbers)
 - too simple genetic operators

Widespread impact on optimization problems, i.e. circuit layout and job-shop scheduling

- 1 Local Search and Optimization
 - General Ideas
 - Hill-Climbing
 - Simulated Annealing
 - Local Beam Search & Genetic Algorithms
- 2 Search with Nondeterministic Actions
- 3 Search with Partial or No Observations (Deterministic/Nondeterministic Actions)
 - Search with No Observations
 - Search with Partial Observations
- 4 Online Search (aka Exploration)

Recall: Generalities

- So far we address a single category of problems:
 - ① observable,
 - ② deterministic,
 - ③ with known environment,
 - ④ s.t. the solution is a sequence of actions.
- What happens when these assumptions are relaxed?
- In order we will:
 - release condition 4 $\implies$ local search
 - release condition 2 $\implies$ search with non-deterministic actions
 - release condition 1 $\implies$ search with no observability or with partial observability
 - release condition 3 $\implies$ online search

Generalities (cont.)

- Assumptions so far (see ch. 2 and 3):

- the environment is deterministic
- the environment is fully observable
- the agent knows the effects of each action

⇒ The agent does not need perception:

- can calculate which state results from any sequence of actions
- always knows which state it is in
- If one of the above does not hold, then percepts are useful
 - the future percepts cannot be determined in advance
 - the agent's future actions will depend on future percepts
- Solution: not a sequence but a contingency plan (aka conditional plan, strategy)
 - specifies the actions depending on what percepts are received
- We analyze first the case of nondeterministic environments

Generalities (cont.)

- Assumptions so far (see ch. 2 and 3):

- the environment is deterministic
- the environment is fully observable
- the agent knows the effects of each action

⇒ The agent does not need perception:

- can calculate which state results from any sequence of actions
- always knows which state it is in
- If one of the above does not hold, then percepts are useful
 - the future percepts cannot be determined in advance
 - the agent's future actions will depend on future percepts
- Solution: not a sequence but a contingency plan (aka conditional plan, strategy)
 - specifies the actions depending on what percepts are received
- We analyze first the case of nondeterministic environments

Generalities (cont.)

- Assumptions so far (see ch. 2 and 3):

- the environment is deterministic
- the environment is fully observable
- the agent knows the effects of each action

⇒ The agent does not need perception:

- can calculate which state results from any sequence of actions
- always knows which state it is in
- If one of the above does not hold, then percepts are useful
 - the future percepts cannot be determined in advance
 - the agent's future actions will depend on future percepts
- Solution: not a sequence but a contingency plan (aka conditional plan, strategy)
 - specifies the actions depending on what percepts are received
- We analyze first the case of nondeterministic environments

Generalities (cont.)

- Assumptions so far (see ch. 2 and 3):

- the environment is deterministic
- the environment is fully observable
- the agent knows the effects of each action

⇒ The agent does not need perception:

- can calculate which state results from any sequence of actions
- always knows which state it is in
- If one of the above does not hold, then percepts are useful
 - the future percepts cannot be determined in advance
 - the agent's future actions will depend on future percepts
- Solution: not a sequence but a contingency plan (aka conditional plan, strategy)
 - specifies the actions depending on what percepts are received
- We analyze first the case of nondeterministic environments

Example: The Erratic Vacuum Cleaner

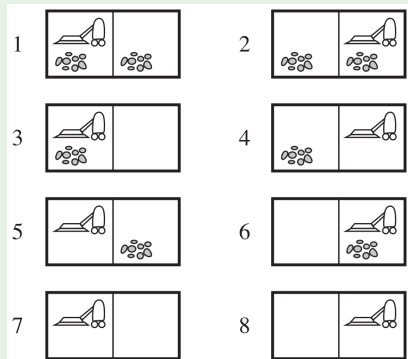
Erratic Vacuum-Cleaner Example

- actions: Left, Right, Suck
- goal: A and B cleaned (states 7, 8)
- if environment is observable, deterministic, and completely known $\implies$ solvable by search algos
- ex: if initially in 1, then [suck,right,suck] leads to 8: [1,5,6,8]

- Nondeterministic version (erratic vacuum cleaner):

- if dirty square: cleans the square, sometimes cleans also the other square. Ex: $1 \xrightarrow{\text{suck}} \{5, 7\}$
- if clean square: sometimes deposits dirt on the carpet

Ex: $5 \xrightarrow{\text{suck}} \{1, 5\}$



(© S. Russell & P. Norwig, AIMA)

Example: The Erratic Vacuum Cleaner

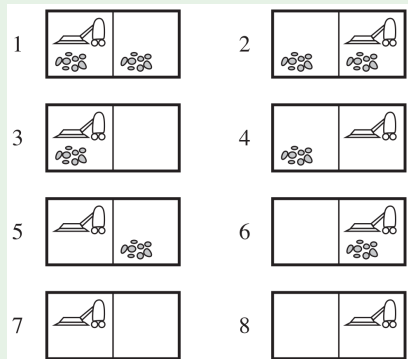
Erratic Vacuum-Cleaner Example

- actions: Left, Right, Suck
- goal: A and B cleaned (states 7, 8)
- if environment is observable, deterministic, and completely known $\implies$ solvable by search algos
- ex: if initially in 1, then [suck,right,suck] leads to 8: [1,5,6,8]

- Nondeterministic version (erratic vacuum cleaner):

- if dirty square: cleans the square, sometimes cleans also the other square. Ex: $1 \xrightarrow{\text{suck}} \{5, 7\}$
- if clean square: sometimes deposits dirt on the carpet

Ex: $5 \xrightarrow{\text{suck}} \{1, 5\}$



(© S. Russell & P. Norwig, AIMA)

Searching with Nondeterministic Actions

Generalized notion of transition model

- $\text{RESULTS}(s,A)$ returns a set of possible outcomes states
 - Ex: $\text{RESULTS}(1,\text{SUCK})=\{5,7\}$, $\text{RESULTS}(5,\text{SUCK})=\{1,5\}$, ...
- A solution is a contingency plan (aka conditional plan, strategy)
 - contains nested conditions on future percepts (if-then-else, case-switch, ...)
 - Ex: from state 1 we can act the following contingency plan:
[SUCK, IF STATE = 5 THEN [RIGHT, SUCK] ELSE []]
- Can cause loops (see later)

Remark

In practice, we don't reason on states, rather on state variable values:
[Suck; if B.Dirty then [Right, Suck] else []]

Searching with Nondeterministic Actions

Generalized notion of transition model

- $RESULTS(S,A)$ returns a set of possible outcomes states
 - Ex: $RESULTS(1,SUCK)=\{5,7\}$, $RESULTS(5,SUCK)=\{1,5\}$, ...
- A solution is a contingency plan (aka conditional plan, strategy)
 - contains nested conditions on future percepts (if-then-else, case-switch, ...)
 - Ex: from state 1 we can act the following contingency plan:
[SUCK, IF STATE = 5 THEN [RIGHT, SUCK] ELSE []]
- Can cause loops (see later)

Remark

In practice, we don't reason on states, rather on state variable values:
[Suck; if B.Dirty then [Right, Suck] else []]

Searching with Nondeterministic Actions

Generalized notion of transition model

- $RESULTS(S,A)$ returns a set of possible outcomes states
 - Ex: $RESULTS(1,SUCK)=\{5,7\}$, $RESULTS(5,SUCK)=\{1,5\}$, ...
- A solution is a contingency plan (aka conditional plan, strategy)
 - contains nested conditions on future percepts (if-then-else, case-switch, ...)
 - Ex: from state 1 we can act the following contingency plan:
[SUCK, IF STATE = 5 THEN [RIGHT, SUCK] ELSE []]
- Can cause loops (see later)

Remark

In practice, we don't reason on states, rather on state variable values:
[Suck; if B.Dirty then [Right, Suck] else []]

Searching with Nondeterministic Actions

Generalized notion of transition model

- $\text{RESULTS}(S,A)$ returns a set of possible outcomes states
 - Ex: $\text{RESULTS}(1,\text{SUCK})=\{5,7\}$, $\text{RESULTS}(5,\text{SUCK})=\{1,5\}$, ...
- A solution is a contingency plan (aka conditional plan, strategy)
 - contains nested conditions on future percepts (if-then-else, case-switch, ...)
 - Ex: from state 1 we can act the following contingency plan:
[SUCK, IF STATE = 5 THEN [RIGHT, SUCK] ELSE []]
- Can cause loops (see later)

Remark

In practice, we don't reason on states, rather on state variable values:
[Suck; if B.Dirty then [Right, Suck] else []]

Searching with Nondeterministic Actions [cont.]

And-Or Search Trees

- In a **deterministic environment**, we branch on **agent's choices**
 - ⇒ **OR nodes**, hence **OR search trees**
 - **OR nodes correspond to states**
- In a **nondeterministic environment**, we branch also on **outcomes for each action**
 - the agent has to handle all such outcomes
 - ⇒ **AND nodes**, hence **AND-OR search trees**
 - **AND nodes correspond to actions**
 - leaf nodes are **goal, dead-end or loop OR nodes**
- A **solution** for an AND-OR search problem is a subtree s.t.:
 - has a goal node at every leaf
 - specifies **one action** at each of its OR nodes
 - includes **all outcome branches** at each of its AND nodes

OR tree: AND-OR tree with 1 outcome each AND node (determinism)

Searching with Nondeterministic Actions [cont.]

And-Or Search Trees

- In a **deterministic environment**, we branch on **agent's choices**
 - ⇒ **OR nodes**, hence **OR search trees**
 - **OR nodes correspond to states**
- In a **nondeterministic environment**, we branch also on **outcomes for each action**
 - the agent has to handle all such outcomes
 - ⇒ **AND nodes**, hence **AND-OR search trees**
 - **AND nodes correspond to actions**
 - leaf nodes are **goal**, **dead-end** or **loop** OR nodes
- A **solution** for an AND-OR search problem is a subtree s.t.:
 - has a goal node at every leaf
 - specifies **one action** at each of its OR nodes
 - includes **all outcome branches** at each of its AND nodes

OR tree: AND-OR tree with 1 outcome each AND node (determinism)

Searching with Nondeterministic Actions [cont.]

And-Or Search Trees

- In a **deterministic environment**, we branch on **agent's choices**
 - ⇒ **OR nodes**, hence **OR search trees**
 - **OR nodes correspond to states**
- In a **nondeterministic environment**, we branch also on **outcomes for each action**
 - the agent has to handle all such outcomes
 - ⇒ **AND nodes**, hence **AND-OR search trees**
 - **AND nodes correspond to actions**
 - leaf nodes are **goal**, **dead-end** or **loop** OR nodes
- A **solution** for an AND-OR search problem is a subtree s.t.:
 - has a goal node at every leaf
 - specifies **one action** at each of its OR nodes
 - includes **all outcome branches** at each of its AND nodes

OR tree: AND-OR tree with 1 outcome each AND node (determinism)

Searching with Nondeterministic Actions [cont.]

And-Or Search Trees

- In a **deterministic environment**, we branch on **agent's choices**
 - ⇒ **OR nodes**, hence **OR search trees**
 - **OR nodes correspond to states**
- In a **nondeterministic environment**, we branch also on **outcomes for each action**
 - the agent has to handle all such outcomes
 - ⇒ **AND nodes**, hence **AND-OR search trees**
 - **AND nodes correspond to actions**
 - leaf nodes are **goal**, **dead-end** or **loop** OR nodes
- A **solution** for an AND-OR search problem is a subtree s.t.:
 - has a goal node at every leaf
 - specifies **one action** at each of its OR nodes
 - includes **all outcome branches** at each of its AND nodes

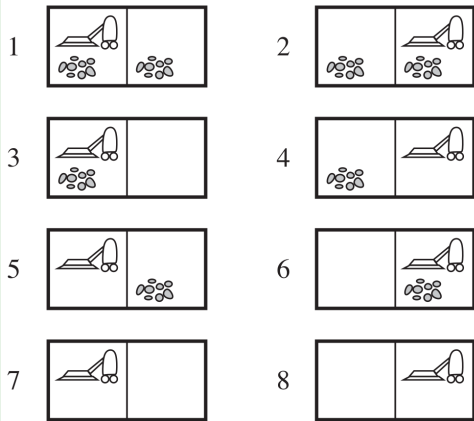
OR tree: AND-OR tree with 1 outcome each AND node (determinism)

And-Or Search Trees: Example

(Part of) And-Or Search Tree for Erratic Vacuum Cleaner Example.

Problem: Init: 1, Goal: 7,8.

Solution: [SUCK, IF STATE = 5 THEN [RIGHT, SUCK] ELSE []] (solid arcs)

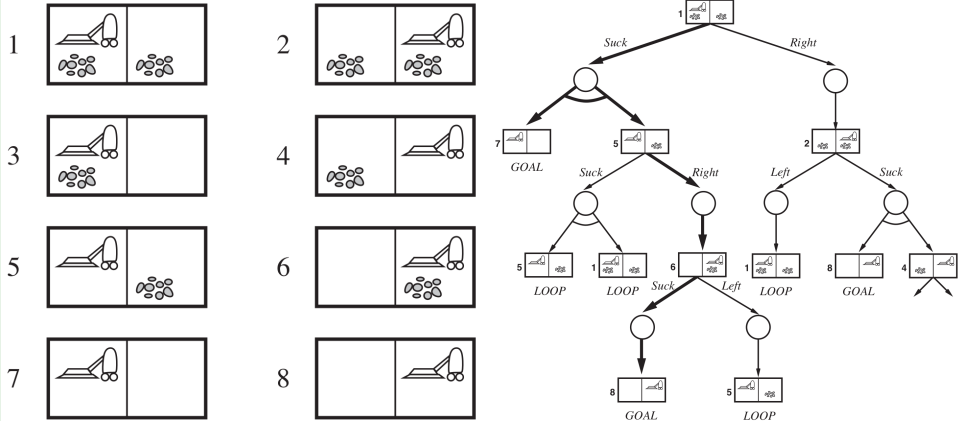


And-Or Search Trees: Example

(Part of) And-Or Search Tree for Erratic Vacuum Cleaner Example.

Problem: Init: 1, Goal: 7,8.

Solution: [SUCK, IF STATE = 5 THEN [RIGHT, SUCK] ELSE []] (solid arcs)



AND-OR Search

Recursive Depth-First (Tree-based) AND-OR Search

function AND-OR-SEARCH(*problem*) **returns** a conditional plan, or *failure*
return OR-SEARCH(*problem*, *problem*.INITIAL, [])

function OR-SEARCH(*problem*, *state*, *path*) **returns** a conditional plan, or *failure*
if *problem*.IS-GOAL(*state*) **then return** the empty plan
if IS-CYCLE(*state*, *path*) **then return** *failure*
for each *action* **in** *problem*.ACTIONS(*state*) **do**
 $plan \leftarrow$ AND-SEARCH(*problem*, RESULTS(*state*, *action*), [*state*] + [*path*])
 if $plan \neq failure$ **then return** [*action*] + [*plan*]
return *failure*

function AND-SEARCH(*problem*, *states*, *path*) **returns** a conditional plan, or *failure*
for each s_i **in** *states* **do**
 $plan_i \leftarrow$ OR-SEARCH(*problem*, s_i , *path*)
 if $plan_i = failure$ **then return** *failure*
return [**if** s_1 **then** $plan_1$ **else if** s_2 **then** $plan_2$ **else ... if** s_{n-1} **then** $plan_{n-1}$ **else** $plan_n$]

(© S. Russell & P. Norvig, AIMA)

Note: nested if-then-else can be rewritten as case-switch

AND-OR Search [cont.]

Recursive Depth-First (Tree-based) AND-OR Search

- Cycles: if the current state already occurs in the path $\implies$ failure
 - cycle detection like with ordinary DFS
 - does not mean “no solution”
 - means “if there is a non-cyclic solution, then it must be reachable from the earlier incarnation of the current state”
 $\implies$ the new incarnation can be discharged

$\implies$ **Complete** (if state space finite): every path must reach a goal, a dead-end or loop state

- Can be augmented with “explored” data structure for avoiding redundant branches (graph-based search)
- Implicitly Depth-First, but can also be explored by breadth-first or best-first method
 - e.g. A^* variant for AND-OR search available (see AIMA book)

AND-OR Search [cont.]

Recursive Depth-First (Tree-based) AND-OR Search

- Cycles: if the current state already occurs in the path $\implies$ failure
 - cycle detection like with ordinary DFS
 - does not mean “no solution”
 - means “if there is a non-cyclic solution, then it must be reachable from the earlier incarnation of the current state”
 $\implies$ the new incarnation can be discharged

$\implies$ **Complete** (if state space finite): every path must reach a goal, a dead-end or loop state

- Can be augmented with “explored” data structure for avoiding redundant branches (graph-based search)
- Implicitly Depth-First, but can also be explored by breadth-first or best-first method
 - e.g. A^* variant for AND-OR search available (see AIMA book)

AND-OR Search [cont.]

Recursive Depth-First (Tree-based) AND-OR Search

- Cycles: if the current state already occurs in the path $\implies$ failure
 - cycle detection like with ordinary DFS
 - does not mean “no solution”
 - means “if there is a non-cyclic solution, then it must be reachable from the earlier incarnation of the current state”
 $\implies$ the new incarnation can be discharged

$\implies$ **Complete** (if state space finite): every path must reach a goal, a dead-end or loop state

- Can be augmented with “explored” data structure for avoiding redundant branches (graph-based search)
- Implicitly Depth-First, but can also be explored by breadth-first or best-first method
 - e.g. A^* variant for AND-OR search available (see AIMA book)

AND-OR Search [cont.]

Recursive Depth-First (Tree-based) AND-OR Search

- Cycles: if the current state already occurs in the path $\implies$ failure
 - cycle detection like with ordinary DFS
 - does not mean “no solution”
 - means “if there is a non-cyclic solution, then it must be reachable from the earlier incarnation of the current state”
 $\implies$ the new incarnation can be discharged

$\implies$ **Complete** (if state space finite): every path must reach a goal, a dead-end or loop state

- Can be augmented with “explored” data structure for avoiding redundant branches (graph-based search)
- Implicitly Depth-First, but can also be explored by breadth-first or best-first method
 - e.g. A^* variant for AND-OR search available (see AIMA book)

AND-OR Search: Cyclic Solutions

- Some problems have no acyclic solutions
- A **cyclic plan** may be considered a **cyclic solution** provided that:
 - **every leaf is a goal state** (loop states not considered leaves), and
 - **a leaf is reachable from every point in the plan**
- Can be expressed by means of introducing
 - **labels**, and backward goto's to labels
 - **loop syntax** (e.g., while-do)

⇒ Executing a cyclic solution eventually reaches a goal,
provided that each outcome of a nondeterministic action eventually occurs

- Is this assumption reasonable?
- Yes, provided we distinguish:
(nondeterministic, observable) ≠ (deterministic, partially-observable)
- Ex: device may not always work $\neq$ device is broken (but we don't know it)

AND-OR Search: Cyclic Solutions

- Some problems have no acyclic solutions
- A **cyclic plan** may be considered a **cyclic solution** provided that:
 - **every leaf is a goal state** (loop states not considered leaves), and
 - **a leaf is reachable from every point in the plan**
- Can be expressed by means of introducing
 - **labels**, and backward goto's to labels
 - **loop syntax** (e.g., while-do)

⇒ Executing a cyclic solution eventually reaches a goal,
provided that each outcome of a nondeterministic action eventually occurs

- Is this assumption reasonable?
- Yes, provided we distinguish:
(nondeterministic, observable) ≠ (deterministic, partially-observable)
- Ex: device may not always work $\neq$ device is broken (but we don't know it)

AND-OR Search: Cyclic Solutions

- Some problems have no acyclic solutions
- A **cyclic plan** may be considered a **cyclic solution** provided that:
 - **every leaf is a goal state** (loop states not considered leaves), and
 - **a leaf is reachable from every point in the plan**
- Can be expressed by means of introducing
 - **labels**, and backward goto's to labels
 - **loop syntax** (e.g., while-do)

⇒ Executing a cyclic solution eventually reaches a goal,
provided that each outcome of a nondeterministic action eventually occurs

- Is this assumption reasonable?
- Yes, provided we distinguish:
 $\langle \text{nondeterministic, observable} \rangle \neq \langle \text{deterministic, partially-observable} \rangle$
- Ex: device may not always work $\neq$ device is broken (but we don't know it)

AND-OR Search: Cyclic Solutions

- Some problems have no acyclic solutions
- A **cyclic plan** may be considered a **cyclic solution** provided that:
 - **every leaf is a goal state** (loop states not considered leaves), and
 - **a leaf is reachable from every point in the plan**
- Can be expressed by means of introducing
 - **labels**, and backward goto's to labels
 - **loop syntax** (e.g., while-do)

⇒ Executing a cyclic solution eventually reaches a goal,
provided that each outcome of a nondeterministic action eventually occurs

- Is this assumption reasonable?
- Yes, provided we distinguish:
 $\langle \text{nondeterministic, observable} \rangle \neq \langle \text{deterministic, partially-observable} \rangle$
- Ex: **device may not always work** $\neq$ **device is broken (but we don't know it)**

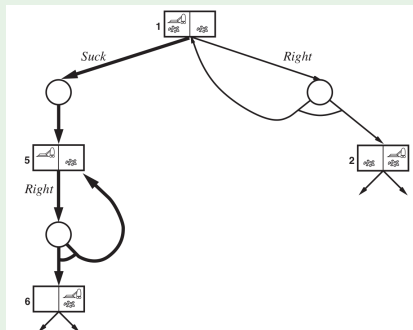
Cyclic Solution: Example

Example: Slippery Vacuum Cleaner

- Movement actions may fail: e.g., $Results(1, Right) = \{1, 2\}$

⇒ A cyclic solution

- Use labels: [Suck, L1 : Right, if State = 5 then L1 else Suck]
- Use cycles: [Suck, While State = 5 do Right, Suck]



(© S. Russell & P. Norwig, AIMA)

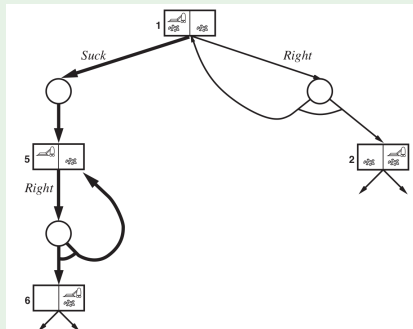
Cyclic Solution: Example

Example: Slippery Vacuum Cleaner

- Movement actions may fail: e.g., $Results(1, Right) = \{1, 2\}$

⇒ A cyclic solution

- Use labels: [Suck, L1 : Right, if State = 5 then L1 else Suck]
- Use cycles: [Suck, While State = 5 do Right, Suck]



(© S. Russell & P. Norwig, AIMA)

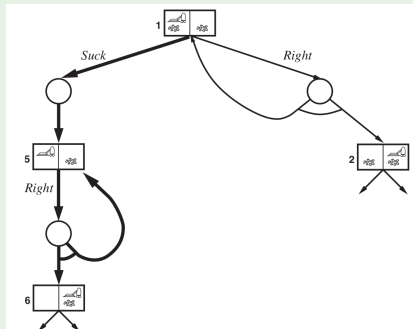
Cyclic Solution: Example

Example: Slippery Vacuum Cleaner

- Movement actions may fail: e.g., $Results(1, Right) = \{1, 2\}$

⇒ A cyclic solution

- Use labels: [Suck, L1 : Right, if State = 5 then L1 else Suck]
- Use cycles: [Suck, While State = 5 do Right, Suck]



(© S. Russell & P. Norwig, AIMA)

Exercises

- ➊ Consider the slippery vacuum cleaner problem s.t. initial is $\langle B, \langle \textit{dirty}, \textit{clean} \rangle \rangle$ and goal is $\langle A, \langle \textit{clean}, \textit{clean} \rangle \rangle$:
 - Using the AND-OR-tree search method, find a plan to achieve the goal.
 - Draw the AND-OR (sub)tree which produces the plan
 - Track explicitly all the recursive calls of AND-OR-SEARCH, OR-SEARCH, and AND-SEARCH, and their returned (sub)plans
- ➋ Do the same with a defective variant of the basic vacuum cleaner, in which “suck” may either succeed or not in making the current location clean.
- ➌ Do the same with other initial/goal state pairs

Exercises

- ➊ Consider the slippery vacuum cleaner problem s.t. initial is $\langle B, \langle \textit{dirty}, \textit{clean} \rangle \rangle$ and goal is $\langle A, \langle \textit{clean}, \textit{clean} \rangle \rangle$:
 - Using the AND-OR-tree search method, find a plan to achieve the goal.
 - Draw the AND-OR (sub)tree which produces the plan
 - Track explicitly all the recursive calls of AND-OR-SEARCH, OR-SEARCH, and AND-SEARCH, and their returned (sub)plans
- ➋ Do the same with a defective variant of the basic vacuum cleaner, in which “suck” may either succeed or not in making the current location clean.
- ➌ Do the same with other initial/goal state pairs

Exercises

- ➊ Consider the slippery vacuum cleaner problem s.t. initial is $\langle B, \langle \textit{dirty}, \textit{clean} \rangle \rangle$ and goal is $\langle A, \langle \textit{clean}, \textit{clean} \rangle \rangle$:
 - Using the AND-OR-tree search method, find a plan to achieve the goal.
 - Draw the AND-OR (sub)tree which produces the plan
 - Track explicitly all the recursive calls of AND-OR-SEARCH, OR-SEARCH, and AND-SEARCH, and their returned (sub)plans
- ➋ Do the same with a defective variant of the basic vacuum cleaner, in which “suck” may either succeed or not in making the current location clean.
- ➌ Do the same with other initial/goal state pairs

- 1 Local Search and Optimization
 - General Ideas
 - Hill-Climbing
 - Simulated Annealing
 - Local Beam Search & Genetic Algorithms
- 2 Search with Nondeterministic Actions
- 3 **Search with Partial or No Observations (Deterministic/Nondeterministic Actions)**
 - Search with No Observations
 - Search with Partial Observations
- 4 Online Search (aka Exploration)

Recall: Generalities

- So far we address a single category of problems:
 - 1 observable,
 - 2 deterministic,
 - 3 with known environment,
 - 4 s.t. the solution is a sequence of actions.
- What happens when these assumptions are relaxed?
- In order we will:
 - release condition 4 $\implies$ local search
 - release condition 2 $\implies$ search with non-deterministic actions
 - release condition 1 $\implies$ search with no observability or with partial observability
 - release condition 3 $\implies$ online search

Generalities

Partial Observability

- **Partial observability**: percepts do not capture the whole state
 - not all state variable values known $\Rightarrow$ corresponds to **a set of possible physical states**
- If the agent is in one of several possible physical states, then an action may lead to one of several possible outcomes, **even if the environment is deterministic**

Belief States

- **Belief state**: the agent's current belief about the possible physical states it might be in, given the previous sequence of actions and percepts
- If the agent has no partial observability, then its belief state is a single state (the state it knows it is in)
- If the agent has partial observability, then its belief state is a set of states (the states it might be in)
- If the belief state contains only one state, then the agent knows it is in that state
- 2^n possible belief states out of n possible physical states!

In practice, we reason in terms of (disjunctions of) **partial states** rather than of **sets of states**

- a **partial state** assigns values to a subset of state variables (e.g. $\langle ?, \langle ?, Clean \rangle \rangle = \{3, 4, 7, 8\}$)

Generalities

Partial Observability

- **Partial observability**: percepts do not capture the whole state
 - not all state variable values known $\implies$ corresponds to **a set of possible physical states**
- If the agent is in one of several possible physical states, then an action may lead to one of several possible outcomes, **even if the environment is deterministic**

Belief States

- **Belief state**: **the agent's current belief about the possible physical states it might be in**, given the previous sequence of actions and percepts
 - is a **set of physical states**: **the agent is in one of these states** (but does not know in which one)
 - contains the actual physical state the agent is in
 - ex: $\{1, 2\}$: the agent is either in state 1 or in state 2 (but it does not know in which one)
 - if the belief state contains only one state, then the agent knows it is in that state
- **2^n possible belief states out of n possible physical states!**

In practice, we reason in terms of (disjunctions of) **partial states** rather than of **sets of states**

- a **partial state** assigns values to a subset of state variables (e.g. $\langle ?, \langle ?, Clean \rangle \rangle = \{3, 4, 7, 8\}$)

Generalities

Partial Observability

- **Partial observability**: percepts do not capture the whole state
 - not all state variable values known $\implies$ corresponds to **a set of possible physical states**
- If the agent is in one of several possible physical states, then an action may lead to one of several possible outcomes, **even if the environment is deterministic**

Belief States

- **Belief state**: **the agent's current belief about the possible physical states it might be in**, given the previous sequence of actions and percepts
 - is a **set of physical states**: **the agent is in one of these states** (but does not know in which one)
 - contains the actual physical state the agent is in
 - ex: $\{1, 2\}$: **the agent is either in state 1 or in state 2** (but it does not know in which one)
 - if the belief state contains only one state, then the agent knows it is in that state
- **2^n possible belief states out of n possible physical states!**

In practice, we reason in terms of (disjunctions of) **partial states** rather than of **sets of states**

- a **partial state** assigns values to a subset of state variables (e.g. $\langle ?, \langle ?, \text{Clean} \rangle \rangle = \{3, 4, 7, 8\}$)

Generalities

Partial Observability

- **Partial observability**: percepts do not capture the whole state
 - not all state variable values known $\implies$ corresponds to **a set of possible physical states**
- If the agent is in one of several possible physical states, then an action may lead to one of several possible outcomes, **even if the environment is deterministic**

Belief States

- **Belief state**: **the agent's current belief about the possible physical states it might be in**, given the previous sequence of actions and percepts
 - is a **set of physical states**: **the agent is in one of these states** (but does not know in which one)
 - contains the actual physical state the agent is in
 - ex: $\{1, 2\}$: **the agent is either in state 1 or in state 2** (but it does not know in which one)
 - if the belief state contains only one state, then the agent knows it is in that state
- 2^n possible belief states out of n possible physical states!

In practice, we reason in terms of (disjunctions of) **partial states** rather than of **sets of states**

- a **partial state** assigns values to a subset of state variables (e.g. $\langle ?, \langle ?, Clean \rangle \rangle = \{3, 4, 7, 8\}$)

Generalities

Partial Observability

- **Partial observability**: percepts do not capture the whole state
 - not all state variable values known $\Rightarrow$ corresponds to **a set of possible physical states**
- If the agent is in one of several possible physical states, then an action may lead to one of several possible outcomes, **even if the environment is deterministic**

Belief States

- **Belief state**: **the agent's current belief about the possible physical states it might be in**, given the previous sequence of actions and percepts
 - is a **set of physical states**: **the agent is in one of these states** (but does not know in which one)
 - contains the actual physical state the agent is in
 - ex: $\{1, 2\}$: **the agent is either in state 1 or in state 2** (but it does not know in which one)
 - if the belief state contains only one state, then the agent knows it is in that state
- **2^n possible belief states out of n possible physical states!**

In practice, we reason in terms of (disjunctions of) **partial states** rather than of **sets of states**

- a **partial state** assigns values to a subset of state variables (e.g. $\langle ?, \langle ?, \text{Clean} \rangle \rangle = \{3, 4, 7, 8\}$)

Generalities

Partial Observability

- **Partial observability**: percepts do not capture the whole state
 - not all state variable values known $\implies$ corresponds to **a set of possible physical states**
- If the agent is in one of several possible physical states, then an action may lead to one of several possible outcomes, **even if the environment is deterministic**

Belief States

- **Belief state**: **the agent's current belief about the possible physical states it might be in**, given the previous sequence of actions and percepts
 - is a **set of physical states**: **the agent is in one of these states** (but does not know in which one)
 - contains the actual physical state the agent is in
 - ex: $\{1, 2\}$: **the agent is either in state 1 or in state 2** (but it does not know in which one)
 - if the belief state contains only one state, then the agent knows it is in that state
- **2^n possible belief states out of n possible physical states!**

In practice, we reason in terms of (disjunctions of) **partial states** rather than of **sets of states**

- a **partial state** assigns values to a subset of state variables (e.g. $\langle ?, \langle ?, \text{Clean} \rangle \rangle = \{3, 4, 7, 8\}$)

- 1 Local Search and Optimization
 - General Ideas
 - Hill-Climbing
 - Simulated Annealing
 - Local Beam Search & Genetic Algorithms
- 2 Search with Nondeterministic Actions
- 3 Search with Partial or No Observations (Deterministic/Nondeterministic Actions)
 - Search with No Observations
 - Search with Partial Observations
- 4 Online Search (aka Exploration)

Search with No Observation

Search with No Observation (aka Sensorless Search or Conformant Search)

- Idea: To solve sensorless problems, **the agent searches in the space of belief states** rather than in that of physical states
 - **fully observable**, because the agent knows its own belief space
 - **solutions are always sequences of actions** (no contingency plan), because percepts are always empty and thus predictable
- Main drawback: 2^N candidate states rather than N

Search with No Observation

Search with No Observation (aka Sensorless Search or Conformant Search)

- Idea: To solve sensorless problems, **the agent searches in the space of belief states** rather than in that of physical states
 - **fully observable**, because the agent knows its own belief space
 - **solutions are always sequences of actions** (no contingency plan), because percepts are always empty and thus predictable
- Main drawback: **2^N candidate states rather than N**

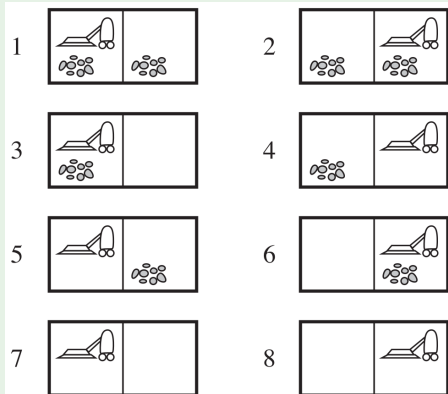
Search with No Observation: Example

Example: Sensorless Vacuum Cleaner

- the vacuum cleaner knows the geography of its world, but it **doesn't know its location or the distribution of dirt**

- initial state: $\{1, 2, 3, 4, 5, 6, 7, 8\}$ (i.e. $\langle ?, \langle ?, ? \rangle \rangle$)
- after action RIGHT, state is $\{2, 4, 6, 8\}$ (i.e. $\langle B, \langle ?, ? \rangle \rangle$)
- after action sequence [RIGHT,SUCK], state is $\{4, 8\}$ (i.e. $\langle B, \langle ?, Clean \rangle \rangle$)
- after action sequence [RIGHT,SUCK,LEFT], state is $\{3, 7\}$ (i.e. $\langle A, \langle ?, Clean \rangle \rangle$)
- after action sequence [RIGHT,SUCK,LEFT,SUCK], state is $\{7\}$ (i.e. $\langle A, \langle Clean, Clean \rangle \rangle$)
- In practice, the information on the state is made progressively less partial by the actions:

$$\begin{array}{ccc} \{1, 2, 3, 4, 5, 6, 7, 8\} & & \{2, 4, 6, 8\} \\ \underbrace{\langle ?, \langle ?, ? \rangle \rangle} & \xRightarrow{\text{RIGHT}} & \underbrace{\langle B, \langle ?, ? \rangle \rangle} \end{array}$$

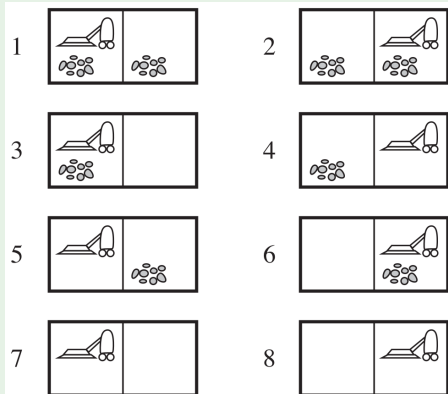


Search with No Observation: Example

Example: Sensorless Vacuum Cleaner

- the vacuum cleaner knows the geography of its world, but it **doesn't know its location or the distribution of dirt**
 - initial state: $\{1, 2, 3, 4, 5, 6, 7, 8\}$ (i.e. $\langle ?, \langle ?, ? \rangle \rangle$)
 - after action RIGHT, state is $\{2, 4, 6, 8\}$ (i.e. $\langle B, \langle ?, ? \rangle \rangle$)
 - after action sequence [RIGHT,SUCK], state is $\{4, 8\}$ (i.e. $\langle B, \langle ?, Clean \rangle \rangle$)
 - after action sequence [RIGHT,SUCK,LEFT], state is $\{3, 7\}$ (i.e. $\langle A, \langle ?, Clean \rangle \rangle$)
 - after action sequence [RIGHT,SUCK,LEFT,SUCK], state is $\{7\}$ (i.e. $\langle A, \langle Clean, Clean \rangle \rangle$)
- In practice, the information on the state is made progressively less partial by the actions:

$\{1, 2, 3, 4, 5, 6, 7, 8\}$ $\{2, 4, 6, 8\}$
 $\langle ?, \langle ?, ? \rangle \rangle$ $\xRightarrow{\text{RIGHT}}$ $\langle B, \langle ?, ? \rangle \rangle$



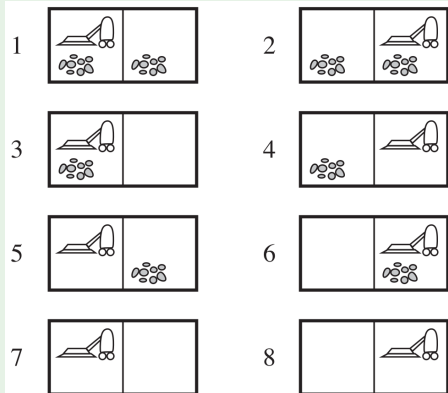
Search with No Observation: Example

Example: Sensorless Vacuum Cleaner

- the vacuum cleaner knows the geography of its world, but it **doesn't know its location or the distribution of dirt**

- initial state: $\{1, 2, 3, 4, 5, 6, 7, 8\}$ (i.e. $\langle ?, \langle ?, ? \rangle \rangle$)
- after action **RIGHT**, state is $\{2, 4, 6, 8\}$ (i.e. $\langle B, \langle ?, ? \rangle \rangle$)
- after action sequence [RIGHT,SUCK], state is $\{4, 8\}$ (i.e. $\langle B, \langle ?, Clean \rangle \rangle$)
- after action sequence [RIGHT,SUCK,LEFT], state is $\{3, 7\}$ (i.e. $\langle A, \langle ?, Clean \rangle \rangle$)
- after action sequence [RIGHT,SUCK,LEFT,SUCK], state is $\{7\}$ (i.e. $\langle A, \langle Clean, Clean \rangle \rangle$)
- In practice, the information on the state is made progressively less partial by the actions:

$\{1, 2, 3, 4, 5, 6, 7, 8\}$ $\{2, 4, 6, 8\}$
 $\langle ?, \langle ?, ? \rangle \rangle$ $\xrightarrow{\text{RIGHT}}$ $\langle B, \langle ?, ? \rangle \rangle$

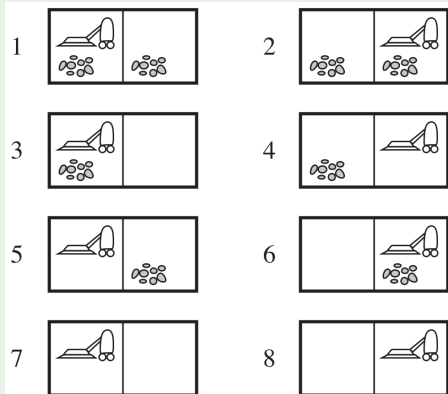


Search with No Observation: Example

Example: Sensorless Vacuum Cleaner

- the vacuum cleaner knows the geography of its world, but it **doesn't know its location or the distribution of dirt**
 - initial state: $\{1, 2, 3, 4, 5, 6, 7, 8\}$ (i.e. $\langle ?, \langle ?, ? \rangle \rangle$)
 - after action **RIGHT**, state is $\{2, 4, 6, 8\}$ (i.e. $\langle B, \langle ?, ? \rangle \rangle$)
 - after action sequence **[RIGHT,SUCK]**, state is $\{4, 8\}$ (i.e. $\langle B, \langle ?, Clean \rangle \rangle$)
 - after action sequence **[RIGHT,SUCK,LEFT]**, state is $\{3, 7\}$ (i.e. $\langle A, \langle ?, Clean \rangle \rangle$)
 - after action sequence **[RIGHT,SUCK,LEFT,SUCK]**, state is $\{7\}$ (i.e. $\langle A, \langle Clean, Clean \rangle \rangle$)
- In practice, the information on the state is made progressively less partial by the actions:

$\{1, 2, 3, 4, 5, 6, 7, 8\}$ $\{2, 4, 6, 8\}$
 $\langle ?, \langle ?, ? \rangle \rangle$ $\xrightarrow{\text{RIGHT}}$ $\langle B, \langle ?, ? \rangle \rangle$

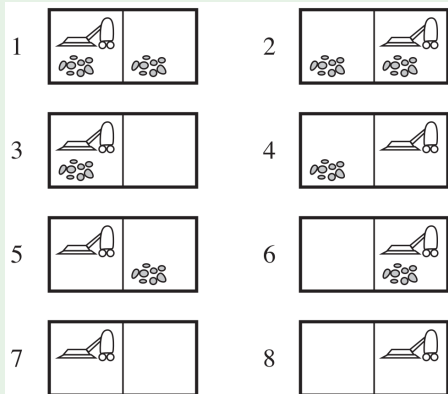


Search with No Observation: Example

Example: Sensorless Vacuum Cleaner

- the vacuum cleaner knows the geography of its world, but it **doesn't know its location or the distribution of dirt**
 - initial state: $\{1, 2, 3, 4, 5, 6, 7, 8\}$ (i.e. $\langle ?, \langle ?, ? \rangle \rangle$)
 - after action **RIGHT**, state is $\{2, 4, 6, 8\}$ (i.e. $\langle B, \langle ?, ? \rangle \rangle$)
 - after action sequence **[RIGHT,SUCK]**, state is $\{4, 8\}$ (i.e. $\langle B, \langle ?, Clean \rangle \rangle$)
 - after action sequence **[RIGHT,SUCK,LEFT]**, state is $\{3, 7\}$ (i.e. $\langle A, \langle ?, Clean \rangle \rangle$)
 - after action sequence **[RIGHT,SUCK,LEFT,SUCK]**, state is $\{7\}$ (i.e. $\langle A, \langle Clean, Clean \rangle \rangle$)
- In practice, the information on the state is made progressively less partial by the actions:

$\{1, 2, 3, 4, 5, 6, 7, 8\}$ $\{2, 4, 6, 8\}$
 $\langle ?, \langle ?, ? \rangle \rangle$ $\xrightarrow{\text{RIGHT}}$ $\langle B, \langle ?, ? \rangle \rangle$

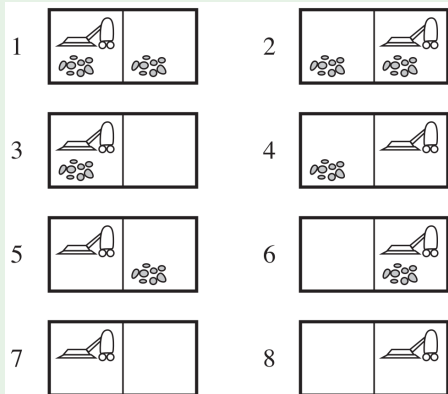


Search with No Observation: Example

Example: Sensorless Vacuum Cleaner

- the vacuum cleaner knows the geography of its world, but it **doesn't know its location or the distribution of dirt**
 - initial state: $\{1, 2, 3, 4, 5, 6, 7, 8\}$ (i.e. $\langle ?, \langle ?, ? \rangle \rangle$)
 - after action **RIGHT**, state is $\{2, 4, 6, 8\}$ (i.e. $\langle B, \langle ?, ? \rangle \rangle$)
 - after action sequence **[RIGHT, SUCK]**, state is $\{4, 8\}$ (i.e. $\langle B, \langle ?, Clean \rangle \rangle$)
 - after action sequence **[RIGHT, SUCK, LEFT]**, state is $\{3, 7\}$ (i.e. $\langle A, \langle ?, Clean \rangle \rangle$)
 - after action sequence **[RIGHT, SUCK, LEFT, SUCK]**, state is $\{7\}$ (i.e. $\langle A, \langle Clean, Clean \rangle \rangle$)
- In practice, the information on the state is made progressively less partial by the actions:

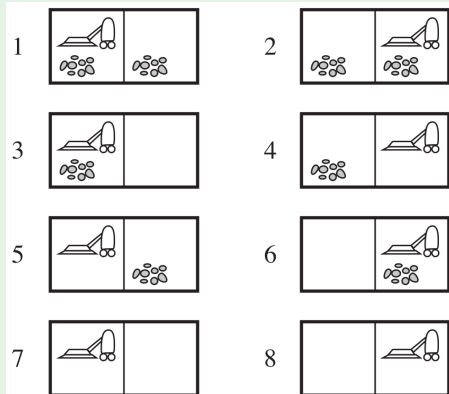
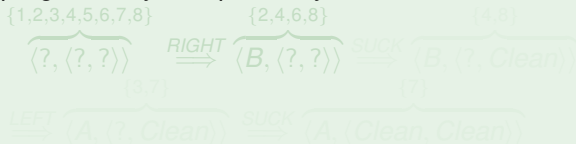
$\{1, 2, 3, 4, 5, 6, 7, 8\}$ $\{2, 4, 6, 8\}$
 $\langle ?, \langle ?, ? \rangle \rangle$ $\xrightarrow{\text{RIGHT}}$ $\langle B, \langle ?, ? \rangle \rangle$



Search with No Observation: Example

Example: Sensorless Vacuum Cleaner

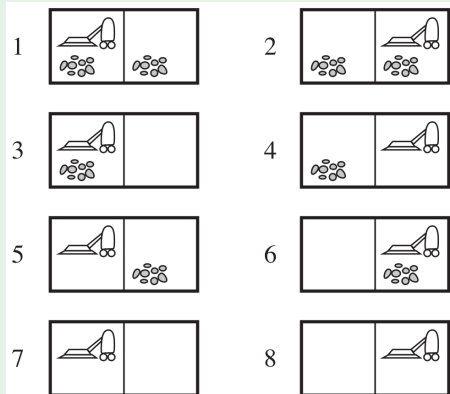
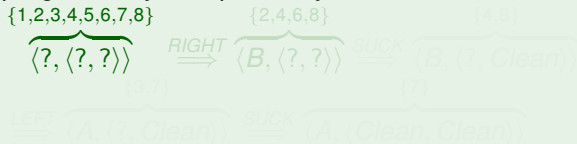
- the vacuum cleaner knows the geography of its world, but it **doesn't know its location or the distribution of dirt**
 - initial state: $\{1, 2, 3, 4, 5, 6, 7, 8\}$ (i.e. $\langle ?, \langle ?, ? \rangle \rangle$)
 - after action **RIGHT**, state is $\{2, 4, 6, 8\}$ (i.e. $\langle B, \langle ?, ? \rangle \rangle$)
 - after action sequence **[RIGHT,SUCK]**, state is $\{4, 8\}$ (i.e. $\langle B, \langle ?, Clean \rangle \rangle$)
 - after action sequence **[RIGHT,SUCK,LEFT]**, state is $\{3, 7\}$ (i.e. $\langle A, \langle ?, Clean \rangle \rangle$)
 - after action sequence **[RIGHT,SUCK,LEFT,SUCK]**, state is $\{7\}$ (i.e. $\langle A, \langle Clean, Clean \rangle \rangle$)
- In practice, the information on the state is made progressively less partial by the actions:



Search with No Observation: Example

Example: Sensorless Vacuum Cleaner

- the vacuum cleaner knows the geography of its world, but it **doesn't know its location or the distribution of dirt**
 - initial state: $\{1, 2, 3, 4, 5, 6, 7, 8\}$ (i.e. $\langle ?, \langle ?, ? \rangle \rangle$)
 - after action **RIGHT**, state is $\{2, 4, 6, 8\}$ (i.e. $\langle B, \langle ?, ? \rangle \rangle$)
 - after action sequence **[RIGHT,SUCK]**, state is $\{4, 8\}$ (i.e. $\langle B, \langle ?, Clean \rangle \rangle$)
 - after action sequence **[RIGHT,SUCK,LEFT]**, state is $\{3, 7\}$ (i.e. $\langle A, \langle ?, Clean \rangle \rangle$)
 - after action sequence **[RIGHT,SUCK,LEFT,SUCK]**, state is $\{7\}$ (i.e. $\langle A, \langle Clean, Clean \rangle \rangle$)
- In practice, the information on the state is made progressively less partial by the actions:



Search with No Observation: Example

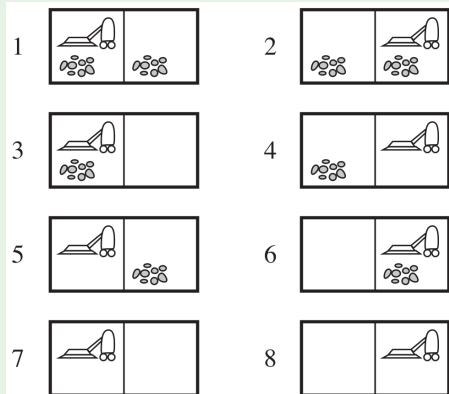
Example: Sensorless Vacuum Cleaner

- the vacuum cleaner knows the geography of its world, but it **doesn't know its location or the distribution of dirt**

- initial state: $\{1, 2, 3, 4, 5, 6, 7, 8\}$ (i.e. $\langle ?, \langle ?, ? \rangle \rangle$)
- after action **RIGHT**, state is $\{2, 4, 6, 8\}$ (i.e. $\langle B, \langle ?, ? \rangle \rangle$)
- after action sequence **[RIGHT,SUCK]**, state is $\{4, 8\}$ (i.e. $\langle B, \langle ?, Clean \rangle \rangle$)
- after action sequence **[RIGHT,SUCK,LEFT]**, state is $\{3, 7\}$ (i.e. $\langle A, \langle ?, Clean \rangle \rangle$)
- after action sequence **[RIGHT,SUCK,LEFT,SUCK]**, state is $\{7\}$ (i.e. $\langle A, \langle Clean, Clean \rangle \rangle$)

- In practice, the information on the state is made progressively less partial by the actions:

$$\begin{array}{c}
 \{1,2,3,4,5,6,7,8\} \quad \{2,4,6,8\} \quad \{4,8\} \\
 \underbrace{\langle ?, \langle ?, ? \rangle}_{\{1,2,3,4,5,6,7,8\}} \xrightarrow{\text{RIGHT}} \underbrace{\langle B, \langle ?, ? \rangle}_{\{2,4,6,8\}} \xrightarrow{\text{SUCK}} \underbrace{\langle B, \langle ?, Clean \rangle}_{\{4,8\}} \\
 \xrightarrow{\text{LEFT}} \underbrace{\langle A, \langle ?, Clean \rangle}_{\{3,7\}} \xrightarrow{\text{SUCK}} \underbrace{\langle A, \langle Clean, Clean \rangle}_{\{7\}}
 \end{array}$$

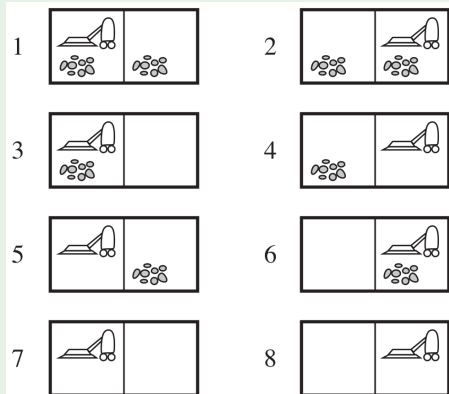


Search with No Observation: Example

Example: Sensorless Vacuum Cleaner

- the vacuum cleaner knows the geography of its world, but it **doesn't know its location or the distribution of dirt**
 - initial state: $\{1, 2, 3, 4, 5, 6, 7, 8\}$ (i.e. $\langle ?, \langle ?, ? \rangle \rangle$)
 - after action **RIGHT**, state is $\{2, 4, 6, 8\}$ (i.e. $\langle B, \langle ?, ? \rangle \rangle$)
 - after action sequence **[RIGHT,SUCK]**, state is $\{4, 8\}$ (i.e. $\langle B, \langle ?, Clean \rangle \rangle$)
 - after action sequence **[RIGHT,SUCK,LEFT]**, state is $\{3, 7\}$ (i.e. $\langle A, \langle ?, Clean \rangle \rangle$)
 - after action sequence **[RIGHT,SUCK,LEFT,SUCK]**, state is $\{7\}$ (i.e. $\langle A, \langle Clean, Clean \rangle \rangle$)
- In practice, the information on the state is made progressively less partial by the actions:

$$\begin{array}{c}
 \{1,2,3,4,5,6,7,8\} \quad \{2,4,6,8\} \quad \{4,8\} \\
 \underbrace{\langle ?, \langle ?, ? \rangle}_{\{1,2,3,4,5,6,7,8\}} \xrightarrow{RIGHT} \underbrace{\langle B, \langle ?, ? \rangle}_{\{2,4,6,8\}} \xrightarrow{SUCK} \underbrace{\langle B, \langle ?, Clean \rangle}_{\{4,8\}} \\
 \xrightarrow{LEFT} \underbrace{\langle A, \langle ?, Clean \rangle}_{\{3,7\}} \xrightarrow{SUCK} \underbrace{\langle A, \langle Clean, Clean \rangle}_{\{7\}}
 \end{array}$$

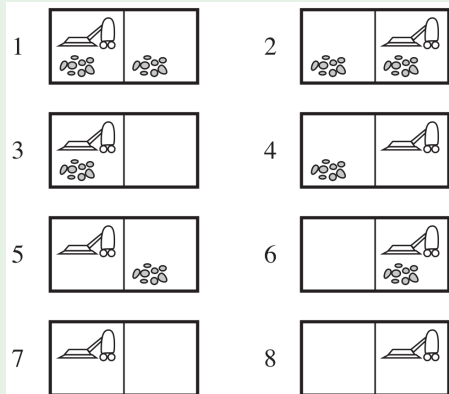


Search with No Observation: Example

Example: Sensorless Vacuum Cleaner

- the vacuum cleaner knows the geography of its world, but it **doesn't know its location or the distribution of dirt**
 - initial state: $\{1, 2, 3, 4, 5, 6, 7, 8\}$ (i.e. $\langle ?, \langle ?, ? \rangle \rangle$)
 - after action **RIGHT**, state is $\{2, 4, 6, 8\}$ (i.e. $\langle B, \langle ?, ? \rangle \rangle$)
 - after action sequence **[RIGHT,SUCK]**, state is $\{4, 8\}$ (i.e. $\langle B, \langle ?, Clean \rangle \rangle$)
 - after action sequence **[RIGHT,SUCK,LEFT]**, state is $\{3, 7\}$ (i.e. $\langle A, \langle ?, Clean \rangle \rangle$)
 - after action sequence **[RIGHT,SUCK,LEFT,SUCK]**, state is $\{7\}$ (i.e. $\langle A, \langle Clean, Clean \rangle \rangle$)
- In practice, the information on the state is made progressively less partial by the actions:

$$\begin{aligned} & \{1, 2, 3, 4, 5, 6, 7, 8\} \xrightarrow{\text{RIGHT}} \{2, 4, 6, 8\} \xrightarrow{\text{SUCK}} \{4, 8\} \\ & \langle ?, \langle ?, ? \rangle \rangle \xrightarrow{\text{RIGHT}} \langle B, \langle ?, ? \rangle \rangle \xrightarrow{\text{SUCK}} \langle B, \langle ?, Clean \rangle \rangle \\ & \quad \quad \quad \{3, 7\} \quad \quad \quad \{7\} \\ & \xrightarrow{\text{LEFT}} \langle A, \langle ?, Clean \rangle \rangle \xrightarrow{\text{SUCK}} \langle A, \langle Clean, Clean \rangle \rangle \end{aligned}$$



Search with No Observation: Example

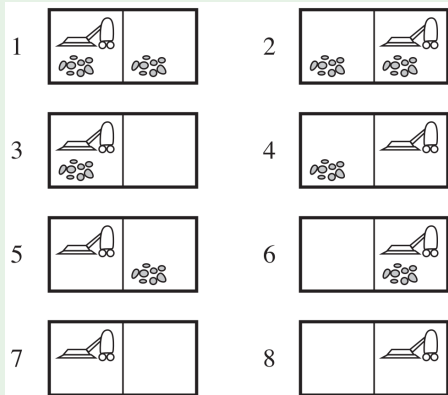
Example: Sensorless Vacuum Cleaner

- the vacuum cleaner knows the geography of its world, but it **doesn't know its location or the distribution of dirt**

- initial state: $\{1, 2, 3, 4, 5, 6, 7, 8\}$ (i.e. $\langle ?, \langle ?, ? \rangle \rangle$)
- after action **RIGHT**, state is $\{2, 4, 6, 8\}$ (i.e. $\langle B, \langle ?, ? \rangle \rangle$)
- after action sequence **[RIGHT,SUCK]**, state is $\{4, 8\}$ (i.e. $\langle B, \langle ?, Clean \rangle \rangle$)
- after action sequence **[RIGHT,SUCK,LEFT]**, state is $\{3, 7\}$ (i.e. $\langle A, \langle ?, Clean \rangle \rangle$)
- after action sequence **[RIGHT,SUCK,LEFT,SUCK]**, state is $\{7\}$ (i.e. $\langle A, \langle Clean, Clean \rangle \rangle$)

- In practice, the information on the state is made progressively less partial by the actions:

$$\begin{aligned} & \{1,2,3,4,5,6,7,8\} \quad \{2,4,6,8\} \quad \{4,8\} \\ & \langle ?, \langle ?, ? \rangle \rangle \xrightarrow{RIGHT} \langle B, \langle ?, ? \rangle \rangle \xrightarrow{SUCK} \langle B, \langle ?, Clean \rangle \rangle \\ & \quad \quad \quad \{3,7\} \quad \quad \quad \{7\} \\ & \xrightarrow{LEFT} \langle A, \langle ?, Clean \rangle \rangle \xrightarrow{SUCK} \langle A, \langle Clean, Clean \rangle \rangle \end{aligned}$$



Remark

Single partial state vs. disjunction of partial states

- Often belief state can be represented as a single partial state:

$$\text{Ex: } \overbrace{\langle ?, \langle ?, ? \rangle}^{\{1,2,3,4,5,6,7,8\}} \xRightarrow{\text{RIGHT}} \overbrace{\langle B, \langle ?, ? \rangle}^{\{2,4,6,8\}}$$

- Sometimes a belief state requires being represented as a **disjunction** of partial states.

$$\text{Ex: } \overbrace{\langle ?, \langle ?, ? \rangle}^{\{1,2,3,4,5,6,7,8\}} \xRightarrow{\text{SUCK}} \overbrace{\langle A, \langle \text{Clean}, ? \rangle}^{\{5,7\}} \text{ or } \overbrace{\langle B, \langle ?, \text{Clean} \rangle}^{\{4,8\}}$$

(one location has been cleaned, but we do not know which one)

Remark

Single partial state vs. disjunction of partial states

- Often belief state can be represented as a single partial state:

$$\text{Ex: } \overbrace{\langle ?, \langle ?, ? \rangle}^{\{1,2,3,4,5,6,7,8\}} \xRightarrow{\text{RIGHT}} \overbrace{\langle B, \langle ?, ? \rangle}^{\{2,4,6,8\}}$$

- Sometimes a belief state requires being represented as a **disjunction of** partial states.

$$\text{Ex: } \overbrace{\langle ?, \langle ?, ? \rangle}^{\{1,2,3,4,5,6,7,8\}} \xRightarrow{\text{SUCK}} \overbrace{\langle A, \langle \text{Clean}, ? \rangle}^{\{5,7\}} \text{ or } \overbrace{\langle B, \langle ?, \text{Clean} \rangle}^{\{4,8\}}$$

(one location has been cleaned, but we do not know which one)

Remark

Single partial state vs. disjunction of partial states

- Often belief state can be represented as a single partial state:

$$\text{Ex: } \overbrace{\langle ?, \langle ?, ? \rangle}^{\{1,2,3,4,5,6,7,8\}} \xRightarrow{\text{RIGHT}} \overbrace{\langle B, \langle ?, ? \rangle}^{\{2,4,6,8\}}$$

- Sometimes a belief state requires being represented as a **disjunction of** partial states.

$$\text{Ex: } \overbrace{\langle ?, \langle ?, ? \rangle}^{\{1,2,3,4,5,6,7,8\}} \xRightarrow{\text{SUCK}} \overbrace{\langle A, \langle \text{Clean}, ? \rangle}^{\{5,7\}} \text{ or } \overbrace{\langle B, \langle ?, \text{Clean} \rangle}^{\{4,8\}}$$

(one location has been cleaned, but we do not know which one)

Belief-State Problem Formulation

Let $Actions_P()$, $Result_P()$, $GoalTest_P()$, $StepCost_P()$ refer to physical System P :

- **Belief states**: subsets of physical states (typically partial states)
 - If P has N states, then the sensorless problem has up to 2^N states
- **Initial state**: typically the set of all physical states in P
- **Actions**: (assumption: illegal actions have no effects)
 - $Actions(b) \stackrel{\text{def}}{=} \bigcup_{s \in b} Actions_P(s)$ (i.e., must consider all possible actions in all possible states)
- **Transition model**:
 - for deterministic actions: $b' = Result(b, a) \stackrel{\text{def}}{=} \{s' \mid s' = Result_P(s, a) \text{ and } s \in b\}$;
 - for nondeterministic actions:^a
 $b' = Result(b, a) \stackrel{\text{def}}{=} \{s' \mid s' \in Result_P(s, a) \text{ and } s \in b\} = \bigcup_{s \in b} Result_P(s, a)$;
 - this step is called **Prediction**: $b' \stackrel{\text{def}}{=} Predict(b, a)$.
- **Goal test**: $GoalTest(b)$ holds iff $GoalTest_P(s)$ holds, $\forall s \in b$
(i.e., all possible states must be goal ones)
- **Path cost**: (assumption: cost of an action the same in all states)
 - $StepCost(a, b) \stackrel{\text{def}}{=} StepCost_P(a, s)$, $\forall s \in b$

^amind the “ $\in$ ” here

Belief-State Problem Formulation

Let $Actions_P()$, $Result_P()$, $GoalTest_P()$, $StepCost_P()$ refer to physical System P :

- **Belief states**: subsets of physical states (typically partial states)
 - If P has N states, then the sensorless problem has up to 2^N states
- **Initial state**: typically the set of all physical states in P
- **Actions**: (assumption: illegal actions have no effects)
 - $Actions(b) \stackrel{\text{def}}{=} \bigcup_{s \in b} Actions_P(s)$ (i.e., must consider all possible actions in all possible states)
- **Transition model**:
 - for deterministic actions: $b' = Result(b, a) \stackrel{\text{def}}{=} \{s' \mid s' = Result_P(s, a) \text{ and } s \in b\}$;
 - for nondeterministic actions:^a
 $b' = Result(b, a) \stackrel{\text{def}}{=} \{s' \mid s' \in Result_P(s, a) \text{ and } s \in b\} = \bigcup_{s \in b} Result_P(s, a)$;
 - this step is called **Prediction**: $b' \stackrel{\text{def}}{=} Predict(b, a)$.
- **Goal test**: $GoalTest(b)$ holds iff $GoalTest_P(s)$ holds, $\forall s \in b$
(i.e., all possible states must be goal ones)
- **Path cost**: (assumption: cost of an action the same in all states)
 - $StepCost(a, b) \stackrel{\text{def}}{=} StepCost_P(a, s)$, $\forall s \in b$

^amind the “ $\in$ ” here

Belief-State Problem Formulation

Let $Actions_P()$, $Result_P()$, $GoalTest_P()$, $StepCost_P()$ refer to physical System P :

- **Belief states**: subsets of physical states (typically partial states)
 - If P has N states, then the sensorless problem has up to 2^N states
- **Initial state**: typically the set of all physical states in P
- **Actions**: (assumption: illegal actions have no effects)
 - $Actions(b) \stackrel{\text{def}}{=} \bigcup_{s \in b} Actions_P(s)$ (i.e., must consider all possible actions in all possible states)
- **Transition model**:
 - for deterministic actions: $b' = Result(b, a) \stackrel{\text{def}}{=} \{s' \mid s' = Result_P(s, a) \text{ and } s \in b\}$;
 - for nondeterministic actions:^a
 $b' = Result(b, a) \stackrel{\text{def}}{=} \{s' \mid s' \in Result_P(s, a) \text{ and } s \in b\} = \bigcup_{s \in b} Result_P(s, a)$;
 - this step is called **Prediction**: $b' \stackrel{\text{def}}{=} Predict(b, a)$.
- **Goal test**: $GoalTest(b)$ holds iff $GoalTest_P(s)$ holds, $\forall s \in b$
(i.e., all possible states must be goal ones)
- **Path cost**: (assumption: cost of an action the same in all states)
 - $StepCost(a, b) \stackrel{\text{def}}{=} StepCost_P(a, s)$, $\forall s \in b$

^amind the “ $\in$ ” here

Belief-State Problem Formulation

Let $Actions_P()$, $Result_P()$, $GoalTest_P()$, $StepCost_P()$ refer to physical System P :

- **Belief states**: subsets of physical states (typically partial states)
 - If P has N states, then the sensorless problem has up to 2^N states
- **Initial state**: typically the set of all physical states in P
- **Actions**: (assumption: illegal actions have no effects)
 - $Actions(b) \stackrel{\text{def}}{=} \bigcup_{s \in b} Actions_P(s)$ (i.e., must consider all possible actions in all possible states)
- **Transition model**:
 - for deterministic actions: $b' = Result(b, a) \stackrel{\text{def}}{=} \{s' \mid s' = Result_P(s, a) \text{ and } s \in b\}$;
 - for nondeterministic actions:^a
 $b' = Result(b, a) \stackrel{\text{def}}{=} \{s' \mid s' \in Result_P(s, a) \text{ and } s \in b\} = \bigcup_{s \in b} Result_P(s, a)$;
 - this step is called **Prediction**: $b' \stackrel{\text{def}}{=} Predict(b, a)$.
- **Goal test**: $GoalTest(b)$ holds iff $GoalTest_P(s)$ holds, $\forall s \in b$
(i.e., all possible states must be goal ones)
- **Path cost**: (assumption: cost of an action the same in all states)
 - $StepCost(a, b) \stackrel{\text{def}}{=} StepCost_P(a, s)$, $\forall s \in b$

^amind the “ $\in$ ” here

Belief-State Problem Formulation

Let $Actions_P()$, $Result_P()$, $GoalTest_P()$, $StepCost_P()$ refer to physical System P :

- **Belief states**: subsets of physical states (typically partial states)
 - If P has N states, then the sensorless problem has up to 2^N states
- **Initial state**: typically the set of all physical states in P
- **Actions**: (assumption: illegal actions have no effects)
 - $Actions(b) \stackrel{\text{def}}{=} \bigcup_{s \in b} Actions_P(s)$ (i.e., must consider all possible actions in all possible states)
- **Transition model**:
 - for deterministic actions: $b' = Result(b, a) \stackrel{\text{def}}{=} \{s' \mid s' = Result_P(s, a) \text{ and } s \in b\}$;
 - for nondeterministic actions:^a
 $b' = Result(b, a) \stackrel{\text{def}}{=} \{s' \mid s' \in Result_P(s, a) \text{ and } s \in b\} = \bigcup_{s \in b} Result_P(s, a)$;
 - this step is called **Prediction**: $b' \stackrel{\text{def}}{=} Predict(b, a)$.
- **Goal test**: $GoalTest(b)$ holds iff $GoalTest_P(s)$ holds, $\forall s \in b$
(i.e., all possible states must be goal ones)
- **Path cost**: (assumption: cost of an action the same in all states)
 - $StepCost(a, b) \stackrel{\text{def}}{=} StepCost_P(a, s)$, $\forall s \in b$

^amind the “ $\in$ ” here

Belief-State Problem Formulation

Let $Actions_P()$, $Result_P()$, $GoalTest_P()$, $StepCost_P()$ refer to physical System P :

- **Belief states**: subsets of physical states (typically partial states)
 - If P has N states, then the sensorless problem has up to 2^N states
- **Initial state**: typically the set of all physical states in P
- **Actions**: (assumption: illegal actions have no effects)
 - $Actions(b) \stackrel{\text{def}}{=} \bigcup_{s \in b} Actions_P(s)$ (i.e., must consider all possible actions in all possible states)
- **Transition model**:
 - for deterministic actions: $b' = Result(b, a) \stackrel{\text{def}}{=} \{s' \mid s' = Result_P(s, a) \text{ and } s \in b\}$;
 - for nondeterministic actions:^a
 $b' = Result(b, a) \stackrel{\text{def}}{=} \{s' \mid s' \in Result_P(s, a) \text{ and } s \in b\} = \bigcup_{s \in b} Result_P(s, a)$;
 - this step is called **Prediction**: $b' \stackrel{\text{def}}{=} Predict(b, a)$.
- **Goal test**: $GoalTest(b)$ holds iff $GoalTest_P(s)$ holds, $\forall s \in b$
(i.e., all possible states must be goal ones)
- **Path cost**: (assumption: cost of an action the same in all states)
 - $StepCost(a, b) \stackrel{\text{def}}{=} StepCost_P(a, s)$, $\forall s \in b$

^amind the “ $\in$ ” here

Belief-State Problem Formulation

Let $Actions_P()$, $Result_P()$, $GoalTest_P()$, $StepCost_P()$ refer to physical System P :

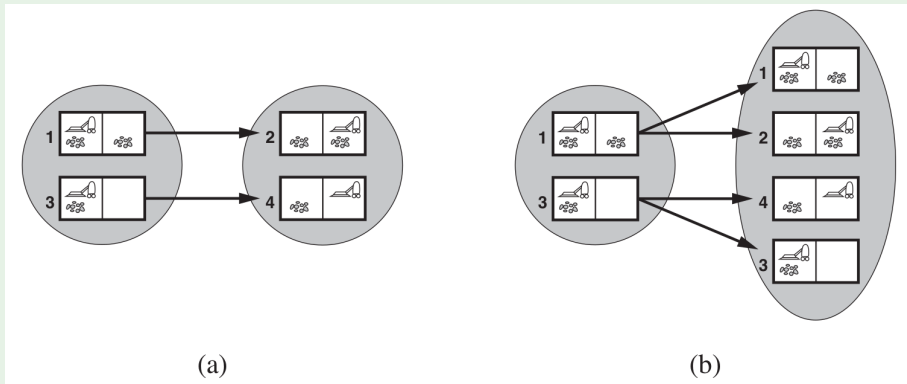
- **Belief states**: subsets of physical states (typically partial states)
 - If P has N states, then the sensorless problem has up to 2^N states
- **Initial state**: typically the set of all physical states in P
- **Actions**: (assumption: illegal actions have no effects)
 - $Actions(b) \stackrel{\text{def}}{=} \bigcup_{s \in b} Actions_P(s)$ (i.e., must consider all possible actions in all possible states)
- **Transition model**:
 - for deterministic actions: $b' = Result(b, a) \stackrel{\text{def}}{=} \{s' \mid s' = Result_P(s, a) \text{ and } s \in b\}$;
 - for nondeterministic actions:^a
 $b' = Result(b, a) \stackrel{\text{def}}{=} \{s' \mid s' \in Result_P(s, a) \text{ and } s \in b\} = \bigcup_{s \in b} Result_P(s, a)$;
 - this step is called **Prediction**: $b' \stackrel{\text{def}}{=} Predict(b, a)$.
- **Goal test**: $GoalTest(b)$ holds iff $GoalTest_P(s)$ holds, $\forall s \in b$
(i.e., all possible states must be goal ones)
- **Path cost**: (assumption: cost of an action the same in all states)
 - $StepCost(a, b) \stackrel{\text{def}}{=} StepCost_P(a, s)$, $\forall s \in b$

^amind the “ $\in$ ” here

Belief-State Problem Formulation [cont.]

Example: Sensorless Vacuum Cleaner, plain and slippery versions

Prediction: *Result*({1, 3}, *Right*), deterministic (a) and nondeterministic action (b)



(© S. Russell & P. Norwig, AIMA)

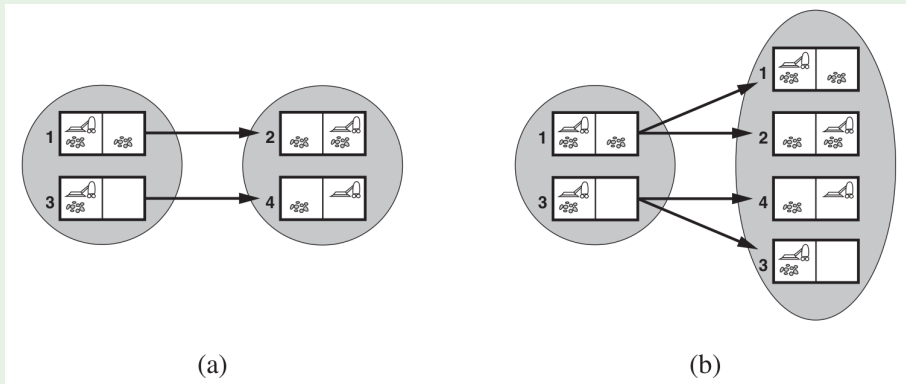
$$(a): \underbrace{\langle A, \langle \text{Dirty}, ? \rangle \rangle}_{\{1,3\}} \xRightarrow{\text{Right}} \underbrace{\langle B, \langle \text{Dirty}, ? \rangle \rangle}_{\{2,4\}}$$

$$(b): \underbrace{\langle A, \langle \text{Dirty}, ? \rangle \rangle}_{\{1,3\}} \xRightarrow{\text{Right}} \underbrace{\langle ?, \langle \text{Dirty}, ? \rangle \rangle}_{\{1,2,3,4\}}$$

Belief-State Problem Formulation [cont.]

Example: Sensorless Vacuum Cleaner, plain and slippery versions

Prediction: *Result*({1, 3}, *Right*), deterministic (a) and nondeterministic action (b)



(© S. Russell & P. Norwig, AIMA)

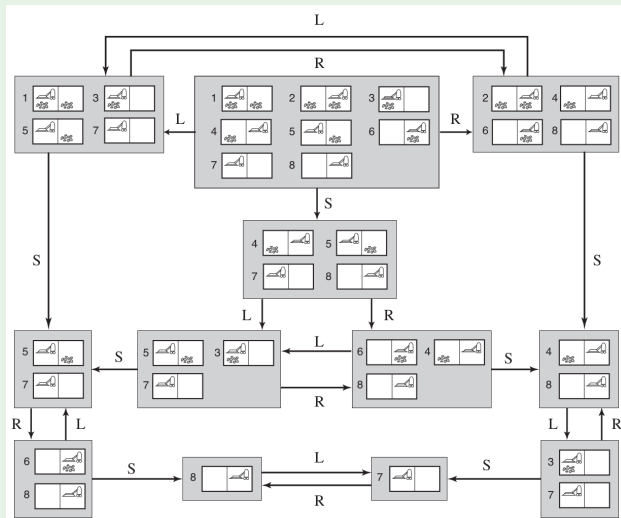
$$(a): \underbrace{\langle A, \langle \text{Dirty}, ? \rangle \rangle}_{\{1,3\}} \xRightarrow{\text{Right}} \underbrace{\langle B, \langle \text{Dirty}, ? \rangle \rangle}_{\{2,4\}}$$

$$(b): \underbrace{\langle A, \langle \text{Dirty}, ? \rangle \rangle}_{\{1,3\}} \xRightarrow{\text{Right}} \underbrace{\langle ?, \langle \text{Dirty}, ? \rangle \rangle}_{\{1,2,3,4\}}$$

Belief-State Problem Formulation [cont.]

Example: Sensorless Vacuum Cleaner: Belief State Space

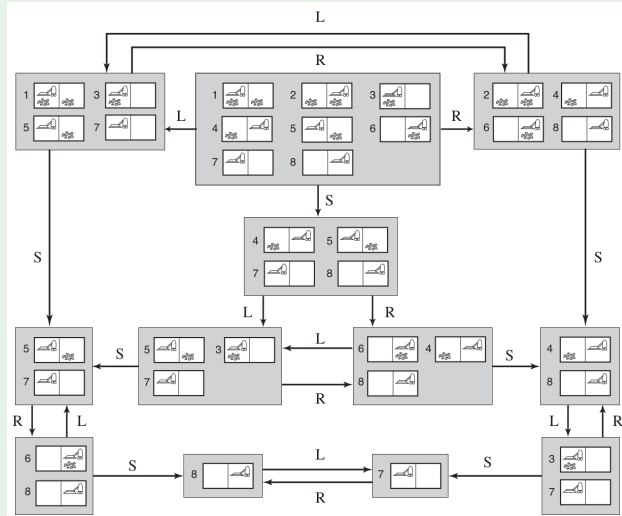
(self-loops are omitted)



Belief-State Problem Formulation [cont.]

Example: Sensorless Vacuum Cleaner: Belief State Space

(self-loops are omitted)



Belief-State Problem Formulation [cont.]

Remarks

- 1 if $b \subseteq b'$, then $\text{Result}(b, a) \subseteq \text{Result}(b', a)$ (b more informative than b')
- 2 If a is deterministic, then $|\text{Result}(b, a)| \leq |b|$
- 3 The agent might achieve the goal earlier than $\text{Goal/Test}(b)$ holds, but it does not know it (because he knows it only when all states in the belief state are goal states)

Properties

- An action sequence is a solution for b iff it leads b to a goal
- If an action sequence is a solution for a belief state b , then it is also a solution for any belief state b' s.t. $b' \subseteq b$
 - if $b \xrightarrow{a_1} \dots \xrightarrow{a_n} g$, then $b' \xrightarrow{a_1} \dots \xrightarrow{a_n} g$

We can apply to the Belief-State space any search algorithm.

- if a solution for b has been found, then any $b' \subseteq b$ is solvable
- if $b' \subseteq b$ has already been generated and discarded, then we can discard a path reaching a belief state b

⇒ Dramatically improves efficiency

Belief-State Problem Formulation [cont.]

Remarks

- 1 if $b \subseteq b'$, then $\text{Result}(b, a) \subseteq \text{Result}(b', a)$ (b more informative than b')
- 2 If a is deterministic, then $|\text{Result}(b, a)| \leq |b|$
- 3 The agent might achieve the goal earlier than $\text{Goal/Test}(b)$ holds, but it does not know it (because he knows it only when all states in the belief state are goal states)

Properties

- An action sequence is a solution for b iff it leads b to a goal
- If an action sequence is a solution for a belief state b , then it is also a solution for any belief state b' s.t. $b' \subseteq b$
 - if $b \xrightarrow{a_1} \dots \xrightarrow{a_n} g$, then $b' \xrightarrow{a_1} \dots \xrightarrow{a_n} g$

We can apply to the Belief-State space any search algorithm.

- if a solution for b has been found, then any $b' \subseteq b$ is solvable
- if $b' \subseteq b$ has already been generated and discarded, then we can discard a path reaching a belief state b

⇒ Dramatically improves efficiency

Belief-State Problem Formulation [cont.]

Remarks

- 1 if $b \subseteq b'$, then $\text{Result}(b, a) \subseteq \text{Result}(b', a)$ (b more informative than b')
- 2 If a is deterministic, then $|\text{Result}(b, a)| \leq |b|$
- 3 The agent might achieve the goal earlier than $\text{GoalTest}(b)$ holds, but it does not know it (because he knows it only when **all** states in the belief state are goal states)

Properties

- An action sequence is a **solution** for b iff it leads b to a goal
- If an action sequence is a solution for a belief state b , then it is also a solution for any belief state b' s.t. $b' \subseteq b$
 - if $b \xrightarrow{a_1} \dots \xrightarrow{a_n} g$, then $b' \xrightarrow{a_1} \dots \xrightarrow{a_n} g$

We can apply to the Belief-State space any search algorithm.

- if a solution for b has been found, then any $b' \subseteq b$ is solvable
- if $b' \subseteq b$ has already been generated and discarded, then we can discard a path reaching a belief state b

⇒ Dramatically improves efficiency

Belief-State Problem Formulation [cont.]

Remarks

- 1 if $b \subseteq b'$, then $\text{Result}(b, a) \subseteq \text{Result}(b', a)$ (b more informative than b')
- 2 If a is deterministic, then $|\text{Result}(b, a)| \leq |b|$
- 3 The agent might achieve the goal earlier than $\text{Goal/Test}(b)$ holds, but it does not know it (because he knows it only when **all** states in the belief state are goal states)

Properties

- An action sequence is a **solution** for b iff it leads b to a goal
- If an action sequence is a solution for a belief state b , then it is also a solution for any belief state b' s.t. $b' \subseteq b$
 - if $b \xrightarrow{a_1} \dots \xrightarrow{a_k} g$, then $b' \xrightarrow{a_1} \dots \xrightarrow{a_k} g$

We can apply to the Belief-State space any search algorithm.

- if a solution for b has been found, then any $b' \subseteq b$ is solvable
- if $b' \subseteq b$ has already been generated and discarded, then we can discard a path reaching a belief state b

⇒ Dramatically improves efficiency

Belief-State Problem Formulation [cont.]

Remarks

- 1 if $b \subseteq b'$, then $\text{Result}(b, a) \subseteq \text{Result}(b', a)$ (b more informative than b')
- 2 If a is deterministic, then $|\text{Result}(b, a)| \leq |b|$
- 3 The agent might achieve the goal earlier than $\text{Goal/Test}(b)$ holds, but it does not know it (because he knows it only when **all** states in the belief state are goal states)

Properties

- An action sequence is a **solution** for b iff it leads b to a goal
- If an action sequence is a solution for a belief state b , then it is also a solution for any belief state b' s.t. $b' \subseteq b$
 - if $b \xrightarrow{a_1} \dots \xrightarrow{a_k} g$, then $b' \xrightarrow{a_1} \dots \xrightarrow{a_k} g$

We can apply to the Belief-State space any search algorithm.

- if a solution for b has been found, then any $b' \subseteq b$ is solvable
- if $b' \subseteq b$ has already been generated and discarded, then we can discard a path reaching a belief state b

⇒ Dramatically improves efficiency

Belief-State Problem Formulation [cont.]

Remarks

- 1 if $b \subseteq b'$, then $\text{Result}(b, a) \subseteq \text{Result}(b', a)$ (b more informative than b')
- 2 If a is deterministic, then $|\text{Result}(b, a)| \leq |b|$
- 3 The agent might achieve the goal earlier than $\text{Goal/Test}(b)$ holds, but it does not know it (because he knows it only when **all** states in the belief state are goal states)

Properties

- An action sequence is a **solution** for b iff it leads b to a goal
- If an action sequence is a solution for a belief state b , then it is also a solution for any belief state b' s.t. $b' \subseteq b$
 - if $b \xrightarrow{a_1} \dots \xrightarrow{a_k} g$, then $b' \xrightarrow{a_1} \dots \xrightarrow{a_k} g$

We can apply to the Belief-State space any search algorithm.

- if a solution for b has been found, then any $b' \subseteq b$ is solvable
- if $b' \subseteq b$ has already been generated and discarded, then we can discard a path reaching a belief state b

⇒ Dramatically improves efficiency

Belief-State Problem Formulation [cont.]

Remarks

- 1 if $b \subseteq b'$, then $\text{Result}(b, a) \subseteq \text{Result}(b', a)$ (b more informative than b')
- 2 If a is deterministic, then $|\text{Result}(b, a)| \leq |b|$
- 3 The agent might achieve the goal earlier than $\text{Goal/Test}(b)$ holds, but it does not know it (because he knows it only when **all** states in the belief state are goal states)

Properties

- An action sequence is a **solution** for b iff it leads b to a goal
- If an action sequence is a solution for a belief state b , then it is also a solution for any belief state b' s.t. $b' \subseteq b$
 - if $b \xrightarrow{a_1} \dots \xrightarrow{a_k} g$, then $b' \xrightarrow{a_1} \dots \xrightarrow{a_k} g$

We can apply to the Belief-State space any search algorithm.

- if a solution for b has been found, then any $b' \subseteq b$ is solvable
- if $b' \subseteq b$ has already been generated and discarded, then we can discard a path reaching a belief state b

⇒ Dramatically improves efficiency

Belief-State Problem Formulation [cont.]

Remarks

- 1 if $b \subseteq b'$, then $\text{Result}(b, a) \subseteq \text{Result}(b', a)$ (b more informative than b')
- 2 If a is deterministic, then $|\text{Result}(b, a)| \leq |b|$
- 3 The agent might achieve the goal earlier than $\text{Goal/Test}(b)$ holds, but it does not know it (because he knows it only when **all** states in the belief state are goal states)

Properties

- An action sequence is a **solution** for b iff it leads b to a goal
- If an action sequence is a solution for a belief state b , then it is also a solution for any belief state b' s.t. $b' \subseteq b$
 - if $b \xrightarrow{a_1} \dots \xrightarrow{a_k} g$, then $b' \xrightarrow{a_1} \dots \xrightarrow{a_k} g$

We can apply to the Belief-State space any search algorithm.

- if a solution for b has been found, then any $b' \subseteq b$ is solvable
- if $b' \subseteq b$ has already been generated and discarded, then we can discard a path reaching a belief state b

⇒ Dramatically improves efficiency

Belief-State Problem Formulation [cont.]

Remarks

- 1 if $b \subseteq b'$, then $\text{Result}(b, a) \subseteq \text{Result}(b', a)$ (b more informative than b')
- 2 If a is deterministic, then $|\text{Result}(b, a)| \leq |b|$
- 3 The agent might achieve the goal earlier than $\text{Goal/Test}(b)$ holds, but it does not know it (because he knows it only when **all** states in the belief state are goal states)

Properties

- An action sequence is a **solution** for b iff it leads b to a goal
- If an action sequence is a solution for a belief state b , then it is also a solution for any belief state b' s.t. $b' \subseteq b$
 - if $b \xrightarrow{a_1} \dots \xrightarrow{a_k} g$, then $b' \xrightarrow{a_1} \dots \xrightarrow{a_k} g$

We can apply to the Belief-State space any search algorithm.

- if a solution for b has been found, then any $b' \subseteq b$ is solvable
- if $b' \subseteq b$ has already been generated and discarded, then we can discard a path reaching a belief state b

⇒ Dramatically improves efficiency

Exercises

- ➊ Consider the Sensorless Vacuum Cleaner (goal: both A,B clean)
 - applying graph BFS or DFS, expansion order $\{L, R, S\}$, find a valid plan or show there is none
 - applying tree BFS or DFS with loop detection, expansion order $\{L, R, S\}$, find a valid plan or show there is none
- ➋ Consider (the sensorless version of) the Slippery vacuum cleaner (goal: both A,B clean)
 - draw the Belief State Space
 - applying graph BFS or DFS, expansion order $\{L, R, S\}$, find a valid plan or show there is none
 - applying tree BFS or DFS with loop detection, expansion order $\{L, R, S\}$, find a valid plan or show there is none
- ➌ Consider (the sensorless version of) the Erratic vacuum cleaner (goal: both A,B clean)
 - do the same as with exercise 2
- ➍ Consider a defective variant of the basic vacuum cleaner, in which “suck” may either succeed or not in making the current location clean
 - do the same as with exercise 2

Outline

- 1 Local Search and Optimization
 - General Ideas
 - Hill-Climbing
 - Simulated Annealing
 - Local Beam Search & Genetic Algorithms
- 2 Search with Nondeterministic Actions
- 3 Search with Partial or No Observations (Deterministic/Nondeterministic Actions)
 - Search with No Observations
 - Search with Partial Observations
- 4 Online Search (aka Exploration)

Search with Observations

Perception and Belief-State Problem Formulation

- $Percept(s)$ returns the percept received in state s
 - ex: local-sensing vacuum cleaner, can perceive dirty/clean only on the current position:
 $Percept(1) = [A, Dirty]$
 - with fully observable problems: $Percept(s) = s, \forall s$
 - with sensorless problems: $Percept(s) = null, \forall s$
- Partial observations: many states can produce the same percept
 - ex: $Percept(1) = Percept(3) = [A, Dirty]$
 $\Rightarrow Percept(s)$ may correspond to many different candidate states s
- $Actions()$, $StepCost()$, $GoalTest()$: as with sensorless case

Note

- If sensing were nondeterministic, then a function $Percepts(s)$ would return a set of possible percepts
- Hereafter, we assume that sensing is deterministic

Search with Observations

Perception and Belief-State Problem Formulation

- *Percept(s)* returns the percept received in state s
 - ex: *local-sensing vacuum cleaner*, can perceive dirty/clean only on the current position:
 $Percept(1) = [A, Dirty]$
 - with *fully observable problems*: $Percept(s) = s, \forall s$
 - with *sensorless problems*: $Percept(s) = null, \forall s$
- *Partial observations*: many states can produce the same percept
 - ex: $Percept(1) = Percept(3) = [A, Dirty]$ $\Rightarrow Percept(s)$ may correspond to many different candidate states s
- *Actions()*, *StepCost()*, *GoalTest()*: as with sensorless case

Note

- If sensing were nondeterministic, then a function *Percepts(s)* would return a set of possible percepts
- Hereafter, we assume that sensing is deterministic

Search with Observations

Perception and Belief-State Problem Formulation

- *Percept(s)* returns the percept received in state s
 - ex: local-sensing vacuum cleaner, can perceive dirty/clean only on the current position:
 $Percept(1) = [A, Dirty]$
 - with fully observable problems: $Percept(s) = s, \forall s$
 - with sensorless problems: $Percept(s) = null, \forall s$
- Partial observations: many states can produce the same percept
 - ex: $Percept(1) = Percept(3) = [A, Dirty]$
 $\Rightarrow Percept(s)$ may correspond to many different candidate states s
- *Actions()*, *StepCost()*, *GoalTest()*: as with sensorless case

Note

- If sensing were nondeterministic, then a function *Percepts(s)* would return a set of possible percepts
- Hereafter, we assume that sensing is deterministic

Search with Observations

Perception and Belief-State Problem Formulation

- *Percept(s)* returns the percept received in state s
 - ex: *local-sensing vacuum cleaner*, can perceive dirty/clean only on the current position:
 $Percept(1) = [A, Dirty]$
 - with *fully observable problems*: $Percept(s) = s, \forall s$
 - with *sensorless problems*: $Percept(s) = null, \forall s$
- *Partial observations*: many states can produce the same percept
 - ex: $Percept(1) = Percept(3) = [A, Dirty]$
 $\Rightarrow Percept(s)$ may correspond to many different candidate states s
- *Actions()*, *StepCost()*, *GoalTest()*: as with sensorless case

Note

- If sensing were nondeterministic, then a function *Percepts(s)* would return a *set of possible percepts*
- Hereafter, we assume that sensing is deterministic

Transition Model with (Partial) Perceptions

The Prediction-Observation-Update process

- Three steps:

- 1 **Prediction** (same as for sensorless): predict the belief state after action a

$$\hat{b} = \text{Predict}(b, a) \stackrel{\text{def}}{=} \text{Result}_{(\text{sensorless})}(b, a) = \{s' \mid s' = \text{Result}_P(s, a) \text{ and } s \in b\}$$

- 2 **Observation prediction**: determines the set of percepts that could be observed in the predicted belief state: $\text{PossiblePercepts}(\hat{b}) \stackrel{\text{def}}{=} \{o \mid o = \text{Percept}(s) \text{ and } s \in \hat{b}\}$

- 3 **Update**: for each percept o , determine the belief state b_o , i.e., the subset of states in $\hat{b}$ that could have produced the percept o :

- $b_o = \text{Update}(\hat{b}, o) \stackrel{\text{def}}{=} \{s \mid s \in \hat{b} \text{ and } o = \text{Percept}(s)\}$

$$\Rightarrow \text{Result}(b, a) = \left\{ b_o \mid \begin{array}{l} b_o = \text{Update}(\text{Predict}(b, a), o) \text{ and} \\ o \in \text{PossiblePercepts}(\text{Predict}(b, a)) \end{array} \right\}$$

- set (not union!) of belief states, one for each possible percepts o
- for each o , $b_o \subseteq \hat{b} \Rightarrow$ sensing reduces uncertainty!
- (assume sensing deterministic) the b_o 's are all disjoint (each s belongs to b_o s.t. $o = \text{Percept}(s)$)
 $\Rightarrow$ each next possible percepts o is used to partition $\hat{b}$ into a smaller belief state b_o

$\Rightarrow$ **Non-deterministic** belief-state problem

- due to the inability to predict exactly the next percept

Transition Model with (Partial) Perceptions

The Prediction-Observation-Update process

- Three steps:

- 1 **Prediction** (same as for sensorless): predict the belief state after action a

$$\hat{b} = \text{Predict}(b, a) \stackrel{\text{def}}{=} \text{Result}_{(\text{sensorless})}(b, a) = \{s' \mid s' = \text{Result}_P(s, a) \text{ and } s \in b\}$$

- 2 **Observation prediction**: determines the set of percepts that could be observed in the predicted belief state: $\text{PossiblePercepts}(\hat{b}) \stackrel{\text{def}}{=} \{o \mid o = \text{Percept}(s) \text{ and } s \in \hat{b}\}$

- 3 **Update**: for each percept o , determine the belief state b_o , i.e., the subset of states in $\hat{b}$ that could have produced the percept o :

- $b_o = \text{Update}(\hat{b}, o) \stackrel{\text{def}}{=} \{s \mid s \in \hat{b} \text{ and } o = \text{Percept}(s)\}$

$$\Rightarrow \text{Result}(b, a) = \left\{ b_o \mid \begin{array}{l} b_o = \text{Update}(\text{Predict}(b, a), o) \text{ and} \\ o \in \text{PossiblePercepts}(\text{Predict}(b, a)) \end{array} \right\}$$

- set (not union!) of belief states, one for each possible percepts o
- for each o , $b_o \subseteq \hat{b} \Rightarrow$ sensing reduces uncertainty!
- (assume sensing deterministic) the b_o 's are all disjoint (each s belongs to b_o s.t. $o = \text{Percept}(s)$)
 $\Rightarrow$ each next possible percepts o is used to partition $\hat{b}$ into a smaller belief state b_o

$\Rightarrow$ **Non-deterministic** belief-state problem

- due to the inability to predict exactly the next percept

Transition Model with (Partial) Perceptions

The Prediction-Observation-Update process

- Three steps:

- 1 **Prediction** (same as for sensorless): predict the belief state after action a

$$\hat{b} = \text{Predict}(b, a) \stackrel{\text{def}}{=} \text{Result}_{(\text{sensorless})}(b, a) = \{s' \mid s' = \text{Result}_P(s, a) \text{ and } s \in b\}$$

- 2 **Observation prediction**: determines the set of percepts that could be observed in the predicted belief state: $\text{PossiblePercepts}(\hat{b}) \stackrel{\text{def}}{=} \{o \mid o = \text{Percept}(s) \text{ and } s \in \hat{b}\}$

- 3 **Update**: for each percept o , determine the belief state b_o , i.e., the subset of states in $\hat{b}$ that could have produced the percept o :

- $b_o = \text{Update}(\hat{b}, o) \stackrel{\text{def}}{=} \{s \mid s \in \hat{b} \text{ and } o = \text{Percept}(s)\}$

$$\Rightarrow \text{Result}(b, a) = \left\{ b_o \mid \begin{array}{l} b_o = \text{Update}(\text{Predict}(b, a), o) \text{ and} \\ o \in \text{PossiblePercepts}(\text{Predict}(b, a)) \end{array} \right\}$$

- set (not union!) of belief states, one for each possible percepts o
- for each o , $b_o \subseteq \hat{b} \Rightarrow$ sensing reduces uncertainty!
- (assume sensing deterministic) the b_o 's are all disjoint (each s belongs to b_o s.t. $o = \text{Percept}(s)$)
 $\Rightarrow$ each next possible percepts o is used to partition $\hat{b}$ into a smaller belief state b_o

$\Rightarrow$ **Non-deterministic** belief-state problem

- due to the inability to predict exactly the next percept

Transition Model with (Partial) Perceptions

The Prediction-Observation-Update process

- Three steps:

- 1 **Prediction** (same as for sensorless): predict the belief state after action a

$$\hat{b} = \text{Predict}(b, a) \stackrel{\text{def}}{=} \text{Result}_{(\text{sensorless})}(b, a) = \{s' \mid s' = \text{Result}_P(s, a) \text{ and } s \in b\}$$

- 2 **Observation prediction**: determines the set of percepts that could be observed in the predicted belief state: $\text{PossiblePercepts}(\hat{b}) \stackrel{\text{def}}{=} \{o \mid o = \text{Percept}(s) \text{ and } s \in \hat{b}\}$

- 3 **Update**: for each percept o , determine the belief state b_o , i.e., the subset of states in $\hat{b}$ that could have produced the percept o :

- $b_o = \text{Update}(\hat{b}, o) \stackrel{\text{def}}{=} \{s \mid s \in \hat{b} \text{ and } o = \text{Percept}(s)\}$

$$\Rightarrow \text{Result}(b, a) = \left\{ b_o \mid \begin{array}{l} b_o = \text{Update}(\text{Predict}(b, a), o) \text{ and} \\ o \in \text{PossiblePercepts}(\text{Predict}(b, a)) \end{array} \right\}$$

- set (not union!) of belief states, one for each possible percepts o
- for each o , $b_o \subseteq \hat{b} \Rightarrow$ sensing reduces uncertainty!
- (assume sensing deterministic) the b_o 's are all disjoint (each s belongs to b_o s.t. $o = \text{Percept}(s)$)
 $\Rightarrow$ each next possible percepts o is used to partition $\hat{b}$ into a smaller belief state b_o

$\Rightarrow$ **Non-deterministic** belief-state problem

- due to the inability to predict exactly the next percept

Transition Model with (Partial) Perceptions

The Prediction-Observation-Update process

- Three steps:

- 1 **Prediction** (same as for sensorless): predict the belief state after action a

$$\hat{b} = \text{Predict}(b, a) \stackrel{\text{def}}{=} \text{Result}_{(\text{sensorless})}(b, a) = \{s' \mid s' = \text{Result}_P(s, a) \text{ and } s \in b\}$$

- 2 **Observation prediction**: determines the set of percepts that could be observed in the predicted belief state: $\text{PossiblePercepts}(\hat{b}) \stackrel{\text{def}}{=} \{o \mid o = \text{Percept}(s) \text{ and } s \in \hat{b}\}$

- 3 **Update**: for each percept o , determine the belief state b_o , i.e., the subset of states in $\hat{b}$ that could have produced the percept o :

- $b_o = \text{Update}(\hat{b}, o) \stackrel{\text{def}}{=} \{s \mid s \in \hat{b} \text{ and } o = \text{Percept}(s)\}$

$$\Rightarrow \text{Result}(b, a) = \left\{ b_o \mid \begin{array}{l} b_o = \text{Update}(\text{Predict}(b, a), o) \text{ and} \\ o \in \text{PossiblePercepts}(\text{Predict}(b, a)) \end{array} \right\}$$

- set (not union!) of belief states, one for each possible percepts o
- for each o , $b_o \subseteq \hat{b} \Rightarrow$ sensing reduces uncertainty!
- (assume sensing deterministic) **the b_o 's are all disjoint** (each s belongs to b_o s.t. $o = \text{Percept}(s)$)
 $\Rightarrow$ each next possible percepts o is used to partition $\hat{b}$ into a smaller belief state b_o

$\Rightarrow$ **Non-deterministic** belief-state problem

- due to the inability to predict exactly the next percept

Transition Model with (Partial) Perceptions

The Prediction-Observation-Update process

- Three steps:

- 1 **Prediction** (same as for sensorless): predict the belief state after action a

$$\hat{b} = \text{Predict}(b, a) \stackrel{\text{def}}{=} \text{Result}_{(\text{sensorless})}(b, a) = \{s' \mid s' = \text{Result}_P(s, a) \text{ and } s \in b\}$$

- 2 **Observation prediction**: determines the set of percepts that could be observed in the predicted belief state: $\text{PossiblePercepts}(\hat{b}) \stackrel{\text{def}}{=} \{o \mid o = \text{Percept}(s) \text{ and } s \in \hat{b}\}$

- 3 **Update**: for each percept o , determine the belief state b_o , i.e., the subset of states in $\hat{b}$ that could have produced the percept o :

- $b_o = \text{Update}(\hat{b}, o) \stackrel{\text{def}}{=} \{s \mid s \in \hat{b} \text{ and } o = \text{Percept}(s)\}$

$$\Rightarrow \text{Result}(b, a) = \left\{ b_o \mid \begin{array}{l} b_o = \text{Update}(\text{Predict}(b, a), o) \text{ and} \\ o \in \text{PossiblePercepts}(\text{Predict}(b, a)) \end{array} \right\}$$

- set (not union!) of belief states, one for each possible percepts o
- for each o , $b_o \subseteq \hat{b} \Rightarrow$ sensing reduces uncertainty!
- (assume sensing deterministic) the b_o 's are all disjoint (each s belongs to b_o s.t. $o = \text{Percept}(s)$)
 $\Rightarrow$ each next possible percepts o is used to partition $\hat{b}$ into a smaller belief state b_o

$\Rightarrow$ Non-deterministic belief-state problem

- due to the inability to predict exactly the next percept

Transition Model with (Partial) Perceptions

The Prediction-Observation-Update process

- Three steps:

- 1 **Prediction** (same as for sensorless): predict the belief state after action a

$$\hat{b} = \text{Predict}(b, a) \stackrel{\text{def}}{=} \text{Result}_{(\text{sensorless})}(b, a) = \{s' \mid s' = \text{Result}_P(s, a) \text{ and } s \in b\}$$

- 2 **Observation prediction**: determines the set of percepts that could be observed in the predicted belief state: $\text{PossiblePercepts}(\hat{b}) \stackrel{\text{def}}{=} \{o \mid o = \text{Percept}(s) \text{ and } s \in \hat{b}\}$

- 3 **Update**: for each percept o , determine the belief state b_o , i.e., the subset of states in $\hat{b}$ that could have produced the percept o :

- $b_o = \text{Update}(\hat{b}, o) \stackrel{\text{def}}{=} \{s \mid s \in \hat{b} \text{ and } o = \text{Percept}(s)\}$

$$\Rightarrow \text{Result}(b, a) = \left\{ b_o \mid \begin{array}{l} b_o = \text{Update}(\text{Predict}(b, a), o) \text{ and} \\ o \in \text{PossiblePercepts}(\text{Predict}(b, a)) \end{array} \right\}$$

- set (not union!) of belief states, one for each possible percepts o
- for each o , $b_o \subseteq \hat{b} \Rightarrow$ sensing reduces uncertainty!
- (assume sensing deterministic) the b_o 's are all disjoint (each s belongs to b_o s.t. $o = \text{Percept}(s)$)
 $\Rightarrow$ each next possible percepts o is used to partition $\hat{b}$ into a smaller belief state b_o

$\Rightarrow$ Non-deterministic belief-state problem

- due to the inability to predict exactly the next percept

Transition Model with (Partial) Perceptions

The Prediction-Observation-Update process

- Three steps:

- 1 **Prediction** (same as for sensorless): predict the belief state after action a

$$\hat{b} = \text{Predict}(b, a) \stackrel{\text{def}}{=} \text{Result}_{(\text{sensorless})}(b, a) = \{s' \mid s' = \text{Result}_P(s, a) \text{ and } s \in b\}$$

- 2 **Observation prediction**: determines the set of percepts that could be observed in the predicted belief state: $\text{PossiblePercepts}(\hat{b}) \stackrel{\text{def}}{=} \{o \mid o = \text{Percept}(s) \text{ and } s \in \hat{b}\}$

- 3 **Update**: for each percept o , determine the belief state b_o , i.e., the subset of states in $\hat{b}$ that could have produced the percept o :

- $b_o = \text{Update}(\hat{b}, o) \stackrel{\text{def}}{=} \{s \mid s \in \hat{b} \text{ and } o = \text{Percept}(s)\}$

$$\Rightarrow \text{Result}(b, a) = \left\{ b_o \mid \begin{array}{l} b_o = \text{Update}(\text{Predict}(b, a), o) \text{ and} \\ o \in \text{PossiblePercepts}(\text{Predict}(b, a)) \end{array} \right\}$$

- set (not union!) of belief states, one for each possible percepts o
- for each o , $b_o \subseteq \hat{b} \Rightarrow$ sensing reduces uncertainty!
- (assume sensing deterministic) **the b_o 's are all disjoint** (each s belongs to b_o s.t. $o = \text{Percept}(s)$)
 $\Rightarrow$ **each next possible percepts o is used to partition $\hat{b}$ into a smaller belief state b_o**

$\Rightarrow$ **Non-deterministic** belief-state problem

- due to the inability to predict exactly the next percept

Transition Model with (Partial) Perceptions

The Prediction-Observation-Update process

- Three steps:

- 1 **Prediction** (same as for sensorless): predict the belief state after action a

$$\hat{b} = \text{Predict}(b, a) \stackrel{\text{def}}{=} \text{Result}_{(\text{sensorless})}(b, a) = \{s' \mid s' = \text{Result}_P(s, a) \text{ and } s \in b\}$$

- 2 **Observation prediction**: determines the set of percepts that could be observed in the predicted belief state: $\text{PossiblePercepts}(\hat{b}) \stackrel{\text{def}}{=} \{o \mid o = \text{Percept}(s) \text{ and } s \in \hat{b}\}$

- 3 **Update**: for each percept o , determine the belief state b_o , i.e., the subset of states in $\hat{b}$ that could have produced the percept o :

- $b_o = \text{Update}(\hat{b}, o) \stackrel{\text{def}}{=} \{s \mid s \in \hat{b} \text{ and } o = \text{Percept}(s)\}$

$$\Rightarrow \text{Result}(b, a) = \left\{ b_o \mid \begin{array}{l} b_o = \text{Update}(\text{Predict}(b, a), o) \text{ and} \\ o \in \text{PossiblePercepts}(\text{Predict}(b, a)) \end{array} \right\}$$

- set (not union!) of belief states, one for each possible percepts o
- for each o , $b_o \subseteq \hat{b} \Rightarrow$ sensing reduces uncertainty!
- (assume sensing deterministic) **the b_o 's are all disjoint** (each s belongs to b_o s.t. $o = \text{Percept}(s)$)
 $\Rightarrow$ each next possible percepts o is used to partition $\hat{b}$ into a smaller belief state b_o

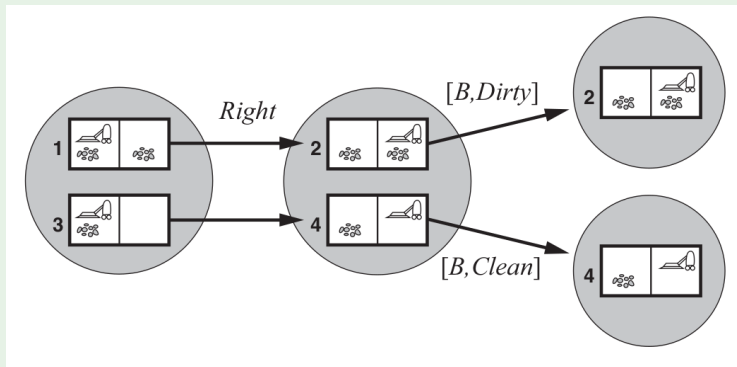
$\Rightarrow$ **Non-deterministic** belief-state problem

- due to the inability to predict exactly the next percept

Transition Model with Perceptions: Example

Deterministic actions: Local-sensing vacuum cleaner

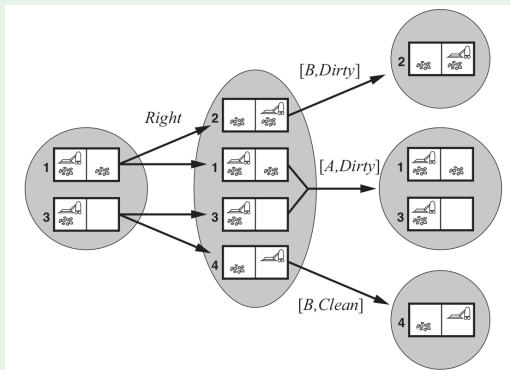
- $\hat{b} = \text{Predict}(\{1, 3\}, \text{Right}) = \{2, 4\}$
- $\text{PossiblePercepts}(\hat{b}) = \{[B, \text{Dirty}], [B, \text{Clean}]\}$
- $\text{Result}(\{1, 3\}, \text{Right}) = \{\{2\}, \{4\}\}$



Transition Model with Perceptions: Example

Nondeterministic actions: Slippery local-sensing vacuum cleaner

- $\hat{b} = \text{Predict}(\{1, 3\}, \text{Right}) = \{1, 2, 3, 4\}$
- $\text{PossiblePercepts}(\hat{b}) = \{[B, \text{Dirty}], [A, \text{Dirty}], [B, \text{Clean}]\}$
- $\text{Result}(\{1, 3\}, \text{Right}) = \{\{2\}, \{1, 3\}, \{4\}\}$



Solving Partially-Observable Problems

- Formulation as a **nondeterministic belief-state search problem**

- non-determinism due to different possible percepts

⇒ The AND-OR search algorithms can be applied

⇒ The solution is a conditional plan

Solution for initial percept [A, Dirty] (deterministic): [Suck, Right, if Bstate = {6} then Suck else []]

First level:
(draw second level
for exercise)



Solving Partially-Observable Problems

- Formulation as a **nondeterministic belief-state search problem**

- non-determinism due to different possible percepts

⇒ The AND-OR search algorithms can be applied

⇒ The solution is a conditional plan

Solution for initial percept [A, Dirty] (deterministic): [Suck, Right, if Bstate = {6} then Suck else []]

First level:
(draw second level
for exercise)



Solving Partially-Observable Problems

- Formulation as a **nondeterministic belief-state search problem**

- non-determinism due to different possible percepts

⇒ The AND-OR search algorithms can be applied

⇒ **The solution is a conditional plan**

Solution for initial percept [A, Dirty] (deterministic): [Suck, Right, if Bstate = {6} then Suck else []]

First level:
(draw second level
for exercise)



Solving Partially-Observable Problems

- Formulation as a **nondeterministic belief-state search problem**

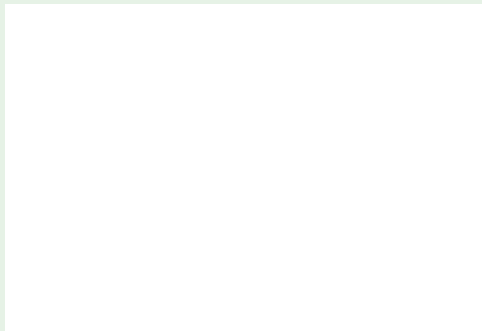
- non-determinism due to different possible percepts

⇒ The AND-OR search algorithms can be applied

⇒ **The solution is a conditional plan**

Solution for initial percept [A, Dirty] (deterministic): [Suck, Right, if Bstate = {6} then Suck else []]

First level:
(draw second level
for exercise)



Solving Partially-Observable Problems

- Formulation as a **nondeterministic belief-state search problem**

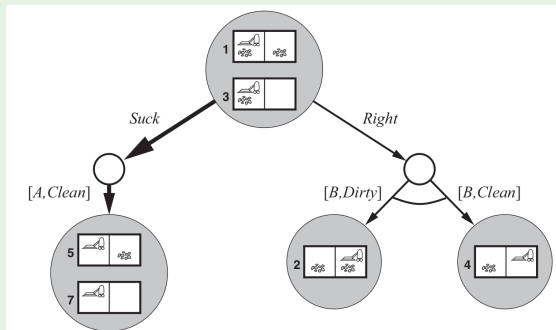
- non-determinism due to different possible percepts

⇒ The AND-OR search algorithms can be applied

⇒ **The solution is a conditional plan**

Solution for initial percept **[A, Dirty]** (deterministic): **[Suck, Right, if Bstate = {6} then Suck else []]**

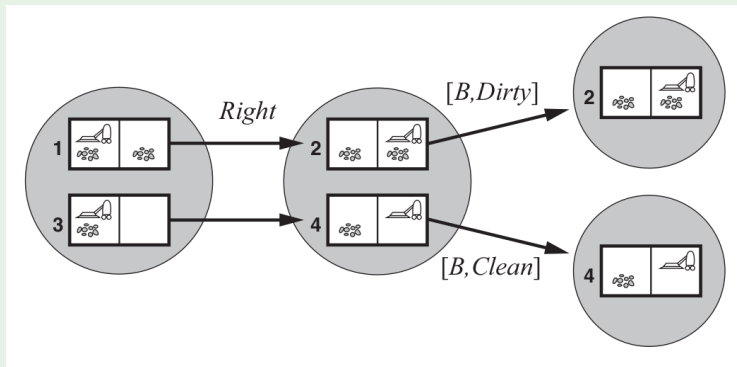
First level:
(draw second level
for exercise)



Transition Model with Perceptions: Example

Deterministic actions: Local-sensing vacuum cleaner

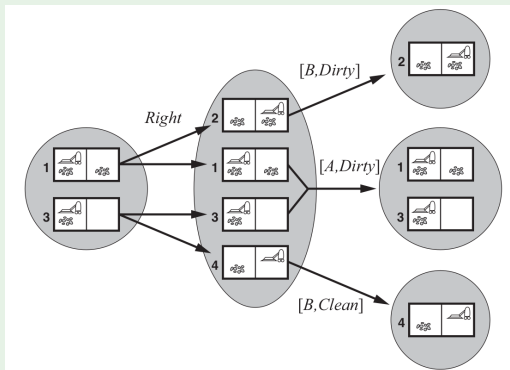
- $\hat{b} = \text{Predict}(\{1, 3\}, \text{Right}) = \{2, 4\}$
- $\text{PossiblePercepts}(\hat{b}) = \{[B, \text{Dirty}], [B, \text{Clean}]\}$
- $\text{Result}(\{1, 3\}, \text{Right}) = \{\{2\}, \{4\}\}$



Transition Model with Perceptions: Example

Nondeterministic actions: Slippery local-sensing vacuum cleaner

- $\hat{b} = \text{Predict}(\{1, 3\}, \text{Right}) = \{1, 2, 3, 4\}$
- $\text{PossiblePercepts}(\hat{b}) = \{[B, \text{Dirty}], [A, \text{Dirty}], [B, \text{Clean}]\}$
- $\text{Result}(\{1, 3\}, \text{Right}) = \{\{2\}, \{1, 3\}, \{4\}\}$



Solving Partially-Observable Problems

- Formulation as a **nondeterministic belief-state search problem**

- non-determinism due to different possible percepts

⇒ The AND-OR search algorithms can be applied

⇒ The solution is a conditional plan

Solution for initial percept [A, Dirty] (deterministic): [Suck, Right, if Bstate = {6} then Suck else []]

First level:
(draw second level
for exercise)



Solving Partially-Observable Problems

- Formulation as a **nondeterministic belief-state search problem**

- non-determinism due to different possible percepts

⇒ The AND-OR search algorithms can be applied

⇒ The solution is a conditional plan

Solution for initial percept [A, Dirty] (deterministic): [Suck, Right, if Bstate = {6} then Suck else []]

First level:
(draw second level
for exercise)



Solving Partially-Observable Problems

- Formulation as a **nondeterministic belief-state search problem**

- non-determinism due to different possible percepts

⇒ The AND-OR search algorithms can be applied

⇒ **The solution is a conditional plan**

Solution for initial percept [A, Dirty] (deterministic): [Suck, Right, if Bstate = {6} then Suck else []]

First level:
(draw second level
for exercise)



Solving Partially-Observable Problems

- Formulation as a **nondeterministic belief-state search problem**

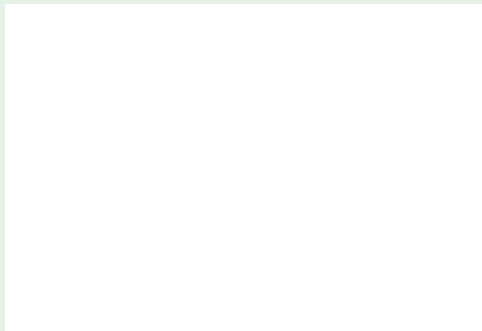
- non-determinism due to different possible percepts

⇒ The AND-OR search algorithms can be applied

⇒ **The solution is a conditional plan**

Solution for initial percept [A, Dirty] (deterministic): [Suck, Right, if Bstate = {6} then Suck else []]

First level:
(draw second level
for exercise)



Solving Partially-Observable Problems

- Formulation as a **nondeterministic belief-state search problem**

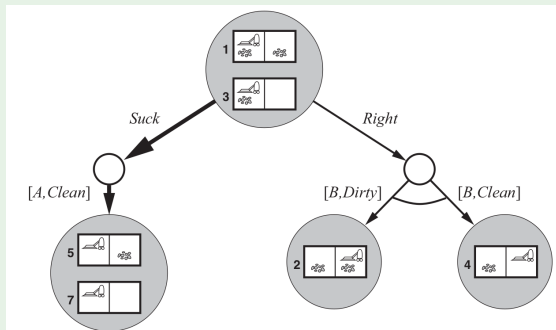
- non-determinism due to different possible percepts

⇒ The AND-OR search algorithms can be applied

⇒ **The solution is a conditional plan**

Solution for initial percept **[A, Dirty]** (deterministic): **[Suck, Right, if Bstate = {6} then Suck else []]**

First level:
(draw second level
for exercise)



Exercises

- 1 Consider the deterministic Local-sensing Vacuum Cleaner (goal: both A,B clean)
 - applying graph AND-OR search expansion order $\{L, R, S\}$, find a valid plan or show there is none
 - do it for different initial belief states (e.g., $\langle ?, \langle ?, ? \rangle \rangle$, $\langle A, \langle ?, ? \rangle \rangle$, $\langle A, \langle Dirty, ? \rangle \rangle$, $\langle A, \langle Clean, ? \rangle \rangle$, ...)
- 2 Same as #1, with the slippery Local-sensing Vacuum Cleaner
- 3 Same as #1, with the defective variant (“Suck” may or may not be effective) of Local-sensing Vacuum Cleaner

An Agent for Partially-Observable Environments

- Agent quite similar to the simple problem-solving agent [Ch.3]:
 - formulates a problem (as a belief-state search)
 - calls a search algorithm (an AND-OR-GRAPH one)
 - executes the solution
- Two main differences:
 - the solution is a conditional plan, not an action sequence
 - in step (3) the agent needs to maintain its belief state as it performs actions and receives percepts (aka monitoring, filtering, state estimation)
- State estimation resembles the prediction-observation-update process:
 - simpler, because the percept o is given by the environment
⇒ no need to calculate it
 - given b , a and o : $b' = \text{Update}(\text{Predict}(b, a), o)$

Remark

The computation has to happen as fast as percepts are coming in
⇒ in some complex applications, compute approximate belief states

An Agent for Partially-Observable Environments

- Agent quite similar to the simple problem-solving agent [Ch.3]:
 - 1 formulates a problem (as a belief-state search)
 - 2 calls a search algorithm (an AND-OR-GRAPH one)
 - 3 executes the solution
- Two main differences:
 - the solution is a conditional plan, not an action sequence
 - in step (3) the agent needs to maintain its belief state as it performs actions and receives percepts (aka monitoring, filtering, state estimation)
- State estimation resembles the prediction-observation-update process:
 - simpler, because the percept o is given by the environment
⇒ no need to calculate it
 - given b , a and o : $b' = \text{Update}(\text{Predict}(b, a), o)$

Remark

The computation has to happen as fast as percepts are coming in
⇒ in some complex applications, compute **approximate** belief states

An Agent for Partially-Observable Environments

- Agent quite similar to the simple problem-solving agent [Ch.3]:
 - 1 formulates a problem (as a belief-state search)
 - 2 calls a search algorithm (an AND-OR-GRAPH one)
 - 3 executes the solution
- Two main differences:
 - the solution is a conditional plan, not an action sequence
 - in step (3) the agent needs to maintain its belief state as it performs actions and receives percepts (aka monitoring, filtering, state estimation)
- State estimation resembles the prediction-observation-update process:
 - simpler, because the percept o is given by the environment
⇒ no need to calculate it
 - given b , a and o : $b' = \text{Update}(\text{Predict}(b, a), o)$

Remark

The computation has to happen as fast as percepts are coming in
⇒ in some complex applications, compute approximate belief states

An Agent for Partially-Observable Environments

- Agent quite similar to the simple problem-solving agent [Ch.3]:
 - 1 formulates a problem (as a belief-state search)
 - 2 calls a search algorithm (an AND-OR-GRAPH one)
 - 3 executes the solution
- Two main differences:
 - the solution is a conditional plan, not an action sequence
 - in step (3) the agent needs to maintain its belief state as it performs actions and receives percepts (aka monitoring, filtering, state estimation)
- State estimation resembles the prediction-observation-update process:
 - simpler, because the percept o is given by the environment
⇒ no need to calculate it
 - given b , a and o : $b' = \text{Update}(\text{Predict}(b, a), o)$

Remark

The computation has to happen as fast as percepts are coming in
⇒ in some complex applications, compute approximate belief states

An Agent for Partially-Observable Environments

- Agent quite similar to the simple problem-solving agent [Ch.3]:
 - 1 formulates a problem (as a belief-state search)
 - 2 calls a search algorithm (an AND-OR-GRAPH one)
 - 3 executes the solution
- Two main differences:
 - the solution is a conditional plan, not an action sequence
 - in step (3) the agent needs to maintain its belief state as it performs actions and receives percepts (aka monitoring, filtering, state estimation)
- State estimation resembles the prediction-observation-update process:
 - simpler, because the percept o is given by the environment
⇒ no need to calculate it
 - given b , a and o : $b' = \text{Update}(\text{Predict}(b, a), o)$

Remark

The computation has to happen as fast as percepts are coming in
⇒ in some complex applications, compute approximate belief states

Example: Belief-State Maintenance

Example: Kindergarden Vacuum-Cleaner

- local sensing $\implies$ partially observable
- any square may become dirty at any time unless the agent is actively cleaning it at that moment
 $\implies$ nondeterministic

● Ex: $\text{Update}(\overbrace{\text{Predict}(\{1, 3\}, \text{Suck})}^{\{5, 7\}}, [A, \text{Clean}]) = \{5, 7\}$

● Ex: $\text{Update}(\overbrace{\text{Predict}(\{5, 7\}, \text{Right})}^{\{2, 4, 6, 8\}}, [B, \text{Dirty}]) = \{2, 6\}$

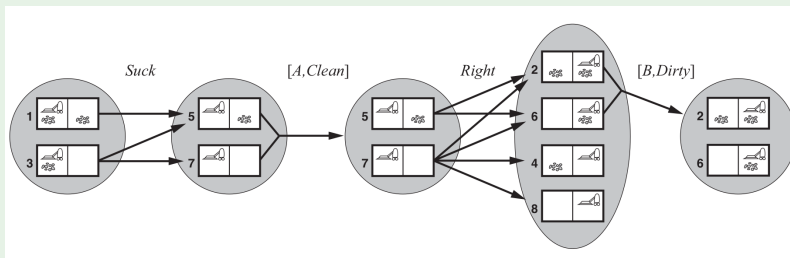
Example: Belief-State Maintenance

Example: Kindergarden Vacuum-Cleaner

- local sensing $\implies$ partially observable
- any square may become dirty at any time unless the agent is actively cleaning it at that moment
 $\implies$ nondeterministic

● Ex: $\text{Update}(\overbrace{\text{Predict}(\{1, 3\}, \text{Suck})}^{\{5,7\}}, [A, \text{Clean}]) = \{5, 7\}$

● Ex: $\text{Update}(\overbrace{\text{Predict}(\{5, 7\}, \text{Right})}^{\{2,4,6,8\}}, [B, \text{Dirty}]) = \{2, 6\}$



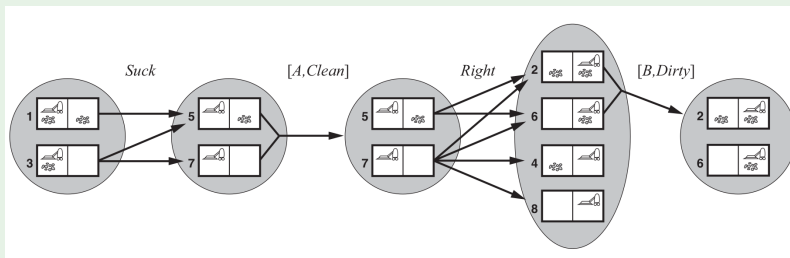
Example: Belief-State Maintenance

Example: Kindergarden Vacuum-Cleaner

- local sensing $\implies$ partially observable
- any square may become dirty at any time unless the agent is actively cleaning it at that moment
 $\implies$ nondeterministic

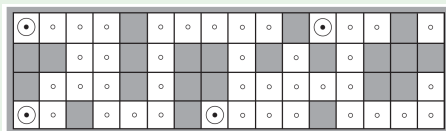
● Ex: $Update(\overbrace{Predict(\{1, 3\}, Suck)}^{\{5,7\}}, [A, Clean]) = \{5, 7\}$

● Ex: $Update(\overbrace{Predict(\{5, 7\}, Right)}^{\{2,4,6,8\}}, [B, Dirty]) = \{2, 6\}$

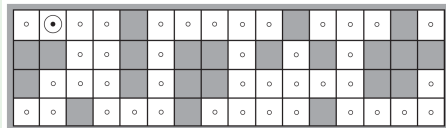


Example:

- Knows the map, senses walls in the four directions (NESW)
 - localization broken: does not know where it is
 - navigation broken: does not know the direction is moving to $\implies$ move is nondeterministic
 - goal: localization (know where it is)
- $b = \{\text{all locations}\}$, $o = \text{NSW}$
 - $b_o = \text{Update}(b, \text{NSW}) = (a)$
 - $b_o = \text{Update}(\text{Predict}(\text{Update}(b, \text{NSW}), \text{Move}), \text{NS}) = (b)$



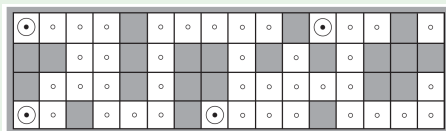
(a) Possible locations of robot after $E_1 = \text{NSW}$



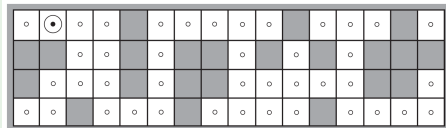
(b) Possible locations of robot After $E_1 = \text{NSW}, E_2 = \text{NS}$

Example:

- Knows the map, senses walls in the four directions (NESW)
 - localization broken: does not know where it is
 - navigation broken: does not know the direction is moving to $\implies$ move is nondeterministic
 - goal: localization (know where it is)
- $b = \{\text{all locations}\}$, $o = \text{NSW}$
 - $b_o = \text{Update}(b, \text{NSW}) = (a)$
 - $b_o = \text{Update}(\text{Predict}(\text{Update}(b, \text{NSW}), \text{Move}), \text{NS}) = (b)$



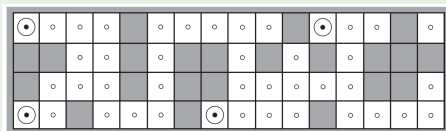
(a) Possible locations of robot after $E_1 = \text{NSW}$



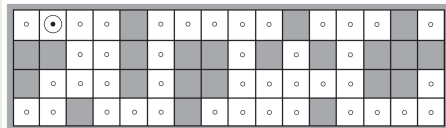
(b) Possible locations of robot After $E_1 = \text{NSW}, E_2 = \text{NS}$

Example:

- Knows the map, senses walls in the four directions (NESW)
 - localization broken: does not know where it is
 - navigation broken: does not know the direction is moving to $\implies$ move is nondeterministic
 - goal: localization (know where it is)
- $b = \{\text{all locations}\}$, $o = \text{NSW}$
 - 1 $b_o = \text{Update}(b, \text{NSW}) = (a)$
 - 2 $b_o = \text{Update}(\text{Predict}(\text{Update}(b, \text{NSW}), \text{Move}), \text{NS}) = (b)$



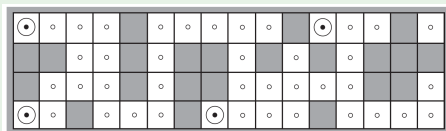
(a) Possible locations of robot after $E_1 = \text{NSW}$



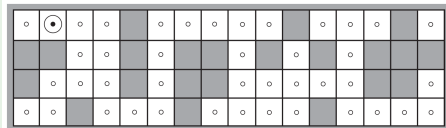
(b) Possible locations of robot After $E_1 = \text{NSW}, E_2 = \text{NS}$

Example:

- Knows the map, senses walls in the four directions (NESW)
 - localization broken: does not know where it is
 - navigation broken: does not know the direction is moving to $\implies$ move is nondeterministic
 - goal: localization (know where it is)
- $b = \{\text{all locations}\}$, $o = \text{NSW}$
 - 1 $b_o = \text{Update}(b, \text{NSW}) = (a)$
 - 2 $b_o = \text{Update}(\text{Predict}(\text{Update}(b, \text{NSW}), \text{Move}), \text{NS}) = (b)$



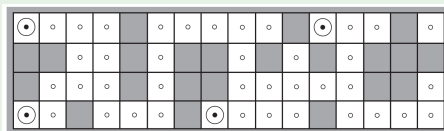
(a) Possible locations of robot after $E_1 = \text{NSW}$



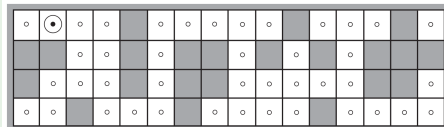
(b) Possible locations of robot After $E_1 = \text{NSW}, E_2 = \text{NS}$

Example:

- Knows the map, senses walls in the four directions (NESW)
 - localization broken: does not know where it is
 - navigation broken: does not know the direction is moving to $\implies$ move is nondeterministic
 - goal: localization (know where it is)
- $b = \{all\ locations\}$, $o = NSW$
 - 1 $b_o = Update(b, NSW) = (a)$
 - 2 $b_o = Update(Predict(Update(b, NSW), Move), NS) = (b)$



(a) Possible locations of robot after $E_1 = NSW$



(b) Possible locations of robot After $E_1 = NSW, E_2 = NS$

Outline

- 1 Local Search and Optimization
 - General Ideas
 - Hill-Climbing
 - Simulated Annealing
 - Local Beam Search & Genetic Algorithms
- 2 Search with Nondeterministic Actions
- 3 Search with Partial or No Observations (Deterministic/Nondeterministic Actions)
 - Search with No Observations
 - Search with Partial Observations
- 4 Online Search (aka Exploration)

Recall: Generalities

- So far we address a single category of problems:
 - 1 observable,
 - 2 deterministic,
 - 3 with known environment,
 - 4 s.t. the solution is a sequence of actions.
- What happens when these assumptions are relaxed?
- In order we will:
 - release condition 4 $\implies$ local search
 - release condition 2 $\implies$ search with non-deterministic actions
 - release condition 1 $\implies$ search with no observability or with partial observability
 - release condition 3 $\implies$ online search

Generalities

Online vs. offline search

- So far: **Offline search**
 - it computes a complete solutions before executing it
- **Online search**: agent interleaves computation and action
 - it takes an action,
 - then it observes the environment and computes the next action
 - (repeat)
- Necessary in **dynamic domains** or **unknown domains**
 - cannot know the states and consequences of actions
 - faces an **exploration problem**: must use actions as experiments in order to learn enough
 - ex: a robot placed in a new building $\implies$ must explore it to build a map for getting from A to B
 - ex: newborn baby $\implies$ acts to learn the outcome of his/her actions
- Useful in **nondeterministic domains**
 - prevents search blowup

Must be solved by executing actions, rather than by pure computation

Generalities

Online vs. offline search

- So far: **Offline search**
 - it computes a complete solutions before executing it
- **Online search**: agent interleaves computation and action
 - it takes an action,
 - then it observes the environment and computes the next action
 - (repeat)
- Necessary in **dynamic domains** or **unknown domains**
 - cannot know the states and consequences of actions
 - faces an **exploration problem**: must use actions as experiments in order to learn enough
 - ex: a robot placed in a new building $\implies$ must explore it to build a map for getting from A to B
 - ex: newborn baby $\implies$ acts to learn the outcome of his/her actions
- Useful in **nondeterministic domains**
 - prevents search blowup

Must be solved by executing actions, rather than by pure computation

Generalities

Online vs. offline search

- So far: **Offline search**
 - it computes a complete solutions before executing it
- **Online search**: agent interleaves computation and action
 - it takes an action,
 - then it observes the environment and computes the next action
 - (repeat)
- Necessary in **dynamic domains** or **unknown domains**
 - cannot know the states and consequences of actions
 - faces an **exploration problem**: must use actions as experiments in order to learn enough
 - ex: a robot placed in a new building $\implies$ must explore it to build a map for getting from A to B
 - ex: newborn baby $\implies$ acts to learn the outcome of his/her actions
- Useful in **nondeterministic domains**
 - prevents search blowup

Must be solved by executing actions, rather than by pure computation

Generalities

Online vs. offline search

- So far: **Offline search**
 - it computes a complete solutions before executing it
- **Online search**: agent interleaves computation and action
 - it takes an action,
 - then it observes the environment and computes the next action
 - (repeat)
- Necessary in **dynamic domains** or **unknown domains**
 - cannot know the states and consequences of actions
 - faces an **exploration problem**: must use actions as experiments in order to learn enough
 - ex: a robot placed in a new building $\implies$ must explore it to build a map for getting from A to B
 - ex: newborn baby $\implies$ acts to learn the outcome of his/her actions
- Useful in **nondeterministic domains**
 - prevents search blowup

Must be solved by executing actions, rather than by pure computation

Generalities

Online vs. offline search

- So far: **Offline search**
 - it computes a complete solutions before executing it
- **Online search**: agent interleaves computation and action
 - it takes an action,
 - then it observes the environment and computes the next action
 - (repeat)
- Necessary in **dynamic domains** or **unknown domains**
 - cannot know the states and consequences of actions
 - faces an **exploration problem**: must use actions as experiments in order to learn enough
 - ex: a robot placed in a new building $\implies$ must explore it to build a map for getting from A to B
 - ex: newborn baby $\implies$ acts to learn the outcome of his/her actions
- Useful in **nondeterministic domains**
 - prevents search blowup

Must be solved by executing actions, rather than by pure computation

Working Hypotheses

- Assumption: a deterministic and fully observable environment
- The agent knows only
 - $Actions(s)$, which returns the list of actions allowed in s
 - the step-cost function $c(s, a, s')$ (cannot be used until s' is known)
 - $GoalTest(s)$
- Remark: The agent cannot determine $Result(s, a)$
 - except by actually being in s and doing a
- The agent knows an admissible heuristic function $h(s)$, that estimates the distance from the current state to a goal state
- Objective: reach goal with minimal cost
 - Cost: total cost of traveled path
 - Competitive ratio: ratio of cost over cost of the solution path if search space is known
($+\infty$ if agent in a deadend)

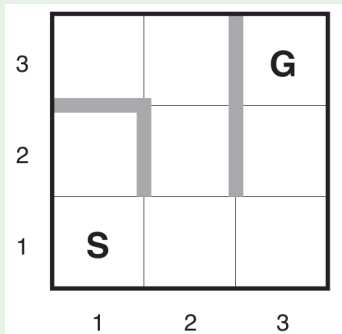
Working Hypotheses

- Assumption: a deterministic and fully observable environment
- The agent knows only
 - $Actions(s)$, which returns the list of actions allowed in s
 - the step-cost function $c(s, a, s')$ (cannot be used until s' is known)
 - $GoalTest(s)$
- Remark: The agent cannot determine $Result(s, a)$
 - except by actually being in s and doing a
- The agent knows an admissible heuristic function $h(s)$, that estimates the distance from the current state to a goal state
- Objective: reach goal with minimal cost
 - Cost: total cost of traveled path
 - Competitive ratio: ratio of cost over cost of the solution path if search space is known
($+\infty$ if agent in a deadend)

Online Search: Example

Example: a simple maze problem

- the agent does not know that going Up from (1,1) leads to (1,2)
- having done that, it does not know that going Down leads to (1,1)

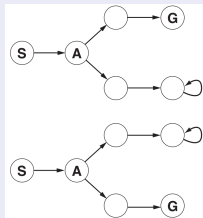


(© S. Russell & P. Norwig, ALMA)

Online Search: Deadends

Inevitability of Deadends

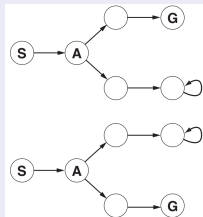
- Online search may face deadends (e.g., with **irreversible actions**)
- **No algorithm can avoid dead ends in all state spaces**
- **Adversary argument**: for each algo, an adversary can construct the state space while the agent explores it
 - If states S and A are visited. What next?
 - ⇒ if algo goes right, adversary builds (top), otherwise builds (bot)
 - ⇒ adversary builds a deadend
- Assumption the state space is **safely explorable**: **some goal state is reachable from every reachable state** (ex: reversible actions)



Online Search: Deadends

Inevitability of Deadends

- Online search may face deadends (e.g., with **irreversible actions**)
- **No algorithm can avoid dead ends in all state spaces**
- **Adversary argument**: for each algo, an adversary can construct the state space while the agent explores it
 - If states S and A are visited. What next?
 - ⇒ if algo goes right, adversary builds (top), otherwise builds (bot)
 - ⇒ adversary builds a deadend
- Assumption the state space is **safely explorable**: **some goal state is reachable from every reachable state** (ex: reversible actions)



Online Search Agents

Online Search Agents: Basic Ideas

- Idea: The agent creates & maintains a map of the environment (*result[s, a]*)
 - map is updated based on percept input after every action
 - map is used to decide next action
- Difference wrt. offline algorithms (ex A^* , BFS)
 - Can only expand the node it is physically in
 - Needs to backtrack physically

Online Search Agents

Online Search Agents: Basic Ideas

- Idea: **The agent creates & maintains a map of the environment** ($result[s, a]$)
 - map is updated based on percept input after every action
 - map is used to decide next action
- Difference wrt. offline algorithms (ex A^* , BFS)
 - Can only expand the node it is physically in
 - Needs to backtrack physically

Online Search Agents

Online Search Agents: Basic Ideas

- Idea: The agent creates & maintains a map of the environment ($result[s, a]$)
 - map is updated based on percept input after every action
 - map is used to decide next action
- Difference wrt. offline algorithms (ex A^* , BFS)
 - Can only expand the node it is physically in
 - ⇒ expand nodes in local order
 - ⇒ DFS natural candidate for an online version
 - Needs to backtrack physically
 - DFS: go back to the state from which the agent most recently entered the current state
 - must keep a table with the predecessor states of each state to which the agent has not yet backtracked ($unbacktracked[s]$)
 - ⇒ backtrack physically (find an action reversing the generation of s)

Online Search Agents

Online Search Agents: Basic Ideas

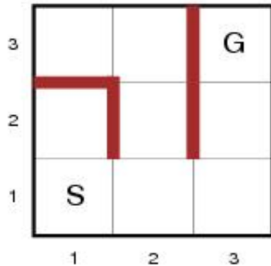
- Idea: The agent creates & maintains a map of the environment ($result[s, a]$)
 - map is updated based on percept input after every action
 - map is used to decide next action
- Difference wrt. offline algorithms (ex A^* , BFS)
 - Can only expand the node it is physically in
 - ⇒ expand nodes in **local order**
 - ⇒ DFS natural candidate for an online version
 - **Needs to backtrack physically**
 - DFS: go back to the state from which the agent most recently entered the current state
 - must keep a table with the **predecessor states** of each state to which the agent has not yet backtracked ($unbacktracked[s]$)
 - ⇒ backtrack physically (**find an action reversing the generation of s**)

Online DFS Search Agents

```
function ONLINE-DFS-AGENT(problem, s') returns an action
    s, a, the previous state and action, initially null
    result, a table mapping (s, a) to s', initially empty
    untried, a table mapping s to a list of untried actions
    unbacktracked, a table mapping s to a list of states never backtracked to

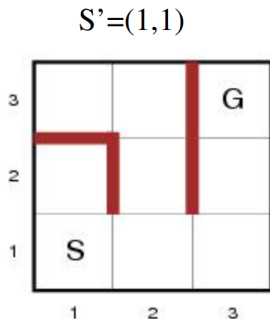
    if problem.IS-GOAL(s') then return stop
    if s' is a new state (not in untried) then untried[s']  $\leftarrow$  problem.ACTIONS(s')
    if s is not null then // if neither initial nor result of backtracking
        result[s, a]  $\leftarrow$  s'
        add s to the front of unbacktracked[s']
    if untried[s'] is empty then //backtrack // results[s',b] exists because untried[s'] is empty
        if unbacktracked[s'] is empty then return stop // added in 4th ed. AIMA
        a  $\leftarrow$  an action b such that result[s', b] = POP(unbacktracked[s']) s'  $\leftarrow$  null
    else a  $\leftarrow$  POP(untried[s']) // all actions in actions(s') have been tried
    s  $\leftarrow$  s'
    return a
```

Online DFS: Example



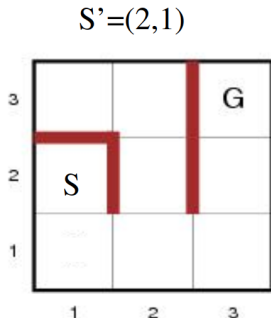
- Assume maze problem on 3x3 grid.
- $s' = (1,1)$ is initial state
- Result, untried, unbacktracked, ... are empty
- S,a are also empty

Online DFS: Example



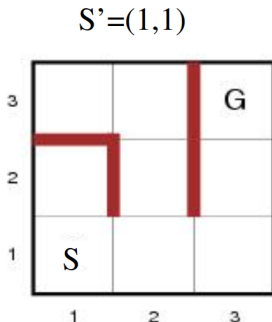
- GOAL-TEST((1,1))?
 - **S not = G thus false**
- (1,1) a new state?
 - **True**
 - **ACTIONS((1,1)) -> untried[(1,1)]**
 - {RIGHT,UP}
- s is null?
 - **True (initially)**
- untried[(1,1)] empty?
 - **False**
- POP(untried[(1,1)])->a
 - **A=UP**
- s = (1,1)
- Return a

Online DFS: Example



- GOAL-TEST((2,1))?
 - **S not = G thus false**
- (2,1) a new state?
 - **True**
 - **ACTION((2,1)) -> untried[(2,1)]**
 - {DOWN}
- s is null?
 - **false (s=(1,1))**
 - **result[UP,(1,1)] <- (2,1)**
 - **unbacktracked[(2,1)]={ (1,1) }**
- untried[(2,1)] empty?
 - **False**
- A=DOWN, s=(2,1) return A

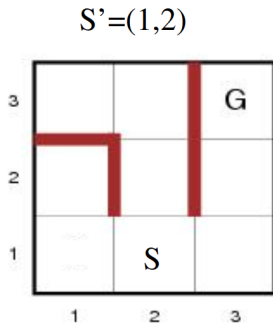
Online DFS: Example



- GOAL-TEST((1,1))?
 - **S not = G thus false**
- (1,1) a new state?
 - **false**
- s is null?
 - **false (s=(2,1))**
 - **result[DOWN,(2,1)] <- (1,1)**
 - **unbacktracked[(1,1)]={ (2,1) }**
- untried[(1,1)] empty?
 - **False**
- A=RIGHT, s=(1,1) return A

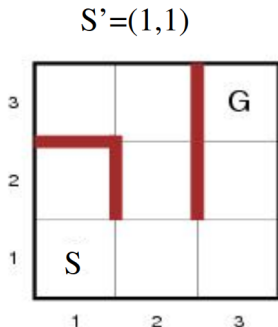
// Notice: here (2,1)->(1,1) is a move, not a form of backtracking

Online DFS: Example



- GOAL-TEST((1,2))?
 - **S not = G thus false**
- (1,2) a new state?
 - **True,**
untried[(1,2)]={RIGHT,UP,LEFT}
- s is null?
 - **false (s=(1,1))**
 - **result[RIGHT,(1,1)] <- (1,2)**
 - **unbacktracked[(1,2)]={{(1,1)}}**
- untried[(1,2)] empty?
 - **False**
- A=LEFT, s=(1,2) return A

Online DFS: Example



- GOAL-TEST($((1,1))$)?
 - **S not = G thus false**
- $(1,1)$ a new state?
 - **false**
- s is null?
 - **false (s=(1,2))**
 - **result[LEFT,(1,2)] <- (1,1)**
 - **unbacktracked[(1,1)]={ (1,2),(2,1) }**
- untried[(1,1)] empty?
 - **True**
 - **unbacktracked[(1,1)] empty?**
False
- $A=b$ for b in result[b,(1,1)]= $(1,2)$
 - **B=RIGHT**
- $A=RIGHT$, $s=(1,1)$...

// Note: here $(1,2) \rightarrow (1,1)$ is a move, not a form of backtracking

Online Search Agents: Facts

- Works only if actions are always reversible
- Worst case: each link $\langle s, a, s' \rangle$ is visited at most twice
 - one as exploration ($a \in \text{untried}[s]$)
 - one as backtracking ($a \in \text{unbacktracked}[s]$)
- An agent can go on a long walk even if it is close to the solution
 - an online iterative deepening approach solves this problem

Online Search Agents: Facts

- Works only if actions are always reversible
- Worst case: each link $\langle s, a, s' \rangle$ is visited at most twice
 - one as exploration ($a \in \text{untried}[s]$)
 - one as backtracking ($a \in \text{unbacktracked}[s]$)
- An agent can go on a long walk even if it is close to the solution
 - an online iterative deepening approach solves this problem

Online Search Agents: Facts

- Works only if actions are always reversible
- Worst case: each link $\langle s, a, s' \rangle$ is visited at most twice
 - one as exploration ($a \in \text{untried}[s]$)
 - one as backtracking ($a \in \text{unbacktracked}[s]$)
- An agent can go on a long walk even if it is close to the solution
 - an online iterative deepening approach solves this problem

Online Local Search

- **Hill Climbing** natural candidate for online search
 - locality of search
 - only one state is stored
 - unfortunately, stuck in local minima
 - **random restarts** not possible
- Possible solution: **Random Walk**
 - selects randomly one available actions from the current state
 - preference can be given to actions that have not yet been tried
 - **eventually finds a goal or complete its exploration** if space is finite
 - unfortunately, **very slow**

Random Walk: example

- random walk takes exponentially many steps to find a goal (backward progress is twice as likely as forward progress)

Online Local Search

- **Hill Climbing** natural candidate for online search
 - locality of search
 - only one state is stored
 - unfortunately, stuck in local minima
 - **random restarts** not possible
- Possible solution: **Random Walk**
 - selects randomly one available actions from the current state
 - preference can be given to actions that have not yet been tried
 - **eventually finds a goal or complete its exploration** if space is finite
 - unfortunately, **very slow**

Random Walk: example

- random walk takes exponentially many steps to find a goal (backward progress is twice as likely as forward progress)

Online Local Search

- **Hill Climbing** natural candidate for online search
 - locality of search
 - only one state is stored
 - unfortunately, stuck in local minima
 - **random restarts** not possible
- Possible solution: **Random Walk**
 - selects randomly one available actions from the current state
 - preference can be given to actions that have not yet been tried
 - **eventually finds a goal or complete its exploration** if space is finite
 - unfortunately, **very slow**

Random Walk: example

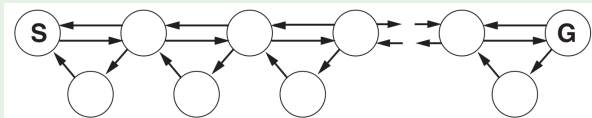
- random walk takes exponentially many steps to find a goal (backward progress is twice as likely as forward progress)

Online Local Search

- **Hill Climbing** natural candidate for online search
 - locality of search
 - only one state is stored
 - unfortunately, stuck in local minima
 - **random restarts** not possible
- Possible solution: **Random Walk**
 - selects randomly one available actions from the current state
 - preference can be given to actions that have not yet been tried
 - **eventually finds a goal or complete its exploration** if space is finite
 - unfortunately, **very slow**

Random Walk: example

- random walk takes exponentially many steps to find a goal (backward progress is twice as likely as forward progress)



(© S. Russell & P. Norvig, AIMA)

Online A^* : $LRTA^*$

$LRTA^*$: General ideas

- Better possible solution: **add memory to hill climbing**
 - Idea: **store a “current best estimate” $H(s)$ of the cost to reach the goal from each state that has been visited**
 - initially $h(s)$
 - updated as the agent gains experience in the state space
- (recall that $h(s)$ is in general “too optimistic”)

⇒ Learning Real-Time A^* ($LRTA^*$)

- builds a map of the environment in the $result[s,a]$ table
 - chooses the “apparently best” move a according to current $H()$
 - updates the cost estimate $H(s)$ for the state s it has just left, using the cost estimate of the target state s'
 - $H(s) := c(s, a, s') + H(s')$
 - “optimism under uncertainty”: untried actions in s are assumed to lead immediately to the goal with the least possible cost $h(s)$
- ⇒ encourages the agent to explore new, possibly promising paths

An $LRTA^*$ agent is guaranteed to find a goal in any finite, safely explorable environment.

Online A^* : $LRTA^*$

$LRTA^*$: General ideas

- Better possible solution: add memory to hill climbing
- Idea: store a “current best estimate” $H(s)$ of the cost to reach the goal from each state that has been visited
 - initially $h(s)$
 - updated as the agent gains experience in the state space

(recall that $h(s)$ is in general “too optimistic”)

⇒ Learning Real-Time A^* ($LRTA^*$)

- builds a map of the environment in the $result[s,a]$ table
- chooses the “apparently best” move a according to current $H()$
- updates the cost estimate $H(s)$ for the state s it has just left, using the cost estimate of the target state s'
 - $H(s) := c(s, a, s') + H(s')$
- “optimism under uncertainty”: untried actions in s are assumed to lead immediately to the goal with the least possible cost $h(s)$

⇒ encourages the agent to explore new, possibly promising paths

An $LRTA^*$ agent is guaranteed to find a goal in any finite, safely explorable environment.

Online A^* : $LRTA^*$

$LRTA^*$: General ideas

- Better possible solution: add memory to hill climbing
 - Idea: store a “current best estimate” $H(s)$ of the cost to reach the goal from each state that has been visited
 - initially $h(s)$
 - updated as the agent gains experience in the state space
- (recall that $h(s)$ is in general “too optimistic”)

⇒ Learning Real-Time A^* ($LRTA^*$)

- builds a map of the environment in the `result[s,a]` table
- chooses the “apparently best” move a according to current $H()$
- updates the cost estimate $H(s)$ for the state s it has just left, using the cost estimate of the target state s'
 - $H(s) := c(s, a, s') + H(s')$
- “optimism under uncertainty”: untried actions in s are assumed to lead immediately to the goal with the least possible cost $h(s)$

⇒ encourages the agent to explore new, possibly promising paths

An $LRTA^*$ agent is guaranteed to find a goal in any finite, safely explorable environment.

Online A^* : $LRTA^*$

$LRTA^*$: General ideas

- Better possible solution: add memory to hill climbing
- Idea: store a “current best estimate” $H(s)$ of the cost to reach the goal from each state that has been visited
 - initially $h(s)$
 - updated as the agent gains experience in the state space

(recall that $h(s)$ is in general “too optimistic”)

⇒ Learning Real-Time A^* ($LRTA^*$)

- builds a map of the environment in the `result[s,a]` table
- chooses the “apparently best” move a according to current $H()$
- updates the cost estimate $H(s)$ for the state s it has just left, using the cost estimate of the target state s'
 - $H(s) := c(s, a, s') + H(s')$
- “optimism under uncertainty”: untried actions in s are assumed to lead immediately to the goal with the least possible cost $h(s)$

⇒ encourages the agent to explore new, possibly promising paths

An $LRTA^*$ agent is guaranteed to find a goal in any finite, safely explorable environment.

Online A^* : $LRTA^*$

$LRTA^*$: General ideas

- Better possible solution: add memory to hill climbing
- Idea: store a “current best estimate” $H(s)$ of the cost to reach the goal from each state that has been visited
 - initially $h(s)$
 - updated as the agent gains experience in the state space

(recall that $h(s)$ is in general “too optimistic”)

⇒ Learning Real-Time A^* ($LRTA^*$)

- builds a map of the environment in the `result[s,a]` table
- chooses the “apparently best” move a according to current $H()$
- updates the cost estimate $H(s)$ for the state s it has just left, using the cost estimate of the target state s'
 - $H(s) := c(s, a, s') + H(s')$
- “**optimism under uncertainty**”: **untried actions in s are assumed to lead immediately to the goal with the least possible cost $h(s)$**

⇒ encourages the agent to explore new, possibly promising paths

An $LRTA^*$ agent is guaranteed to find a goal in any finite, safely explorable environment.

Online A^* : $LRTA^*$

$LRTA^*$: General ideas

- Better possible solution: add memory to hill climbing
- Idea: store a “current best estimate” $H(s)$ of the cost to reach the goal from each state that has been visited
 - initially $h(s)$
 - updated as the agent gains experience in the state space

(recall that $h(s)$ is in general “too optimistic”)

⇒ Learning Real-Time A^* ($LRTA^*$)

- builds a map of the environment in the `result[s,a]` table
- chooses the “apparently best” move a according to current $H()$
- updates the cost estimate $H(s)$ for the state s it has just left, using the cost estimate of the target state s'
 - $H(s) := c(s, a, s') + H(s')$
- “**optimism under uncertainty**”: untried actions in s are assumed to lead immediately to the goal with the least possible cost $h(s)$

⇒ encourages the agent to explore new, possibly promising paths

An $LRTA^*$ agent is guaranteed to find a goal in any finite, safely explorable environment.

function $LRTA^*$ -AGENT(s') **returns** an action

inputs: s' , a percept that identifies the current state

persistent: $result$, a table, indexed by state and action, initially empty

H , a table of cost estimates indexed by state, initially empty

s , a , the previous state and action, initially null

if GOAL-TEST(s') **then return** *stop*

if s' is a new state (not in H) **then** $H[s'] \leftarrow h(s')$

if s is not null

$result[s, a] \leftarrow s'$

$H[s] \leftarrow \min_{b \in \text{ACTIONS}(s)} LRTA^*\text{-COST}(s, b, result[s, b], H)$

$a \leftarrow$ an action b in $\text{ACTIONS}(s')$ that minimizes $LRTA^*\text{-COST}(s', b, result[s', b], H)$

$s \leftarrow s'$

return a

function $LRTA^*\text{-COST}(s, a, s', H)$ **returns** a cost estimate

if s' is undefined **then return** $h(s)$

else return $c(s, a, s') + H[s']$

Example: $LRTA^*$

Five iterations of $LRTA^*$ on a one-dimensional state space

- states labeled with current $H(s)$, arcs labeled with step cost
- shaded state marks the location of the agent,
- updated cost estimates at each iteration are circled

